VISIONIQ / T&M – TRANSPORT OPERATIONS

Historical Intelligence.

Safer engineering decisions.

MASTER ENGINEERING & SYSTEM DESIGN

Engineering requirements, knowledge-base creation, historical search, evidence qualification, technology architecture, testing and human-controlled delivery.

LEARN Capture history and verified outcomes

BRIEF Deliver ticket intelligence with sources

DEFECT / ENHANCEMENT Generate proposed changes and guidance

HUMAN GATE Validate; engineer decides

Version 1.11 | 27 October 2026

Document navigation and reading paths

DOCUMENT CONTENTS • Click a section to jump directly to it

1. Executive design position
2. Organizer requirements and traceability
3. User journeys and low-level behavior
4. Complete product capability model
5. High-level system architecture [PROPOSED]5. High-level system architecture
6. Canonical data design and knowledge relationships
7. Historical matching, reasoning and algorithm design (LLD)
8. Workflows, contracts and integration behavior (LLD)
9. Security, architecture governance and operational reliability
10. Technology architecture: Google Cloud, RAG, OKF, ADK and model providers
11. Detailed non-functional acceptance and experimental design
12. Development plan and definition of done
13. Issues, challenges and risk register

14. Architecture decision records and sign-off
15. Source reconciliation: what stays, what changes
16. Independent validation assessment and release gates
17. Proof-first development and validation roadmap
18. External review consolidation — decisions and rejected assumptions
19. Engineering integration reference and change management

Reading paths: New team members → §§1–5 and §10.8; Knowledge Base engineers → §§6–7; integration developers → §8; model orchestration → §10.9; QA and release engineering → §§8.6, 11 and 17.

This document is designed for team review and implementation planning. It separates organizer requirements, proposed engineering decisions, and validations still needed with real data. Sections 16–19 consolidate external-review implementation corrections and evidence gates; earlier technical chapters are preserved.

1. Executive design position

1.1 Problem, goal and boundaries

A newly assigned defect or enhancement often begins without the relevant organizational memory: earlier incidents, incorrect diagnoses, fixes that regressed, dependencies outside the immediate repository, and operational constraints. This wastes investigation effort and allows repeated failures. The problem is not simply poor keyword search. It is loss of **causal, architectural, time-sensitive and verifiable context** at the moment engineers make decisions.

VisionIQ is a **pipeline-native capability, not a separate developer-facing application**. Engineers continue working in their existing Azure DevOps/Jira work items, repositories, CI pipelines and pull-request review screens. The system runs without requiring the engineer to visit a VisionIQ site. VisionIQ should automatically attach an evidence-backed brief to eligible tickets; guide work for both defects and enhancements; prepare a reviewable defect PR when safe and feasible; validate it against current and previously failed behavior; enforce engineer approval; and preserve the review decision *plus later verified outcome* for future analysis. **A previously known defect is retrieved from reusable knowledge; it is not relearned and reindexed from scratch for every occurrence.**

The required product is **not** an autonomous production fixer, a generic chatbot, a separate ticket-triage UI, a replacement for CI, or a system that declares the historical record infallible. The existing flagship **Impact Agent must display a permitted, evidence-backed historical view** supplied by VisionIQ. The desired behavior is fixed by the organizer; only the Impact Agent-owned integration mechanism and authorization contract need confirmation. We must not invent proprietary endpoints.

1.2 One memorable differentiation

Failure Memory Firewall: a historical incident is useful not only because it can be cited, but because it can warn about and, where executable evidence exists, **test for a failure the current fix might reintroduce**. A strong demonstration shows an unsafe patch pass the narrow new-defect test, fail a recovered historical regression case, and be corrected before an engineer approves the draft PR. This is the hypothesis to prove, not an already achieved result.

Supporting capabilities: **Engineering Time Machine**, **Cross-Project Root Cause Detective**, **Root Cause Evidence Validator**, **Living Source of Truth**, and **Confidence-Aware Matching with Abstention**. These are interfaces over shared historical evidence and should **not** become separate AI products or independent microservices by default.

1.3 Three meanings of completion

Level	What success means	Status
Hackathon win-condition implementation	One sandboxed real defect **and** one enhancement each receive an automatic history-based brief; defect produces evidence-backed PR; engineer reviews it; tests green; no demonstrated touched-path regression; human gate enforced; hotspot sweep and an actual history view in the flagship Impact Agent demonstrated.	**Target, not yet demonstrated**
Working supported application	Robust source sync, source-aware permissions, real query evaluation, versioning, reliable events/retries, monitored quality on a representative set.	Longer MVP-to-beta work
Enterprise product	Many organizations/repos/languages, policy administration, deployments/hardware, recovery, governance and sustained SLOs at scale.	Roadmap; cannot be promised in a hackathon

1.4 Design rules that overrule any vendor or AI suggestion

1. Historical **association is not causation**; an LLM diagnosis is a hypothesis until reproduced or otherwise verified.
2. Semantic/vector similarity **is not a calibrated probability**; confidence scores must be separated by meaning and evaluated.
3. A rule written in Markdown/YAML is **not enforced** unless actual code/tests/policy checks execute it.
4. Passing targeted tests cannot mathematically guarantee a defect is fixed or that no regression exists.
5. Automated code runs in a restricted environment; PR creation is permitted only within explicitly authorized scope; **no auto-merge**.
6. Cross-project intelligence never bypasses source-system authorization.
7. Fast brief and deep patch generation are **different execution paths with different latency budgets**.
8. The published document must distinguish **measured outcomes, design targets, illustrative examples and open assumptions**.
9. **Pipeline-native is mandatory:** default ticket events must initiate analysis; briefs must appear in the original ticket; code/tests must flow through the existing repo/CI/PR policies; a standalone application cannot be a mandatory step.
10. **Learning is cumulative, not repetitive:** reuse deduplicated patterns and only revise knowledge when there is materially new, traceable evidence.
11. **Review is not a verified fix:** distinguish a proposed patch, an approved PR, a deployed change and an observed outcome; lack of incidents is not proof of success.

1.5 Engineering dependencies and integration assumptions

This engineering design describes system boundaries, workflows, interfaces, examples and validation methods. Exact organizational integration details, credentials and benchmark results are determined in the implementation environment.

Non-negotiable organizer behavior: automatic brief for a live defect and enhancement, actual defect PR with green tests and engineer approval, no auto-merge, native ticket/pipeline execution and history visible in the flagship Impact Agent. **Additional product capability:** post-merge observation and selective knowledge improvement are internal background services—not a fifth or sixth required phase for the developer.

2. Organizer requirements and traceability

2.1 Required four-phase lifecycle

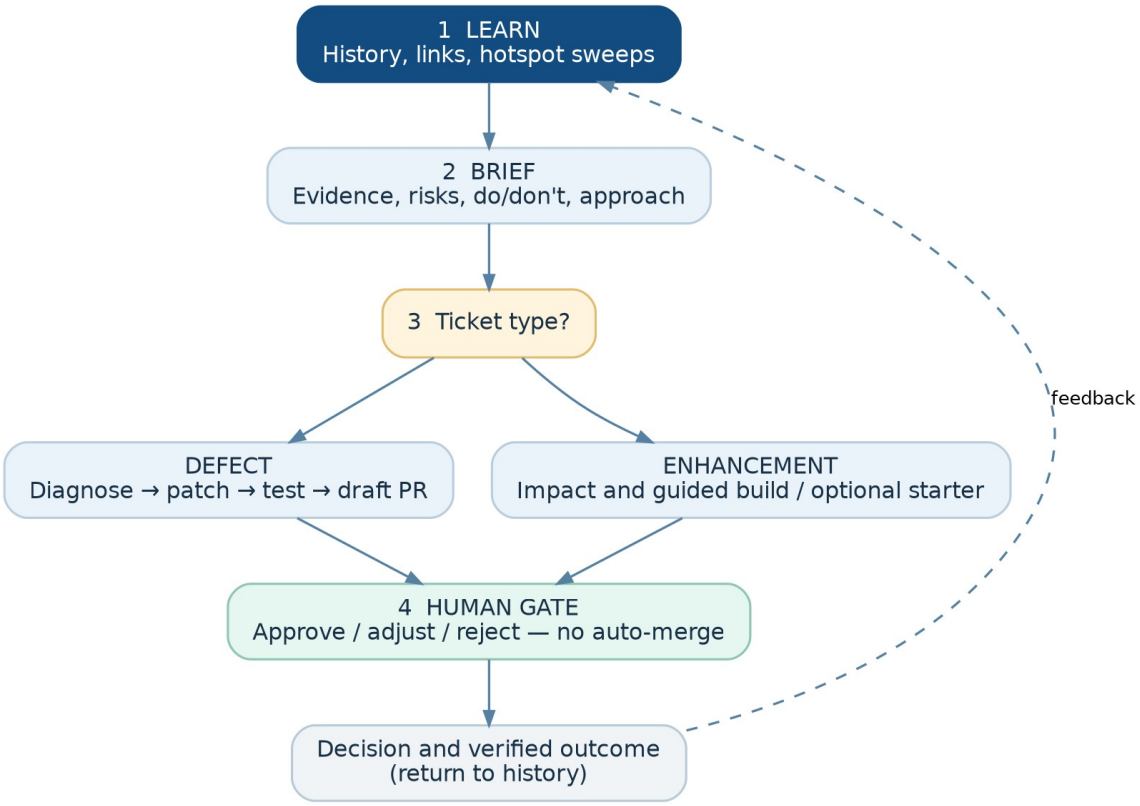


Diagram 1 — Required workflow and evidence feedback.

Delivery interpretation [REQUIRED]: This diagram describes a pipeline embedded in existing work-item and repository systems. VisionIQ may provide an optional internal operations page for health, audit and diagnostics, but **no separate VisionIQ UI is needed to pick up a ticket, read its brief, review its draft PR or approve the change**. The work-item system remains the user entry point and the repository/CI system remains the approval authority.

Learn — always on: mine past defect tickets, incidents, associated fix commits and architecture. Identify repeating patterns and their supported causes; perform a periodic hotspot sweep. A missing/incorrect historical root cause must be labeled missing/unverified rather than silently filled in.

Brief — on every eligible ticket: automatically deliver **risk areas** (modules, services, interfaces), **do/don't guidance** from previous successes/backfires, and a **best approach with its historical evidence**. Both defects and enhancements must receive it. The source links, coverage and uncertainty belong in the brief.

Defect → fix to PR: investigate and propose a root-cause-oriented change in the right layer, include appropriate tests, document the trail **defect → root-cause hypothesis/evidence → risk → change**, create a **reviewable PR** for an authorized repo. If the agent cannot safely produce a patch, it must explicitly stop and escalate; this is a product failure-handling policy, not permission to falsely claim the official PR requirement was met.

Enhancement → guided build: recommend implementation approach, contracts, impact and risks using history; optional starter PR when suitable, not mandatory. The result should meaningfully improve decisions instead of offering generic code templates.

Human gate: engineer approves/modifies/rejects guidance or PR; branch protection, CI and security policy remain outside model control; nothing merges on its own. Every human decision is **recorded** in Learn as an audit event. It does not automatically become a new canonical lesson. Only new, validated or explicitly disputed outcomes update reusable knowledge. After a merge, independently observed build/deployment/incident evidence may revise the historical conclusion.

Win and quality bar: demonstrate a live sandbox defect and enhancement using real historical source data, each with an automatic brief, a defect PR approved by an engineer, green tests, no shown slowdown on the touched path and no reproduced historical regression; record advice correctness. Add the historical view to the existing Impact Agent, subject to actual ownership/interface access.

2.2 Traceability and proof

Organizer requirement	Responsible subsystems	Proof needed
Continuous history + architecture mapping	Source adapters, entity/link resolver, indexes, HIE	Dated versioned evidence and correct code/module links
Periodic hotspot sweep	Scheduled Learn job	Rerunnable sweep with evidence and clear denominators
Automatic brief on **both** ticket kinds	Work-item event receiver, durable workflow, brief publisher	Actual defect and enhancement receive briefs in their original tickets without a manual prompt or separate UI
Default native SDLC integration	Work-item hooks, idempotent queue, source adapters	Configured pickup/assignment event triggers the workflow automatically; missed events are reconciled
Existing CI / PR policy integration	Git adapter, CI status/check publisher, branch protection, reviewer gate	Build/test/performance evidence attached to draft PR; bot cannot bypass required checks or merge
Post-merge observation (product assurance)	Merge/release watcher, outcome tracker	Observe actual merge/release/revert/recurrence and mark outcome as verified, contradicted or unknown
Non-repetitive knowledge improvement	Pattern resolver, revisioned lessons, evidence provenance	Repeat of an existing failure links to one known pattern; only material new evidence revises the canonical lesson
Risk, do/don't, best approach	Retrieval + evidence qualification	Exact citations and unknowns; reviewer evaluation
Defect fix/test/PR	Diagnoser, runner, VCS adapter	Real diff, test logs, PR URL, approval event
Enhancement guidance	Enhancement analyzer	Specific contract impact, approach, validation plan
Human control and learning	Gate/outcomes	Bot cannot merge; action and later result are distinguishable
No old failure or touched-path slowdown	Failure Memory Firewall + paired benchmarks	Executed historical test and repeatable base/patch report
Impact Agent history view	Adapter	Owner-approved integration test or explicit blocker if unavailable

3. User journeys and low-level behavior

3.1 Defect example: payment succeeds, ticket does not print

Illustrative Transport Ops example; not a claim that these records already exist. A defect is raised in the TVM UI project. A historical incident in another repository describes a printer retry problem; a previous patch introduced duplicate printing. The current system's deployed version and printer hardware state may be unknown.

Expected journey:

12. Eligible work-item assignment/state change emits a signed/validated event. Duplicates, old revisions and bot-created updates are filtered.
13. Ticket context is assembled: title, description, comments, repro details, branch/release, tags, environment, user access scope and attached logs where authorized.
14. Hybrid retrieval searches **across authorized projects** using keywords, embeddings, components, dependency edges, code symbols and version context. Do **not** first filter exclusively by the reported project or a guessed root-cause class.
15. HIE ranks potential prior defects, reconstructs explicit and inferred links, checks reverted fixes and conflicting records, and identifies what remains unknown.
16. Ticket receives an evidence bundle with affected services, historical do/don't, likely test scenarios, and appropriately labeled hypotheses.
17. Deep diagnosis attempts the smallest discriminating reproduction and checks competing causes; if no cause is supported, it stops with findings.
18. For an authorized repository, create candidate patch on an isolated branch/worktree; run compilation, relevant tests, historical-regression checks, static/policy controls, and repeated paired performance checks.
19. A failing unsafe candidate is not presented as safe. The agent can produce another candidate within time/diff budgets or stop with the evidence of failure.
20. A passing proposed change becomes a **draft PR** with evidence links, base/patch commits, commands, results, limitations, and reviewer approval controls.
21. Decisions, CI results, later reverts and real verified recurrence outcomes are fed back to history; approval alone is not proof of correctness.

3.2 Enhancement example: change a payment response contract

The assigned enhancement adds new payment status semantics; downstream kiosks, printing and reconciliation systems may depend on old responses.

The system should derive or ask for missing acceptance criteria, map affected contracts and owners, identify historical API-breaking regressions, specify compatibility/migration and versioning advice, enumerate **do** and **don't** with citations, and produce a phased implementation/checklist. It may optionally create a narrow starter PR. It must not silently decide that every change is backward compatible or auto-notify external teams on an unverified dependency guess.

3.3 State transitions, uncertainty and stop conditions

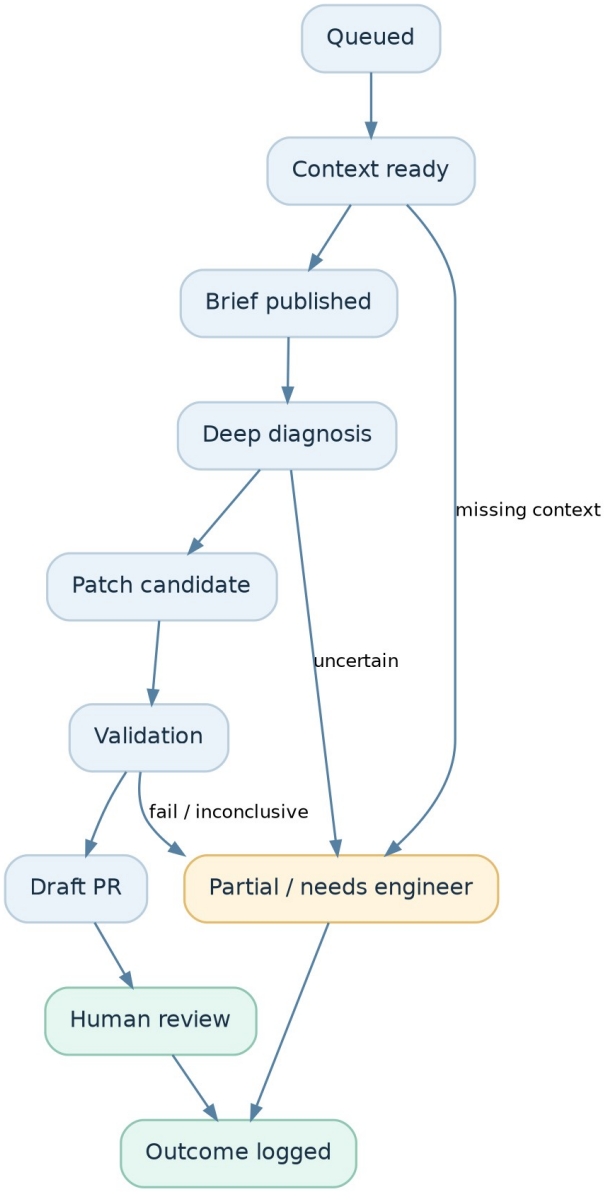


Diagram 2 — Durable per-ticket workflow; each state has timestamp, reason and correlation ID.

Stop rather than fabricate when: identity/scope is unauthorized, required branch or tests unavailable, retrieval has contradictions, root-cause hypothesis cannot be discriminated, model exceeds budget, generated change exceeds allowed files, tests are flaky/irreproducible, performance result is statistically inconclusive, or source content includes possible prompt injection. Distinguish **not-run**, **inconclusive**, **fail**, **pass**.

3.4 Always-on Learn path

Initial controlled backfill obtains ticket revisions, linked commits/PRs/tests, source documentation and architectural dependencies. Scheduled or event-driven incremental ingestion updates artifacts, link status, content revisions and search indexes. A periodic hotspot process reports incident frequency/severity and historical risk trends but should normalize for activity where feasible; a busy code area is not necessarily defective. Source changes do not instantly mean previous hypotheses are false; version-aware revalidation is required.

3.5 Native SDLC integration — no separate developer application [REQUIRED]

User experience: Engineers keep their normal Azure DevOps/Jira ticket, Git branch, CI run and PR review flow. VisionIQ is a **background engineering service** exposing native comments, traceable links, checks and policies in those tools. An optional operator console may be built for support and metrics, but must never be necessary to use the four-phase workflow.

Four connected execution surfaces:

Existing surface	VisionIQ addition	What stays owned by the existing platform
Work item / ticket	Pickup hook automatically starts Brief; publishes attributed risk/do/don't/approach with evidence, freshness, run ID, explicit uncertainty and links to PR/checks	Ticket status, assignment, comments, user access and authoritative work-item history
Repository / pull request	Creates an allowed branch and **draft** PR with cited defect → hypothesis → risk → fix, human-readable diff and test coverage	Code ownership, branching, merge rights, reviewer identities and PR conversation
CI/CD pipeline	Requests/reads relevant build, unit/integration tests, historical regression replay, security/static checks and repeatable touched-path benchmarks; publishes their **actual status**, including *not-run/inconclusive*	Existing CI system, test jobs, deployment approval, required checks and branch policies
Impact Agent history view	Returns authorized scoped historical evidence and risk/dependency findings via an owner-agreed adapter	Flagship impact UX, identity model and integration contract (currently open)

Trigger policy [PROPOSED; configure per source project]: fire when a supported defect/enhancement is *assigned and enters an actionable state* (e.g., In Progress), or on a project-approved pickup transition. Avoid generating a new brief on every comment or bot update. Use a work-item ID + source revision + analysis-policy revision to deduplicate; explicitly support manual reanalysis after material changes. Do not treat 'assigned' and 'In Progress' as necessarily identical in every organization.

Operational split: Learn backfill/indexing/hotspot scans run continuously or on schedule. Ticket pickup triggers the **fast Brief** without waiting for sandbox execution. The defect **deep path** is queued separately to investigate, patch and validate. The enhancement path publishes guidance; it does not automatically require a code PR. A webhook should authenticate, persist/queue an event and acknowledge promptly; heavy reasoning occurs in workers. The precise webhook source/transport and CI integration adapter are environment-dependent decisions.

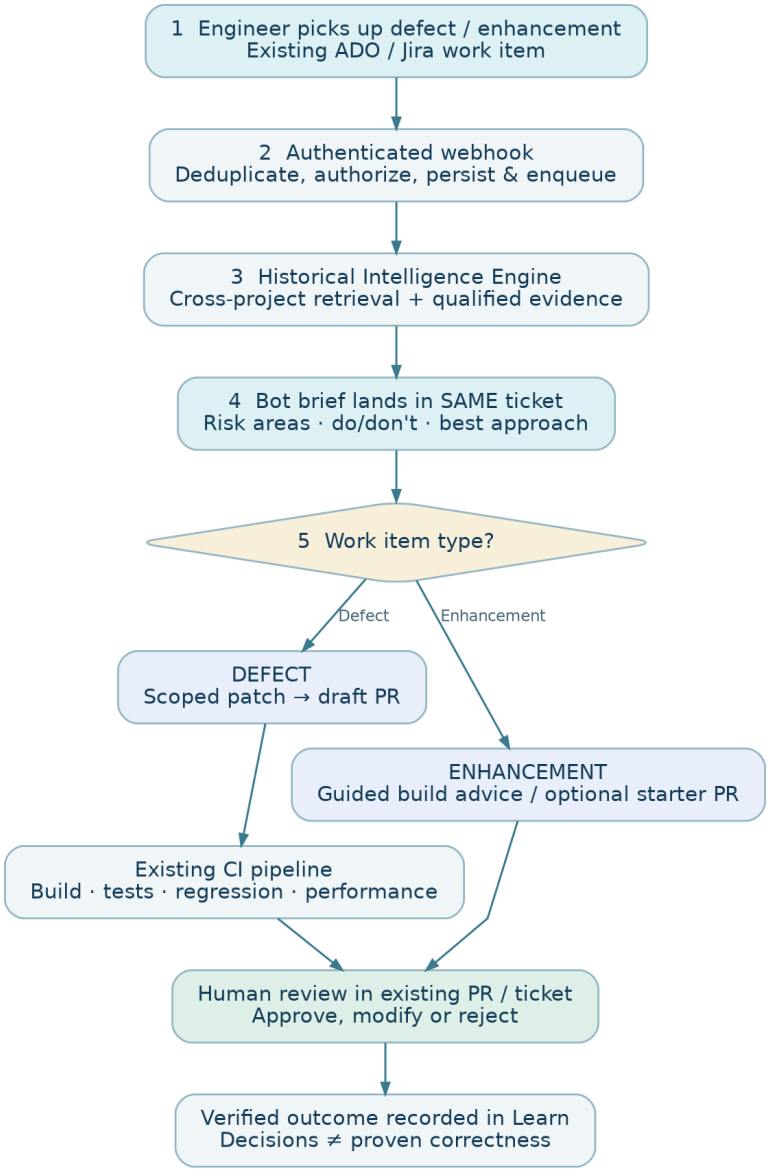


Diagram 3 — Ticket-native workflow; engineer interaction remains in ADO/Jira, Git and CI.

Failure paths: if delivery fails, retain a durable run and retry idempotently; reconcile tickets periodically against source revisions to recover missed webhook deliveries. Publish a partial brief with missing-source warnings when permitted; never write a fabricated success or overwrite developer-authored ticket fields silently. If PR checks fail, show failed/inconclusive checks in the existing PR; do not mark the change validated. A missing Impact Agent contract must remain visibly OPEN rather than being substituted by a separate dashboard.

Acceptance proof: configure two real actionable tickets (one defect, one enhancement), observe both events without visiting a VisionIQ UI, see both briefs in original work items, complete a defect draft PR through existing build/tests and engineer approval, verify the bot cannot merge, inspect feedback provenance, and demonstrate missed-event reconciliation plus duplicate-event suppression.

3.6 Post-Merge Observation — an internal assurance loop [REQUIRED FOR COMPLETE PRODUCT]

Why: a patch can pass CI and be approved while still causing a production incident after deployment. The four official phases end at the engineer's gate, but VisionIQ's **background evidence lifecycle continues**: follow later PR/merge, CI, release, rollback/revert and issue events where accessible. Post-merge observation must never auto-deploy, auto-revert or auto-merge.

Observed event	Recorded state or action	Why it matters
PR approved but not merged	`APPROVED_AWAITING_MERGE`	Approval alone is not a delivered fix
PR merged to a branch	`MERGED_NOT_DEPLOYED` if deployment evidence absent	Merge is not proof the affected production path changed
Release containing the patch deployed	`DEPLOYED_OBSERVING` with SHA, version, environment, start time	Bind outcomes to the actual running version
CI, regression, SLO or incident signal linked to touched path	Append independent evidence; reevaluate hypothesis; notify owner if material	Discover backfires that ticket/PR reviews missed
Revert, rollback or recurrence	Mark outcome `REGRESSED_OR_REVERTED` and supersede earlier positive claims	Convert a backfire into actionable negative knowledge
Defined observation window completes with no reliable telemetry	`UNKNOWN_OR_UNOBSERVED`, **not** `VERIFIED_SUCCESS`	Absence of a reported failure is not proof of correctness
Relevant success evidence + engineering verification	`VERIFIED_EFFECTIVE` with scope and date	Promote supported lessons cautiously

Linking rule: correlating a deployed change and incident requires repository SHA/release map, affected path, time window and cause investigation. Mere temporal proximity is weak evidence. Source permissions, retention, environments and observability quality apply throughout. The watcher uses existing CI/CD and incident platforms; it does not invent a second operations UI.

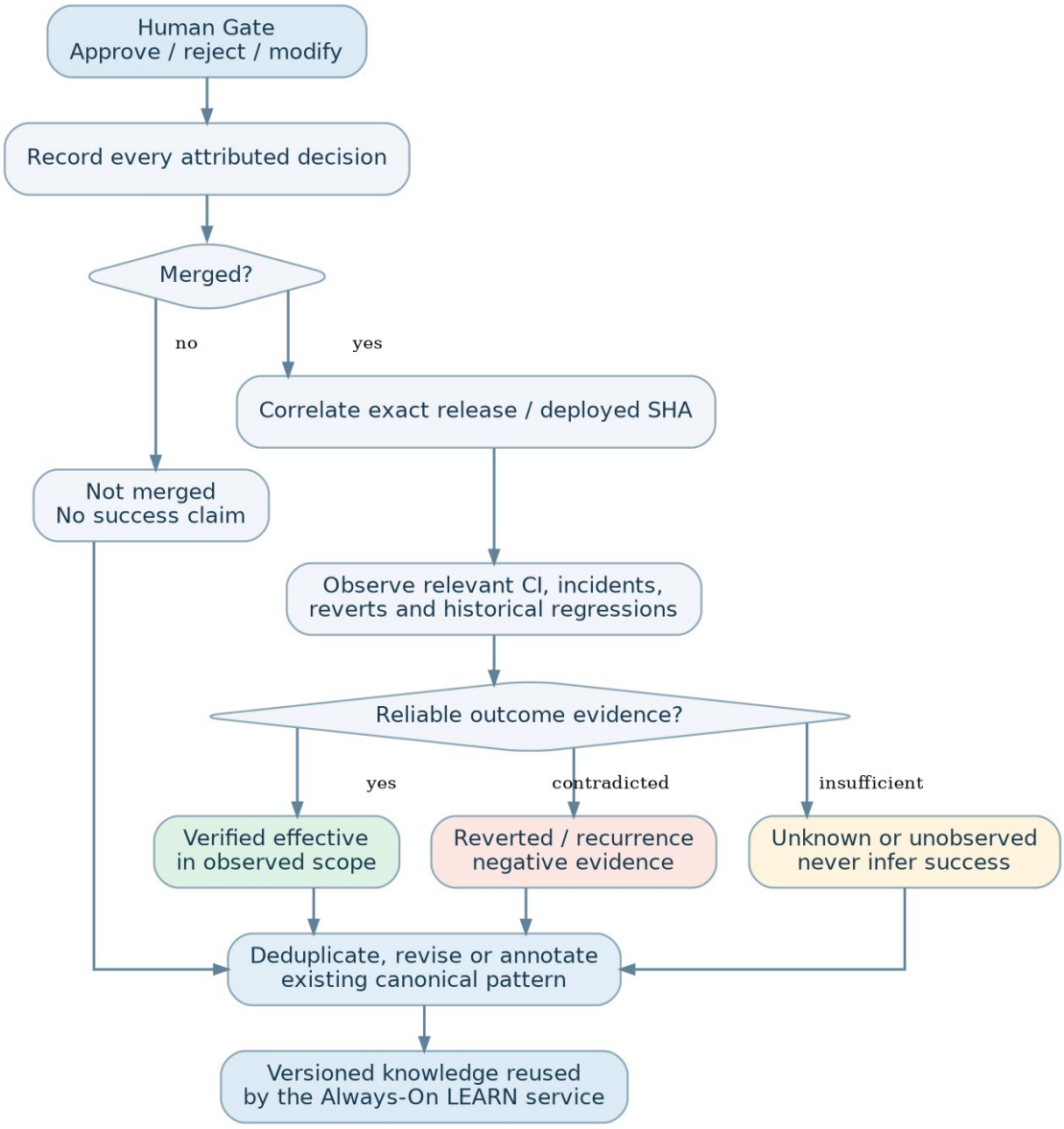


Diagram 4 — Post-merge outcome observation and evidence-qualified learning; not an additional developer-facing phase.

3.7 Continuous Knowledge Improvement — learn new facts, reuse known lessons [REQUIRED]

Distinguish four operations: (1) **RETRIEVE** an existing historical pattern when a similar ticket arrives; (2) **RECORD** this ticket and human decision as new source evidence; (3) **REVISE** a known pattern only when new verified/contradictory facts emerge; (4) **CREATE** a new pattern only when the failure mechanism cannot be explained adequately by existing version-aware patterns. Retrieval must not trigger automatic re-embedding of all similar tickets or another duplicate lesson.

Example: ticket A documents a printer retry causing duplicate printing. A new ticket B with the same failure mechanism should retrieve A's curated lesson and link B as another observation. When B confirms the same cause, the lesson gains evidence and a recurrence count—not a duplicate 'root cause'. If B involves a different deployment version or contradicting test, the same lesson is revised, branched by scope/version, or marked disputed. Reverts can demote previously accepted advice.

Rules: canonical pattern identity is stable and source-qualified; each lesson has a version, affected components/releases, evidence links, verification state, owner, `first_seen`/`last_evidence_at`, supersedes relationships, and separate observation count. Use idempotent source event IDs/content hashes; re-index only

changed objects and impacted relationships. Periodically review low-quality/aged lessons; never treat AI preference, similar wording, engineer approval or a passing CI run as independent root-cause truth.

Acceptance demonstration: present two tickets with a shared verified failure mechanism, demonstrate that only one reusable pattern/lesson exists, and show the second ticket adds provenance without duplicating the lesson. Next, introduce a later reverted fix and demonstrate versioned correction, retrieval of the negative lesson and relevant historical regression checks.

4. Complete product capability model

Rather than 31 independent features, the product has **eight grouped capabilities**, each housing corresponding prior blueprint features. This grouping removes duplication, **not** required behavior.

Capability group	Responsibilities	Prior feature IDs	Scope
A. Historical acquisition and provenance	Continuous sync, ACL, canonical artifact revisions, defect–commit–PR–test linkage	F01, F03, F05, F18	P0
B. Matching and engineering context	Hybrid retrieval, code symbols, dependency graph, deployed-version relevance, uncertainty	F04, F07, F17, F23, F31	P0 core; advanced P1
C. Historical reasoning and Source of Truth	Evidence validator, contradictions, root-cause hypotheses, history timeline, rule freshness	F06, F20, F21, F25	P0 validator; P1 UI/replay
D. Ticket intelligence and native SDLC integration	Default ticket trigger, in-ticket brief, native PR/CI checks, outcome events, Impact Agent history adapter	F02, F08, F09, F16	P0
E. Fix and safe code verification	Defect diagnosis, minimal patch, isolated tests, regression/performance verification, draft PR	F10–F13, F22	P0 + signature proof
F. Enhancement intelligence	Contract impact, cross-project dependency and guided implementation, optional starter	F09, F19, F26	P0 guidance; P1 extras
G. Continuous risk and outcomes	Scheduled hotspots, defect fingerprints, feedback quality, operational evaluation	F15, F24, F27, F28, F30	P0 sweeps/metrics; P2 forecasting
H. Experimentation and comparison	Alternative fixes and repeatable comparative evidence	F29	P2

4.1 Historical Intelligence Engine (HIE) — central product service

HIE accepts `TicketContext` and `RequesterScope`, reads an immutable source/index snapshot, and returns an `EvidenceBundle`: ranked prior items; exact proof links; component/dependency impact; supported, disputed or unverified hypotheses; temporal context; historical do/don't; recommendations and explicit unknowns. It is **shared** by all four phases, not a downstream search widget. HIE does not itself merge code, grant access or turn similarity into causality.

Component layering: source adapters → canonical evidence and provenance → relationship resolver → keyword/vector/structural retrieval → contextual reranker → evidence qualification → typed evidence bundle → brief/diagnosis/enhancement/gate consumers.

Knowledge-base creation contract: Section 6.6–6.9 specifies the full lifecycle: authorized backfill, incremental sync, canonical revisions, structural extraction, link qualification, multi-index publication, curated lessons, freshness/ACL reconciliation and outcome-driven updates. Vector search is one *consumer* of this knowledge base, not the knowledge base itself.

4.2 Engineering Time Machine and Source of Truth

For a component, present a dated line of defect, change, deployment, regression, revert, test and decision events. Every relationship must be tagged **explicit / inferred / contradicted / unknown**. If design documents conflict with current code or release artifacts, show drift and propose a reviewed correction; do **not** auto-rewrite authoritative Confluence pages.

4.3 Cross-project detective

Map imports, API contracts, shared packages, message schemas, configuration and deployed service calls where known. Cross-project retrieval is **scope-aware** and should include both candidate hits and relevant negatives. Report a **probable dependency path** with evidence; don't assert the reported team's repository is the cause. Deep analysis may inspect code only in repos authorized for that run.

4.4 Root Cause Evidence Validator and confidence-aware matching

Maintain **separate** values/labels for:

- **Retrieval relevance:** ranked degree of ticket/code/history relationship (not probability of same cause).
- **Evidence support:** explicit links, test outcomes, actual change diffs, timeliness, contradictory signals.
- **Root-cause status:** unverified → supported / disputed → reproduced where possible.
- **Patch-validation status:** commands executed; pass/fail/inconclusive/not-run, with environment and scope.

A 0.93 cosine score or hand-weighted formula cannot be displayed as "93% chance correct." Model verbal confidence and engineer approval are not objective evidence. Calibrate probabilistic outputs only when sufficiently labeled, representative cases and held-out validation exist; otherwise show qualitative categories and matching reasons. False positives, false negatives, absent history, deployment drift and model disagreement are explicit challenge cases.

4.5 Failure Memory Firewall

For changed symbols/services: retrieve prior backfires and regression tests; attempt to map historical failure conditions to the current branch; run recoverable existing tests; generate proposed additional tests that an engineer can review; compare before/after behavior under reproducible fixtures; annotate coverage gaps. Historical prose alone is a **test idea**, not a verified regression check. Performance checks compare identical touched paths and workloads with repeat runs and variability, not a fragile universal ">1% slower means fail" rule.

4.5.1 Historical regression-test discovery and evidence qualification

Problem: historical defects do not always have a test tagged `regression`; test-to-defect relationships may be incomplete or inferred. The Failure Memory Firewall must search across three ranked sources **without claiming that a matching name proves coverage**.

Evidence tier	Retrieval rule	Verification condition
1 — Explicit	Ticket/PR/test-case association, CI run linked to the work item, reviewed regression catalog entry	Resolve immutable test identifier, source SHA, relevant failure behavior and runnable command
2 — Historically supported	Tests added/modified in a linked fix PR, previously failing CI jobs, change-linked test files	Review test assertion and affected behavior; store supporting source references and version
3 — Inferred candidates	Dependency/coverage map, touched symbols, co-change history, semantic test-description matching	Label INFERRED, require verification before treating as historical regression proof

Result contract: each candidate has `test_id`, `repo_id`, `test_source_sha`, `runner_adapter`, `discovery_method`, `confidence_label` (not probability), `expected_failure_signature`, `availability`, `evidence_refs`, and `last_verified_at`. Classify `AVAILABLE`, `PARTIAL`, `UNAVAILABLE`, or `STALE`. Deleted tests can be inspected from Git history, but a historical test cannot be assumed runnable against a newer API or dependency graph. An absent test leads to a proposed new characterization/regression test and explicit `HISTORICAL_TEST_NOT_FOUND`, never an invented green pass.

Proof: for a labeled sample, report regression-test Recall@K, precision of verified associations, runnable fraction and coverage gaps. In the official demo, select a defect with at least one executable current or recovered historical regression fixture, and disclose other untested paths.

4.6 Enforceable architecture guardrails (adopted from Whitepaper, corrected)

Repo-local versioned rules in YAML/Markdown or a suitable knowledge format can express architectural constraints, e.g., allowed dependency directions, prohibited API changes, protected directories, authorization requirements, target tests. **An executable validator** enforces machine-checkable rules. Free-text conventions are advisory until converted into enforceable tests/static checks. Rule updates require owned reviews; never accept model-authored edits automatically.

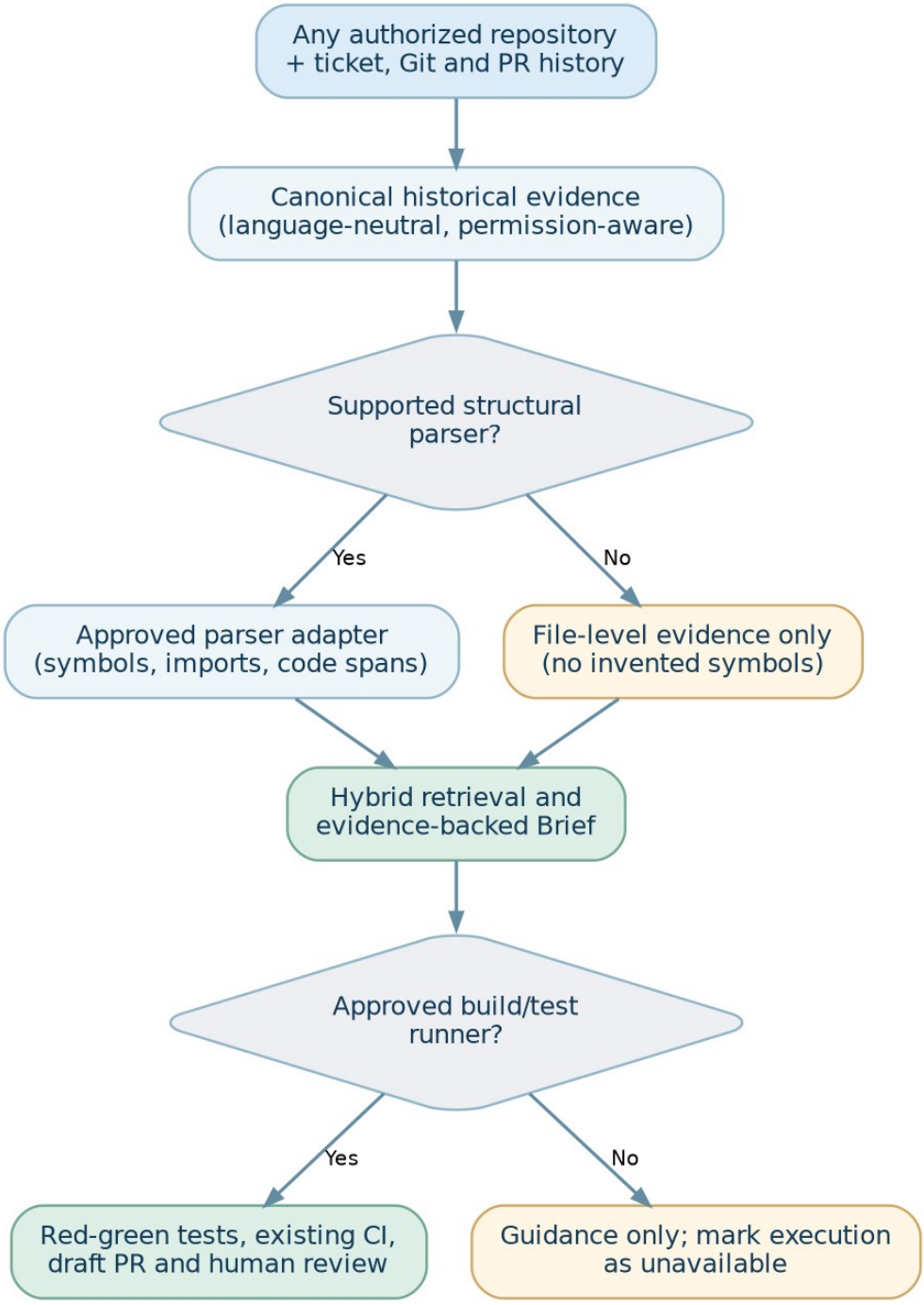
4.7 Language-neutral structural code intelligence [CORE DESIGN]

VisionIQ is repository- and language-agnostic at the platform boundary. It can ingest versioned evidence and produce cited historical briefs from repositories regardless of implementation language. Structural interpretation is performed through a Code Intelligence Adapter Registry, not a .NET-specific compiler service or a single mandatory parser. Use Tree-sitter grammars, supported language servers, build-system metadata, or other reviewed parsers only where the adapter is tested. A parser detects symbols and relationships; it does not prove runtime causality, complete call graphs or a correct root cause. For languages without an approved structural adapter, preserve file-level Git paths, metadata, diffs, ownership and text evidence. Mark symbol_analysis=UNAVAILABLE rather than producing invented method names or blast-radius claims. The Fix-to-PR path separately requires a supported build/test adapter and repeatable sandbox commands; retrieval and Brief must remain usable even when patch execution is unavailable. A polyglot repository can have multiple per-language adapters and a cross-language dependency layer for API contracts/configuration. Never assume all languages are universally supported.

4.7.1 Language-neutral adapter responsibilities and graceful degradation

Adapter / service	Language-neutral input and output	If unsupported
Repository inventory	Repo ID, commit SHA, paths, extension/build manifests, service metadata	Still ingest and index raw history
Structural parser	source_ref, language, symbol_id, span, imports/calls, parser version, extraction quality	Emit UNAVAILABLE; preserve file-level evidence
Dependency resolver	Typed, cited links among symbols, repositories, endpoints, contracts and runtime services	Surface unverified candidate relationships; do not invent a blast radius
Patch and test runner	Toolchain image, approved commands, expected test IDs, resource limits and output artifacts	No autonomous patch/green-test claim; guide engineer and request an adapter
Evidence publisher	Source citations, version/time context, parser/toolchain status, ACL scope	Brief still cites tickets/commits with explicit limitations

Supported-language policy: Detecting a language is not the same as supporting its parser or test runner. An adapter becomes SUPPORTED only after passing symbol mapping, clean checkout, build/test, injected defect, isolation and reproducibility tests on an authorized repository. Publish adapter capability and parser/toolchain versions in each run's audit record.



Architecture supplement — language-neutral code intelligence and safe fallback.

4.8 Post-merge assurance and non-repetitive intelligence maintenance

A **Release/Outcome Observer** correlates reviewed patches to actual deployments and subsequent evidence. A **Pattern Consolidator** distinguishes new observations from new knowledge, deduplicates recurring defect mechanisms, keeps older conclusions traceable, and promotes, disputes or supersedes lessons based on verified results. Both are background functions supporting **Learn**; they do not add extra ticket stages or a separate interface. Only warranted changes rebuild affected indexes. Full enterprise telemetry integration is a roadmap extension; the hackathon can prove the watcher using real merge/CI/revert events from the sandbox.

4.9 Optional advanced capabilities

Selective `git bisect` is useful **only** when a stable reproducing test and executable historical revisions exist; it is not a universal root-cause locator. SonarQube can add static findings/coverage context if available, but no fixed “+10% confidence” uplift is justified. Runtime APM, deployment events, Slack notifications, broad Jira/Confluence connectors, Decision Replay visualizations, forecasting and alternative-fix comparison are valuable **after** the core proof, not first-week dependencies.

---

5. High-level system architecture

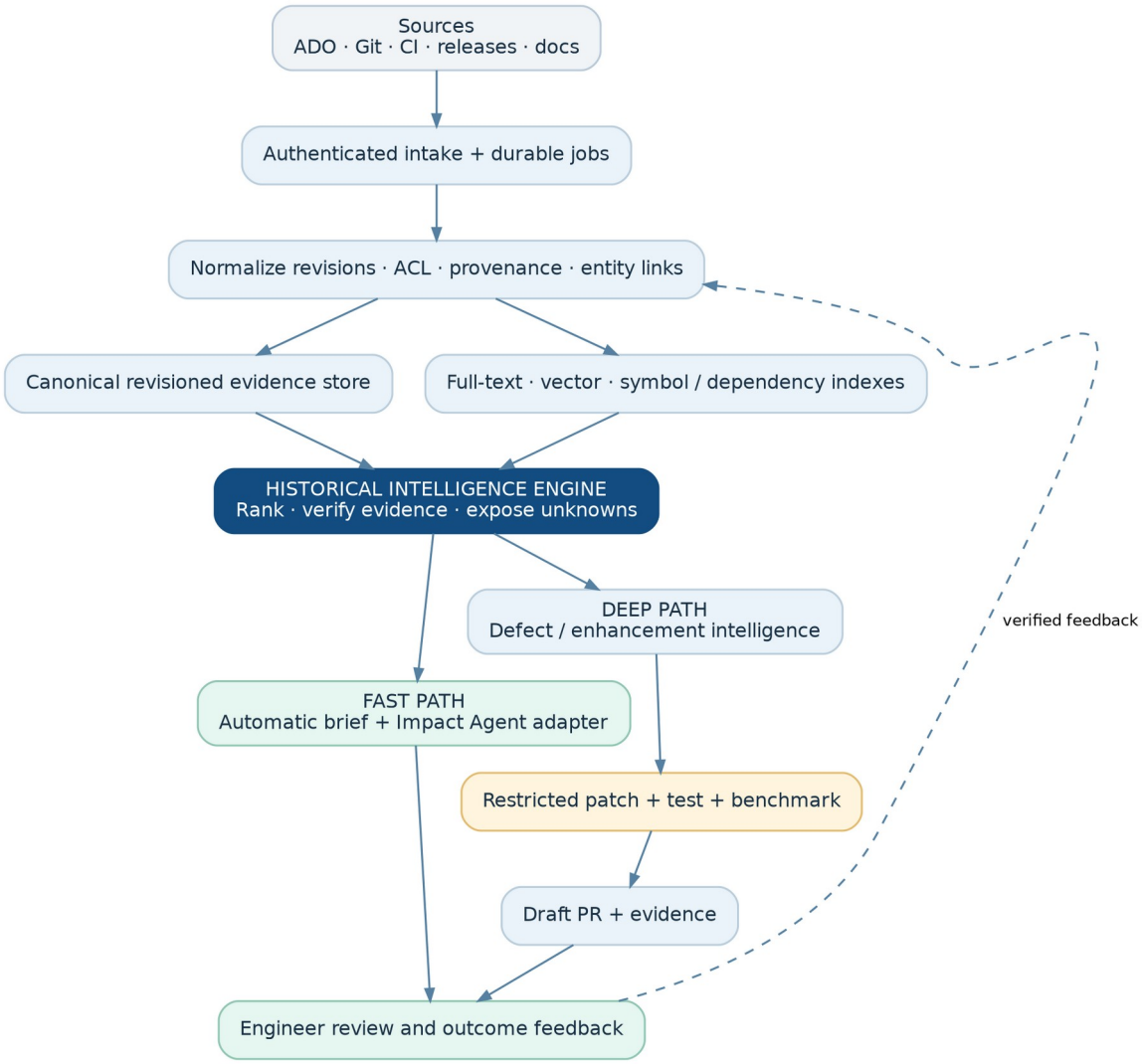


Diagram 5 — Logical architecture, not a promise that every box needs a microservice.

Deployment principle: the blocks above are background services and adapters, **not a standalone user application**. The engineer-facing endpoints are the existing ticket, PR and CI interfaces. Integrations must be event-driven and durable; a manually opened dashboard is not the workflow trigger.

5.1 Recommended physical MVP topology

Default recommendation: modular Python API/worker codebase, a relational source of truth (candidate PostgreSQL), full-text plus vector index (candidate pgvector), a durable job mechanism, approved LLM via provider adapter, isolated Git worktrees/CI jobs, and **no mandatory VisionIQ UI**. An optional internal operations page may be added only for diagnostics, administration or evaluation. Cloud Run can host API/worker depending on access; choose a constrained **runner** separately. Existing Azure DevOps remains the ticket and PR authority. This minimizes operational sprawl while preserving service boundaries in code.

Not locked: BigQuery + Vertex Vector Search + Dataflow, Qdrant + SQL, Google ADK, or alternative cloud components may be justified **by measured corpus size, security model, cost, platform constraints and demonstrated retrieval quality**. At 10,000 tickets, expensive distributed ETL is not automatically necessary. The P4 design's cloud stack is a useful deployment option, **not** a validated minimal architecture.

5.2 Subsystem contracts

Module	Inputs	Outputs	Must not do
Source adapters	Scope, cursor, credentials ref, source events	Canonical artifact revisions, ACL metadata	Assume company-wide universal access
Link and version resolver	Work-item links, commits, PRs, CI, release	Qualified edges, effective versions	Equate any commit reference with true root cause
Indexer and hybrid retriever	Canonical revisions and query context	Candidate histories and match evidence	Hard-filter cross-project candidates prematurely
HIE	Ticket, actor scope, snapshot	Typed EvidenceBundle with citations	Present guessed probabilities as truth
Brief publisher	EvidenceBundle + ticket revision	Structured native ticket comment, run ID, evidence links and status	Rewrite title/description by default
CI / PR adapter	Pinned SHAs, approved pipeline and validation artifacts	Draft PR, status checks, logs, approval events	Replace or bypass branch protection and existing CI authority
Diagnosis/enhancement workflows	Evidence and authorized checkout	Hypotheses, change plan or guidance	Continue past unsupported stop gates
Code execution controller	Whitelisted repo/SHA, patch, test recipe	Hash-linked logs/results	Access production credentials, merge PR
Human gate/outcome handler	PR state, reviewer feedback, CI + later outcome	Audit trail and new labeled evidence	Equate approval with correctness
Impact Agent adapter	Approved historical context contract	Scoped history payload	Invent proprietary endpoints

5.3 Fast path versus deep path

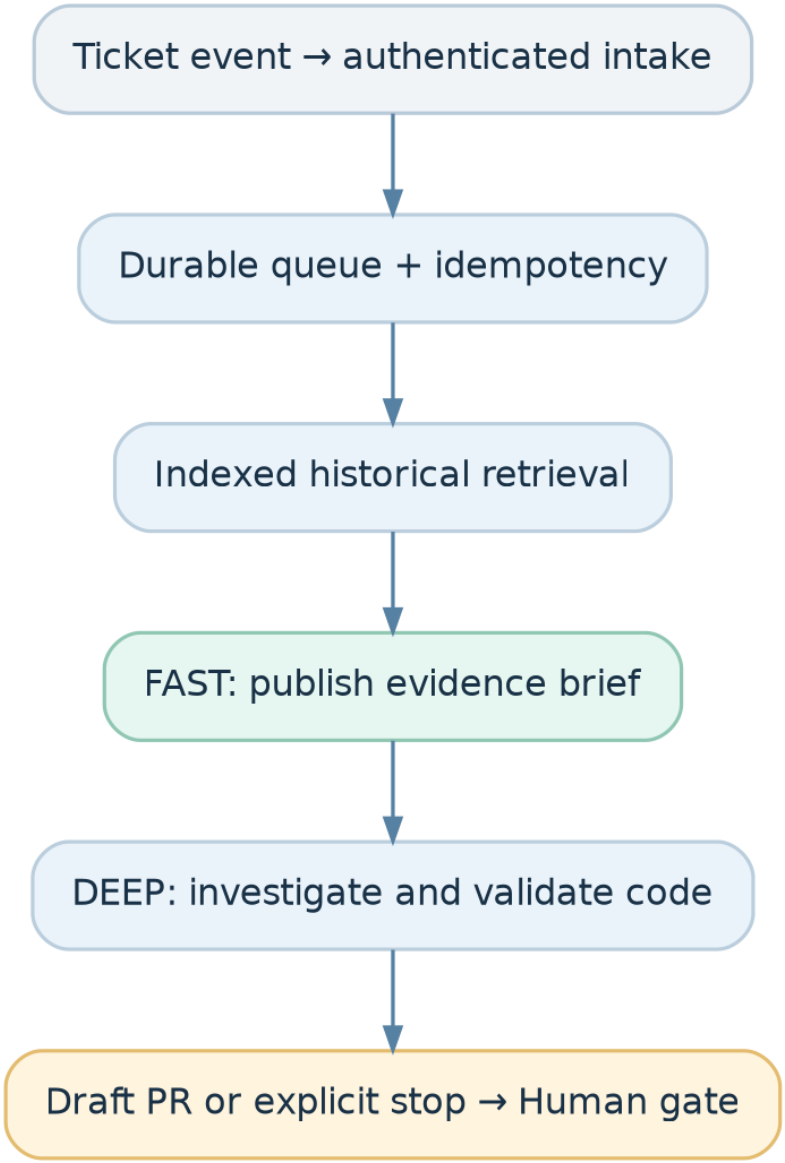


Diagram 6 — A fast brief is not delayed until code generation, CI or benchmarking completes.

6. Canonical data design and knowledge relationships

6.1 Stable identities, revisions and access

Every canonical artifact contains `source_system`, `organization`, `project`, `external_id`, `source_uri`, `source_revision`, `observed_at`, `source_updated_at`, `ingestion_run_id`, content hash and `access_scope`. A content-addressed snapshot enables reproducible analysis without pretending links or rules remain permanently valid. Preserve data provenance and field-level sensitivity; secrets do not belong in model contexts.

6.2 Logical entities

Entity	Important fields and invariants
`connection`	Source type, owner, allowed projects/scopes, secret reference, cursor, enabled/revoked
`artifact_revision`	Canonical identity, type, source revision, body/metadata hash, updated time, URI, ACL; append-only versions
`component`	Repo, branch or release range, path, symbol/contract, ownership; distinct across repos
`evidence_edge`	From/to identities, kind (`fixes`,`mentions`,`reverted_by`,`test_for`,`depends_on`,`deployed_as`), explicit/inferred/verified label, evidence URI, effective period
`index_entry`	Artifact revision, embedding model/version, lexical tokens, structural symbols, access scope, freshness
`analysis_run`	Ticket revision, actor, snapshot IDs, policy/model/prompt versions, state, started/completed, costs, errors
`match_candidate`	Rank, signal contributions, source refs, contradictions, relevance label, not fake probability
`cause_hypothesis`	Claimed cause, support/dispute, evidence IDs, discriminating test, investigation status
`brief_revision`	Structured risks/do-don't/approach/evidence coverage, ticket comment ref, regeneration history
`patch_candidate`	Base SHA, patch SHA, allowed files, revision, validator outcomes
`execution_result`	Build/test/perf recipe, runner image digest, base/patch SHA, attempts, raw logs URI, status
`review_outcome`	Human decision, edits, CI result, later revert/recurrence, verification date; separate approval from correctness
`pattern` / `lesson_revision`	Stable fingerprint, impacted components/version range, dedup key, hypothesis, curated do/don't, evidence set, disputed/superseded status, owned review
`pattern_observation`	Unique source artifact/event ref, pattern ID, similarity/rationale, labeled outcome; multiple ticket observations may share one pattern
`deployment_observation`	PR and patch SHA, release/environment, observed version, observation window, linked incidents/reverts, evidence strength and final status

6.3 Evidence relationship semantics

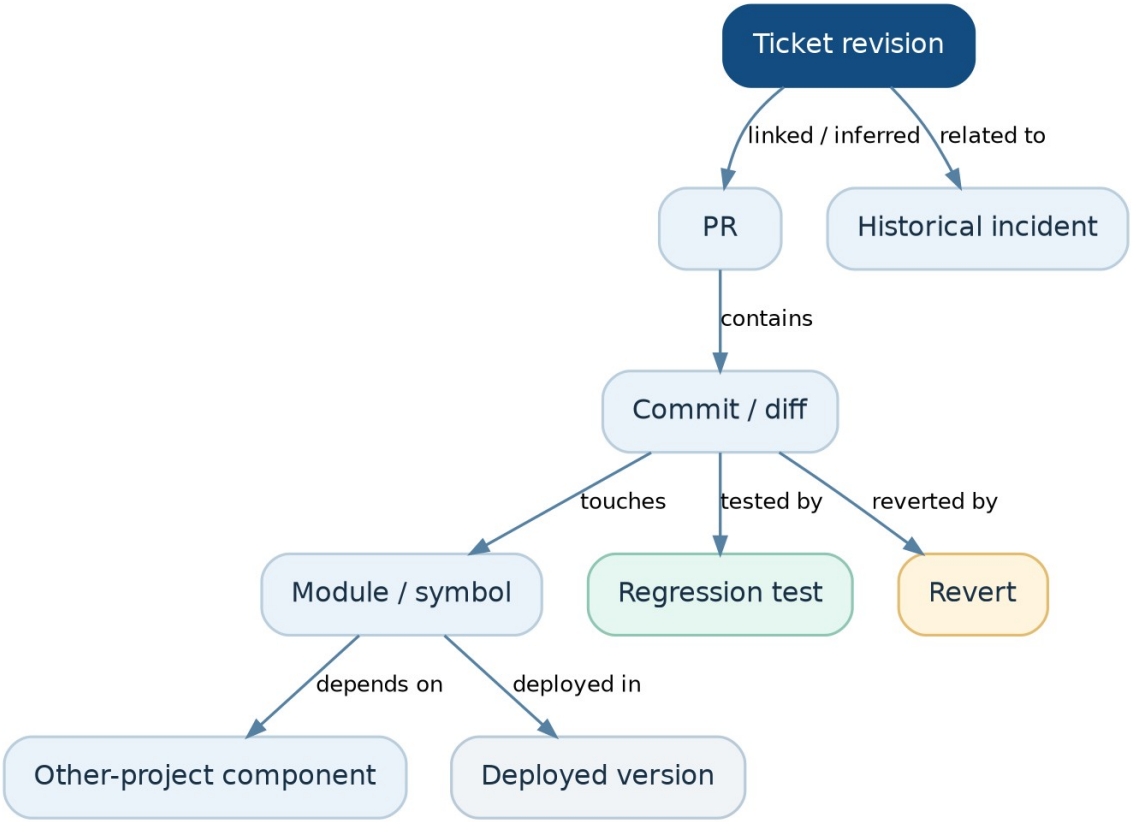


Diagram 7 — Edges are not automatically causal; every edge has source, time and verification status.

6.4 Data-linkage policy

Prefer native ADO/PR links and precise repository commit history; detect ticket IDs in messages as **candidate evidence only**; use time, touched files and tests as secondary signals. A merged commit before ticket closure does not prove it fixed the issue. Track reverted patches, fix-forward commits, cherry-picks, squash merges and multiple defects sharing a commit. If linkage is sparse, continue with reduced coverage and explicit uncertainty; **do not automatically abandon the entire project on an arbitrary 85% cutoff**.

6.5 Storage decisions and portability

A transactional store should own audit states, artifacts and revisions. A retrieval index is derived and rebuildable; never treat a vector DB as authoritative truth. Graph edges can initially live in SQL, with graph infrastructure added only if real workload shows benefit. Index version, embedding model ID, source ACL, deletion propagation and retention must be specified. Historical source data must be retrievable for citations subject to current authorization.

6.6 Knowledge Base Creation — the core LEARN pipeline [REQUIRED]

Purpose. The knowledge base is not simply a vector collection containing ticket descriptions. It is a continuously maintained, **permission-aware engineering memory** that connects historical defects to source code, changes, incidents, deployments, decisions, tests and actual outcomes. The **source systems remain authoritative**; VisionIQ holds attributed, versioned copies and derived interpretations. A retrieved result must be explainable and traceable back to an authorized source.

The first implementation should ingest two or three connected repositories and their actual histories end-to-end. The same adapter and provenance contracts must work when the corpus grows to 10,000+ tickets across additional projects.

Stage	Processing and artifacts	Required controls / fallback
1. Register sources	Discover permitted Azure DevOps projects, Git repositories, incident records, PRs, tests and optional Confluence/CI data. Record source identifiers, owner, credential reference and per-project scope.	Test each connection and least-privilege permissions. An unavailable source is marked *not synchronized*, never silently assumed empty.
2. Historical backfill	Page through tickets, revisions, comments, linked PRs, commits, merge events, changed files and test references; retain original source URI and revision.	Resumable cursors/checkpoints, rate-limit backoff, bounded batches, duplicate-safe upserts and failed-item quarantine. Avoid a single unbounded 10,000-record model call.
3. Incremental updates	Use work-item/repo events when supported, plus periodic reconciliation for missed or changed items, reverts, permission changes and deletions.	Event IDs and source-revision hashes enforce idempotency; track ingestion lag and gaps; repair from the last valid cursor.
4. Normalize and preserve	Create canonical `artifact_revision` records with source identity, timestamp, branch/release and ACL. Store permitted raw snapshots in controlled storage; redact or restrict sensitive material before indexing or model use.	Preserve source-versus-derived fields, immutable revision history and an explicit unavailable/redacted state.
5. Extract engineering context	Deterministically parse explicit work-item links, PR work items, commit associations, file paths, changed symbols, test names and package/API imports. AI may propose additional symptom, root-cause or decision labels.	Treat AI output as **candidate interpretation**, not canonical truth. Store extraction method, parser/model version, evidence span and validation status.
6. Resolve relationships	Link tickets ↔ PRs ↔ commits ↔ files/symbols ↔ tests ↔ incidents ↔ deployed versions. Build cross-project edges from confirmed references, contract usage and declared dependencies.	Edges record `explicit`, `inferred`, `verified`, `disputed` or `unknown`; don't equate time proximity with cause. Support one-to-many fixes and reverts.
7. Build search indexes	Produce lexical search documents, semantic chunks/embeddings, symbol-path lookup, component/dependency adjacency and metadata on evidence freshness.	Index all chunks with their revision, project ACL, source URI, model version and effective dates. Changing embedding model triggers controlled reindexing.
8. Qualify knowledge	Detect conflicting resolutions, stale design rules, superseded fixes and reverted PRs. Promote reviewed patterns into versioned curated lessons or OKF-style Markdown/YAML if chosen.	Label lessons `candidate`, `reviewed`, `disputed` or `retired`. Owner approval is required for enforceable rules; never auto-edit Confluence or authoritative source material.
9. Publish snapshots	Atomically make a consistent index/evidence snapshot available to HIE; record counts, checkpoint, version and age.	Queries use a known snapshot. Partial imports cannot masquerade as a complete up-to-date knowledge base.
10. Learn from outcomes	After the Human Gate, record engineer feedback	Approval is **not proof of**

Stage	Processing and artifacts	Required controls / fallback
	separately from verified test results, later reverts, recurring incidents and post-deployment observations. Recompute affected links/patterns and scheduled hotspots.	**correctness**. New observations can downgrade or dispute an older lesson; preserve prior evidence and audit history.

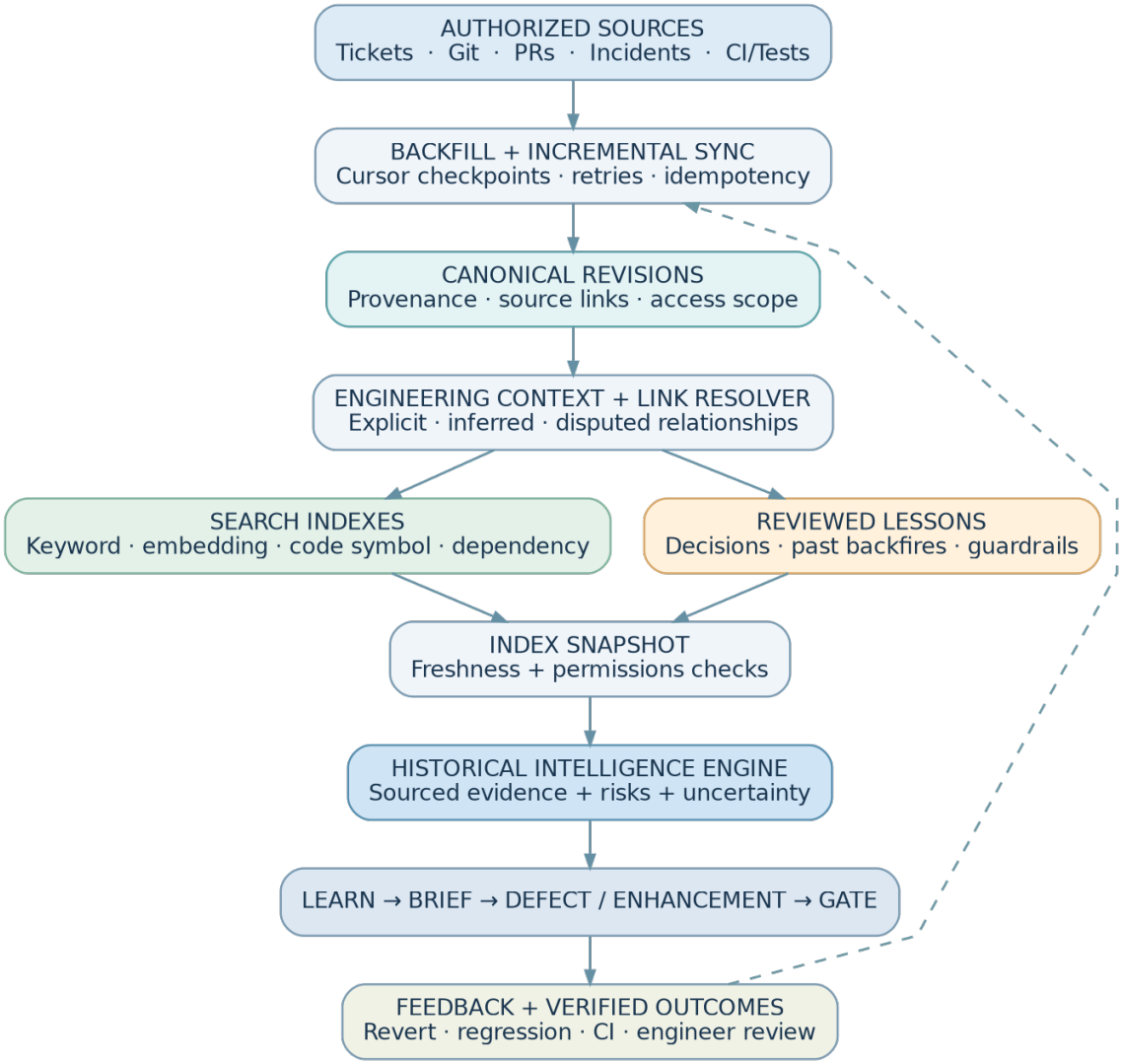


Diagram 8 — Knowledge-base creation, indexing and continuous maintenance. Retrieval is downstream of trustworthy ingestion and evidence linkage.

6.7 Knowledge objects, chunking and relationship quality [REQUIRED]

Three layers, with different authority:

22. **Source archive:** Exact permitted versions of tickets, discussions, incident reports, Git objects, CI logs and design documents. These are the historical observations, with canonical links and source ACLs.
23. **Derived evidence graph and indexes:** Reproducible relationship edges, code/component identities, document chunks, embeddings and search metadata. These are interpretations and retrieval aids, **not independent facts**.
24. **Curated engineering memory:** Reviewed root-cause narratives, do/don't patterns, known unsafe fixes, architectural decisions, reusable regression recipes and versioned guardrails. A curator or designated engineer owns review and retirement.

Chunking policy: Use artifact-aware units instead of uniform whole-ticket embeddings: ticket description and resolution as separate chunks, discussions with author/time context, PR overview and review comments, code diffs grouped by file/function, design documents by heading and incident timelines by event. Preserve

`parent_artifact_id`, `revision_id`, `chunk_type`, `location`, `embedding_version` and `access_scope`. Do not embed secrets, arbitrary CI output or entire repositories without filtering and size limits.

Link quality policy: Prefer native ADO work-item/PR references, Git metadata and explicit test links; allow commit-message and code proximity only as secondary signals. Store each inferred relationship with rationale and contrary evidence. A single ticket may have multiple fixes, projects, regressions or unrelated commits. When no reliable link exists, retain the ticket as searchable history but state **'fix commit unknown'** rather than fabricating a chain.

Illustrative evidence object (not a committed API schema):

```
{
  "artifact_id": "ado/orgA/projectA/workitem/2451",
  "revision": "17",
  "source_uri": "https://example.invalid/ado/workitem/2451",
  "access_scope": ["projectA-engineering"],
  "observed_at": "2026-10-26T09:15:00Z",
  "linked_changes": [
    {
      "target": "git/projectB/repo-printer/commit/abc123",
      "relationship": "potential_fix",
      "basis": "PR linked to work item; affected printer-retry symbol",
      "evidence_status": "inferred",
      "verified_by": null
    }
  ],
  "root_cause": {"status": "unverified", "hypothesis": "retry-state handling"},
  "index_snapshot_id": "snapshot-2026-10-26-01"
}
```

The example identifier and URL are illustrative; actual source links must be directly resolvable and permission-checked. JSON content is not proof that the stated relationship is correct.

6.8 Knowledge lifecycle, refresh and governance [REQUIRED]

- **Freshness:** New ticket/PR events trigger incremental sync, while scheduled reconciliation checks omissions. Publish `last_successful_sync`, source watermark, per-project completeness and index snapshot age to operators and to the Brief response.
- **Corrections, reverts and contradictions:** Reevaluate relevant lessons and matches when a linked PR is reverted, an incident is reopened, a test starts failing or a source document is superseded. Do not overwrite the old lesson; version it and mark prior conclusions superseded or disputed.
- **Authorization and removal:** Apply source ACL changes to canonical records, chunks, embeddings, relationship traversal, citations and cached briefs. On revocation/deletion, invalidate derived indexes and cached answers; do not continue serving historical snippets based on earlier access.
- **Controlled publication:** AI-discovered patterns remain candidates. A human-reviewed lesson may include owner, scope, affected release ranges, positive/negative examples, supporting and contradicting evidence, recommended validation, date and expiry/review schedule.
- **Observability and recovery:** Record connector failures, ingest lag, parsing errors, unresolved/contradictory edges, indexing failures, search misses, backlog, cost and reprocessing attempts. Support deterministic replay of a chosen import against a versioned parser/index recipe.
- **No blind self-training:** Feedback adjusts evidence and quality evaluations; it does not silently retrain foundation models or increase confidence merely because a patch was approved.

6.9 Knowledge Base acceptance tests and initial build slice

Acceptance test	What the team must demonstrate
Historically complete for test scope	Backfill a selected authorized set of source tickets and repo history; reconcile the source and ingested counts, revision hashes and known linked PRs. Document missing fields.
Idempotent and recoverable	Process the same event twice and restart during a backfill: no duplicated artifacts/relationships; checkpoint resumes, and failed items remain visible.
Cross-project link traceability	For one known defect, show a ticket → PR → commit → symbol/test chain across two projects with source links, explicit/inferred status and correct project permissions.
Contradictory knowledge	Inject a historical revert or conflicting root-cause statement; ensure the curated lesson

Acceptance test	What the team must demonstrate
	and Brief expose the disagreement rather than presenting a verified cause.
Searchable but uncertain history	Ingest a defect missing its fix commit and confirm it remains retrievable with `unknown` relationship status.
Freshness and revocation	Modify one source item and revoke access to another; ensure new revisions become visible and inaccessible chunks/links are no longer returned or cited.
Retrieval quality at scale	On labeled tickets—including cross-project matches and same-symptom different-cause negatives—measure Recall@10, Precision@5, source citation correctness, and p95 retrieval latency on the actual corpus.
Outcome update	After a PR is approved, record the decision; later mark a test regression/revert and confirm the derived knowledge updates without falsely labeling approval as correctness.

Initial implementation slice: two to three authorized projects, their relevant full ticket/PR/commit histories, one known reverted fix, one historical regression test and one enhancement. Scale to the complete dataset only after source mapping, ACL handling, link resolution and retrieval evaluation work on this smaller *real* slice. This plan is compatible with the user's confirmed repository and historical-data access; history quality and runnable tests still require verification.

6.10 Pattern consolidation, provenance and outcome-driven index maintenance [REQUIRED]

Knowledge unit ≠ ticket. One reusable lesson can be supported by many tickets, while a single ticket can contribute evidence to multiple hypotheses. Store raw/revisioned tickets independently from curated, version-aware `pattern` and `lesson_revision` objects. Never collapse contradictory fixes into one authoritative instruction simply because their titles resemble one another.

Consolidation sequence: incoming change event → canonical source revision → candidate existing patterns from hybrid retrieval → compare failure mechanism, affected symbols, release context and evidence → `LINK_EXISTING`, `REVISE_EXISTING`, `FORK_BY_SCOPE`, `CREATE_NEW`, or `HUMAN_REVIEW`. Persist the decision rationale. Pattern revisions are append-only; older lessons remain auditably addressable. `last_seen` and recurrence counts are not substitutes for confidence or correctness.

Refresh policy: maintain material-change hashes for ticket text, commit/PR links, tests, release and incident events. Only affected chunks, embeddings, indexes and graph edges are recomputed. Deletion/access revocation propagates to all derived indexes. A historical claim cited in a past brief keeps its *as-of snapshot* but new briefs use the latest authorized and non-superseded revision. Track index lag, duplicate-rate, stale-lesson-rate and resolved contradiction rate.

Review ownership: engineers or service owners adjudicate disputed lessons and rule changes. VisionIQ may propose edits to OKF/Confluence-style curated knowledge, but must not silently rewrite authoritative design rules or promote its own recommendation to verified truth.

6.11 Knowledge Base implementation contract — what developers actually build

Implementation boundary. Build one **Knowledge Service** with four independently testable jobs: **source synchronization**, **evidence/link resolution**, **derived-index publication**, and **knowledge qualification**. The Historical Intelligence Engine (HIE) reads published, permission-checked snapshots; it never directly edits source tickets or silently promotes an LLM explanation into an approved rule. A single deployable service with background workers is sufficient for the MVP; four separate microservices are not required.

Contract of an ingested object: organization_id + source_system + source_instance + external_artifact_id + source_revision identifies an immutable revision. Every record must carry its original URL, observed time, source modification time, ingestion checkpoint, content digest, owner/repository/project scope, and source access state. Normalized fields may change between parser versions, but the raw source record must remain attributable and replayable under its retention policy.

Producer → consumer	Required contract	Failure behavior
Connector → canonical store	Versioned source revision + source identity + ACL/ownership metadata	Preserve failed cursor/item, retry boundedly; never mark scope complete if pages failed
Extractor → evidence resolver	Explicit identifiers, referenced symbols, candidate cause labels, extractor version and supporting spans	Unparseable records remain searchable as raw evidence; unsupported cause stays UNKNOWN
Resolver → relationship store	Typed edge, target scope, source URL, method (NATIVE_LINK, COMMIT_REFERENCE, INFERRED), status and effective time	Uncertain edges are candidates; do not promote inferred links to verified automatically
Indexer → HIE	Published snapshot ID, source revision IDs, index recipe/version, freshness watermark, ACL check	Serve last *authorized* consistent snapshot with age warning, or abstain
Outcome watcher → lesson curator	Event IDs, PR/commit/release associations and contradictory test/incident evidence	Record observation and dispute; no silent authoritative wiki/rule update

Invariants: no LLM call is required to preserve a source revision or an explicit ADO/Git link; vector embeddings are derived data, not the source of truth; no evidence citation resolves to an inaccessible or deleted artifact; a ticket can support several hypotheses; a lesson can aggregate many tickets without replacing them.

6.12 Concrete extraction and synchronization plan

Historical import — preferred Azure DevOps strategy: use the **Reporting Work Item Revisions** API where available for a revision-level warehouse feed, persist `nextLink`/continuation and `isLastBatch`, and checkpoint only after the corresponding database transaction commits. Use WIQL plus work-item batch retrieval for initial inventory, targeted repair and source-specific cases. Microsoft's REST documentation lists a work-item batch maximum of **200 IDs per call**; treat that as a boundary rather than asking one endpoint for the entire history. Collect comments, work-item relations, PR associations, Git commits, merge parents, changed paths, files and test evidence through their relevant authorized APIs. Exact field names, revision semantics, link types and rate limits must be confirmed on your ADO organization before coding the production connector.

Start with a source-field mapping table, populated from five real samples for each supported work-item type. Normalize `source_id`, `work_item_type`, `state`, `area_path`, `iteration`, `changed_date`, `description`, `comments`, `relations`, `repo_id`, `branch`, `commit_sha`, `pr_id`, `build_id`, `test_case_ref` and `release_ref`. Mark optional/unavailable values explicitly. Do **not** infer a canonical root cause from state `Resolved` or from a commit subject.

Idempotent ingest outline (illustrative):

```
for each configured source and its persisted cursor:
  page = fetch_next_page(cursor)
  for each record in page:
    normalize canonical source key + revision
    redact/restrict sensitive payload for authorized indexing
    upsert immutable revision by (source_key, source_revision)
    enqueue deterministic link-extraction/index jobs if digest changed
  commit records, outbox jobs, and next cursor in one transaction
on retries: same revision is a no-op; changed revision is append-only
on permission revocation/deletion: block retrieval immediately; purge derived
  index/caches, retain only minimal audit metadata allowed by policy
```

Incremental handling: event subscriptions accelerate updates; the revision cursor/reconciliation job recovers missed events. Use a queue with retry/backoff and a dead-letter/quarantine list. Each job key contains source identity + revision + processor version so bot-generated ticket comments cannot cause endless duplicate briefs or duplicated lessons. A full reimport must be safe to replay. Operational views expose per-project source counts, gaps, cursor age, failed pages and last published snapshot.

6.13 Minimum persistent schema — reference design, not a locked database

For the MVP, **PostgreSQL with relational edges, full-text indexing and pgvector is a candidate**, not a mandatory decision. The schema below is illustrative SQL to make requirements concrete; engineers should convert it into reviewed migrations after confirming permissions, retention and query loads. Split large raw

payloads/diffs into an authorized object store rather than blindly storing secrets or entire binaries in embeddings.

```
CREATE TABLE kb_source_revision (
 revision_id UUID PRIMARY KEY,
 org_id TEXT NOT NULL,
 source_system TEXT NOT NULL,
 source_instance TEXT NOT NULL,
 artifact_id TEXT NOT NULL,
 revision_key TEXT NOT NULL,
 artifact_type TEXT NOT NULL,
 source_url TEXT NOT NULL,
 source_modified_at TIMESTAMPTZ,
 observed_at TIMESTAMPTZ NOT NULL,
 content_sha256 TEXT NOT NULL,
 content_ref TEXT,            -- authorized snapshot, not public text
 acl_policy_ref TEXT NOT NULL,
 access_state TEXT NOT NULL,   -- ALLOWED / RESTRICTED / DELETED
 UNIQUE(org_id, source_system, source_instance, artifact_id, revision_key)
);
CREATE INDEX kb_revision_lookup ON kb_source_revision
 (org_id, source_system, source_instance, artifact_id);

CREATE TABLE kb_evidence_edge (
 edge_id UUID PRIMARY KEY,
 from_revision_id UUID NOT NULL REFERENCES kb_source_revision(revision_id),
 to_revision_id UUID NOT NULL REFERENCES kb_source_revision(revision_id),
 relation_type TEXT NOT NULL,  -- fixes/mentions/touches/tests/reverts/depends
 evidence_method TEXT NOT NULL, -- native-link / parser / inference
 evidence_state TEXT NOT NULL,  -- EXPLICIT / INFERRED / VERIFIED / DISPUTED
 supporting_uri TEXT NOT NULL,
 method_version TEXT NOT NULL,
 UNIQUE(from_revision_id, to_revision_id, relation_type, supporting_uri)
);
```

Other required logical tables/collections: kb_sync_cursor (per-source checkpoint and health); kb_chunk (source revision, heading/line range, chunk recipe and ACL); kb_index_manifest (embedding/lexical/symbol model versions, snapshot ID and publication state); kb_pattern (stable pattern identity, component/version applicability, review owner and state); kb_pattern_observation (pattern ID, linked source/evidence, distinct occurrence key); kb_lesson_revision (append-only do/don't, contradictory evidence and approval); and kb_change_outbox (deduplicated index/ACL invalidation operations). The existing analysis_run, brief_revision, advice_id and execution_result entities remain as defined in §§6.2 and 7.4.1.

Schema review gates: stable keys distinguish the same ticket ID in different organizations; a work item can link to multiple PRs/commits; SHA-plus-repository identity distinguishes code across repos; edge evidence must be attributable; revoked sources cannot appear in query counts, previews or citations. Keep history revisioned; never overwrite old conclusions without an auditable supersession link.

6.14 Knowledge extraction, relationships and deduplication

Evidence precedence: (1) explicit native ADO work-item/PR relationships and Git topology; (2) deterministic reference matching with verified repository scope; (3) time, changed paths, symbols and release context as supporting correlation; (4) semantic similarity and LLM hypotheses as exploratory candidates. A temporal neighbor or a matching commit message is **not proof** that a commit resolved a defect. An extracted root_cause_class must be UNVERIFIED until a test, engineer-reviewed investigation or other convincing evidence supports it.

Chunking: create independently retrievable ticket-summary, discussion, incident-timeline, PR-description, function-level diff (where parser support exists), and test-failure chunks. Record origin span and source revision; keep structured IDs, errors, filenames and symbols in lexical metadata rather than embedding everything as undifferentiated prose. Chunk boundaries should follow engineering units first; any token-size limit is **tunable by corpus experiments**, not a locked 500/1,000-token rule. Never include raw access tokens, credentials or forbidden content in model inputs.

Cross-project relationships: start from explicit service/API references, manifests, shared libraries and known owned contracts. Code-level imports indicate dependency candidates, not necessarily runtime call graphs.

Attach repo, path, symbol, version, evidence_kind and verified_at. Historical retrieval should search both local and authorized cross-project indices in parallel; project/module/root-cause predictions are boosts for ranking, **not global exclusion filters**. Apply per-user ACL before retrieval, traversal and final citation rendering.

One defect pattern, many occurrences: group an incoming observation against candidate patterns using affected mechanism, component/contract, release conditions and validated fix/revert evidence. Decide LINK_EXISTING, REVISE_EXISTING, FORK_BY_SCOPE, CREATE_NEW, or HUMAN_REVIEW. Store decision provenance and leave contradictory causes separate until adjudicated. A semantic similarity score alone must never merge two engineering lessons. Increment a recurrence count only for a distinct observed occurrence; feedback on the same ticket must not create a new occurrence.

6.15 Index publication, freshness, security and recovery

- **Parallel indexes:** lexical/BM25-style search for exact errors, IDs and filenames; vector search for paraphrases; symbol/path lookup for code; relational adjacency for dependencies; late evidence-based reranking. Measure recall of the **union**, not just the top vector hits. Exact relevance, evidence support, and probable cause are distinct user-facing labels.
- **Publish consistent snapshots:** all index entries point to canonical revision IDs and a `snapshot_id`; build a candidate snapshot and publish it only after consistency and ACL checks. A stale snapshot must be labeled with its age. In-flight reindexing cannot mix incompatible embedding recipes without recorded versions.
- **Update only affected parts:** ticket revision → its chunks/links/index keys; revert/failed test → linked patterns and affected lesson states; changed service contract → affected dependency edges; model upgrade → explicitly scheduled embedding rebuild. Source deletion or authorization revocation blocks retrieval immediately and triggers cache/index invalidation, including historic links in newly generated briefs.
- **Security checks:** enforce scope using the current requesting identity before ranking AND before displaying snippets; connector service accounts do not imply the viewing engineer has access. Audit cross-project retrieval and model-bound prompts; test malicious instructions embedded in tickets/README files as untrusted source content.
- **Recovery:** checkpoints, deterministic replay, dead-letter processing, rollback to last healthy published index, and verifiable restore of canonical records. Logs report source revisions consumed, skipped, errored, permission-changed and indexed rather than simply reporting “sync succeeded.”

6.16 First development slice and exact knowledge-base exit gate

Use your confirmed repo/history access to begin, but do not assume history quality or test availability is verified. Pick **2–3 connected projects** with a known cross-project dependency, one historical fix, one later regression/revert and one enhancement. Inventory actual ticket types, associations, release/build data and source permission boundaries. Do not import the entire organization into a live search index before checking source-level access and the schema on this slice.

Milestone	Concrete deliverable	Evidence needed to pass
KB-0: source mapping	Source inventory, ADO field map, security owner and 5 real samples/type	No unidentified source ID collisions or missing ACL assumptions
KB-1: replayable import	Backfill revisions, comments, PRs and changes; resumable cursor + quarantine	Source-to-store counts reconciled; same page replay creates no duplicates
KB-2: typed relationships	At least one ticket→PR→commit→code/test path and one cross-project candidate	Native vs inferred edges visibly distinguished and cited
KB-3: hybrid search	Lexical, semantic, code and dependency candidates merged; provenance checked	Labeled relevant and hard-negative matches; metrics and latency recorded
KB-4: lesson lifecycle	One pattern with multiple observations; later revert disputes/supersedes prior advice	No duplicate lessons; old source and lesson revisions preserved
KB-5: safety and recovery	Source revoked, malicious ticket text, connector outage, forced reindex rollback	No cross-scope leakage, silent stale answer or unrecoverable cursor

Validation dataset: label a representative set of resolved/unresolved defects, missing-link cases, cross-project positives and same-symptom/different-root-cause negatives. Start with a modest expert-labeled sample and enlarge as practical; document size, selection method, disagreements and held-out cases. Report Recall@10, Precision@5, evidence/citation correctness, false-positive and abstention behavior, p95 retrieval latency, sync lag and duplicate-pattern rate **with denominators**. Do not claim “95% confidence” or “<1s at 10k tickets” until actual measurements support it.

KB gate: Development of the automatic BRIEF can start against the indexed slice once the system can replay ingestion, prove one traceable historical path, retrieve cross-project evidence safely, show correct uncertainty, and repair a reverted lesson. PR generation is a subsequent independent safety gate, not a reason to bypass the KB tests. This section is a **reviewable build contract**, not a claim that integration or performance has already been demonstrated.

6.17 Repository and Code Intelligence Adapter Registry

The Knowledge Base and Historical Intelligence Engine use common, versioned contracts regardless of programming language. repo_adapter and parser_capability records describe what each authorized repository can actually support. **Do not hard-code C#/.NET assumptions or make Roslyn a dependency.** Proposed integration fields: org_id, repo_id, commit_sha, detected_languages, build_manifests, parser_adapter_id, parser_version, symbol_extraction_status, test_adapter_id, toolchain_image_digest, test_commands, permission_scope, last_validated_at and capability_state (SUPPORTED, PARTIAL, UNAVAILABLE).

Stable cross-language identities: At the minimum, use (org, repo, commit_sha, relative_path) for a file; use (file identity, parser_adapter/version, language, qualified_symbol, source_span) for a candidate symbol. Version symbol references rather than assuming offsets survive refactors. For dependencies, store edge_type (imports, calls, uses_endpoint, publishes_topic, consumes_contract, etc.), supporting location, confidence category and verification state. An API/service relationship must have an observable contract or explicit architecture evidence, not just similar names.

Polyglot operation: Ingest all authorized sources; determine candidate adapters per file; extract typed structural facts only where supported; join with repository/build manifests and documented service contracts; merge into the canonical evidence graph. Retrieval remains hybrid across projects and languages. For unsupported code, preserve source citations and avoid automatic fix claims. This abstraction does not promise comprehensive analysis for arbitrary languages.

Acceptance tests: (1) a supported repository produces stable cited symbols; (2) an unsupported language still returns useful historical brief with explicit UNAVAILABLE status; (3) a mixed-language repo can link verified API contracts without invented internal call edges; (4) the fix agent refuses autonomous patch validation when no runner exists; (5) parser upgrades do not silently corrupt old edges and can be reprocessed; (6) ACL checks apply equally to adapter output and raw records.

6.17.1 Adapter registry integrity and fault recovery

An adapter manifest declares adapter_id, semantic version, supported file types/languages, parser or runner binary digest, capability flags (parse, symbols, build, test, benchmark), supported runtime bounds and approved repository scope. **Version claims must be backed by a small conformance suite**, not inferred from filename extensions.

- **Isolation:** execute adapters under time, memory and input limits; untrusted parsing output is validated against a typed schema. Unknown/corrupted relationships are quarantined before index publication.
- **Failure behavior:** timeouts, crashes, malformed data or missing toolchains produce `ADAPTER_FAILED` / `UNAVAILABLE`, a traceable diagnostic and file-level fallback. An adapter failure never wipes the last consistent published knowledge snapshot.
- **Upgrade and rollback:** stage a candidate adapter against a reference corpus; compare entity count, deterministic source-to-symbol mappings, error rates and retrieval effects. Publish atomically only after checks pass; retain the previously approved version for rollback. Automatic rollback thresholds are **operational policy to measure**, not a universal 5% constant.
- **Refactor-aware identity:** use `repo_id + commit_sha + path + language + qualified_symbol + signature/fingerprint + parser_version`. Identity across renames or moves is a **candidate link** supported by Git move history, syntax similarity and human review where critical; it is not guaranteed by qualified names alone.

Acceptance: deliberately crash an adapter, send malformed symbols, install a missing version and rename a method between SHAs. The pipeline must preserve raw historical evidence, reject corrupt relationships, mark capabilities unavailable and serve the last permitted index (or fail closed on ACL uncertainty).

6.18 Knowledge Graph: exact engineering entities, edge contracts, and traversals

Architecture approach: A knowledge graph is a logical, typed, temporal relationship model stored in Cloud SQL for PostgreSQL. The graph is not the vector index, not an LLM-generated diagram, and not a separate Neo4j/Spanner service at MVP scale. Entities remain traceable to source-specific immutable IDs, and graph traversals enforce caller authorization at every hop.

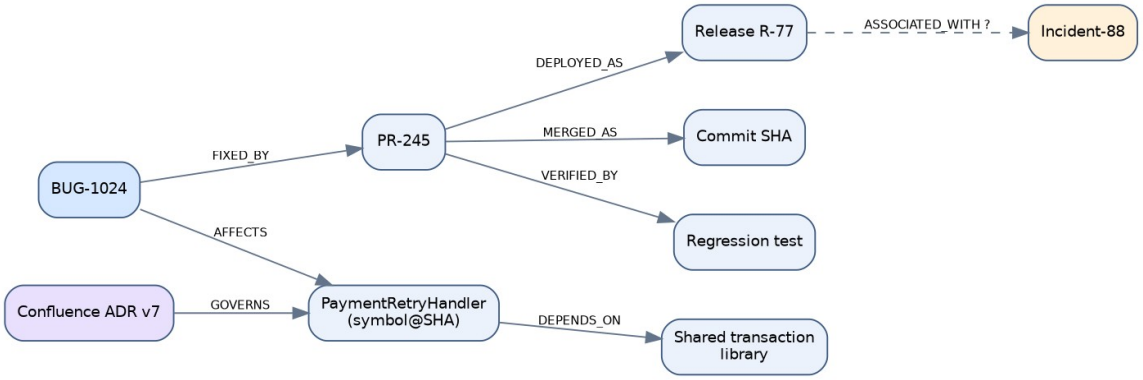


Figure 14 — Sample evidence graph; each edge needs provenance, scope, timestamps and independent authorization. IDs are illustrative.

Minimum node types: PROJECT, REPOSITORY, BRANCH, COMMIT, WORK_ITEM, INCIDENT, PR, PR_ITERATION, FILE_REVISION, SYMBOL_REVISION, TEST_CASE, TEST_RUN, BUILD, DEPLOYMENT, CONFLUENCE_PAGE_REVISION, COMPONENT, DEPENDENCY, PATTERN, LESSON, ADVICE, ENGINEER_DECISION. An independent logical component ID can span repos; it must never automatically merge unrelated same-named projects.

Minimum edge types: LINKED_TO, AFFECTS, CHANGED_IN, FIXED_BY, INTRODUCED_BY_CANDIDATE, TESTS, VERIFIED_BY, REVERTS, REOPENS, DEPENDS_ON, IMPLEMENTS_CONTRACT, GOVERNED_BY, DEPLOYED_AS, OBSERVED_AFTER, SIMILAR_TO, SUPPORTS_CLAIM, CONTRADICTS_CLAIM, INSTANCE_OF_PATTERN. The word FIXED_BY is allowed only when link semantics establish an actual fix relationship; inferred candidates use POTENTIALLY_FIXED_BY instead.

Edge class	Evidence requirement	Example	Publication rule
EXPLICIT	ADO development link, PR work-item relation, exact commit/tree reference	BUG-1024 FIXED_BY PR-245	Preserve external ID, source revision and observation timestamp
DERIVED	Parser-resolved code symbol at pinned commit, verified CI test identity	PR-245 CHANGED_IN PaymentRetryHandler@SHA	Record extractor version, range and extraction proof
INFERRED	Semantic or file-overlap matching, not source-guaranteed	Incident-88 POTENTIALLY_RELATED_TO BUG-1024	Separate from verified edges; expose uncertainty
REVIEWED	Engineer or rule-owner adjudicates a causality or architecture claim	ADR-7 GOVERNS RetryHandler	Store reviewer, decision and evidence revision

Temporal and scope model: Source event time (source_at) is distinct from ingestion time (observed_at). Edge version fields include first_observed_at, last_seen_at, valid_from, valid_to, evidence_ids, extractor_version, assertion_state, project_scope, and acl_reference. Point-in-time analysis includes only evidence *observable* before the evaluation cutoff; creation date alone does not prevent future leakage.

Reference DDL (illustrative; adapt to migrations and your tenancy/ACL design):

```
CREATE TABLE graph_node (
 node_id UUID PRIMARY KEY, tenant_id TEXT NOT NULL, kind TEXT NOT NULL,
 external_key TEXT NOT NULL, revision_key TEXT NOT NULL,
 source_at TIMESTAMPTZ, observed_at TIMESTAMPTZ NOT NULL,
 acl_ref TEXT NOT NULL, metadata JSONB NOT NULL DEFAULT '{}',
 UNIQUE(tenant_id, kind, external_key, revision_key)
);
CREATE TABLE graph_edge (
```

```
  edge_id UUID PRIMARY KEY, tenant_id TEXT NOT NULL,
  from_id UUID NOT NULL REFERENCES graph_node(node_id),
  to_id UUID NOT NULL REFERENCES graph_node(node_id),
  kind TEXT NOT NULL, assertion_state TEXT NOT NULL,
  evidence_ids JSONB NOT NULL, extractor_version TEXT,
  first_observed_at TIMESTAMPTZ NOT NULL, valid_to TIMESTAMPTZ,
  UNIQUE(tenant_id, from_id, to_id, kind, first_observed_at)
);
CREATE INDEX edge_from_active ON graph_edge(tenant_id, from_id, kind)
  WHERE valid_to IS NULL;
CREATE INDEX edge_to_active ON graph_edge(tenant_id, to_id, kind)
  WHERE valid_to IS NULL;
```

Example traversal: Given a defect in `PaymentRetryHandler`, traverse (1) `AFFECTS` to candidate symbols, (2) previous `CHANGED_IN` and `FIXED_BY` paths, (3) `VERIFIED_BY` tests, (4) `REVERTS` and incident association, and (5) `GOVERNED_BY` Confluence architecture decisions. Collect independent supporting and counter-evidence. Limit hop count, cap candidates, and apply ACL checks before fetching/returning any node details. Similarity edges must not silently become causal claims.

Graph quality gates: Every returned edge is backed by a resolvable evidence ID; unauthorized cross-project links are not traversable; conflicting links are represented rather than overwritten; renames retain revision history; a backfill replay yields the same logical active graph; a disputed observation cannot silently overwrite a reviewed lesson.

6.19 Codebase Intelligence: language-neutral repository ingestion and indexing

Input: Repository connectors read permitted repositories, branch policy, default/head SHA, commit parents, diffs, file changes, selected test manifests, build descriptors, dependency manifests and repository metadata. For each repo the system first runs **capability discovery**, not an assumed language-specific parser. Minimum product support is file- and commit-level history for any text-based repository; richer symbols, dependencies and automated tests require tested adapters.

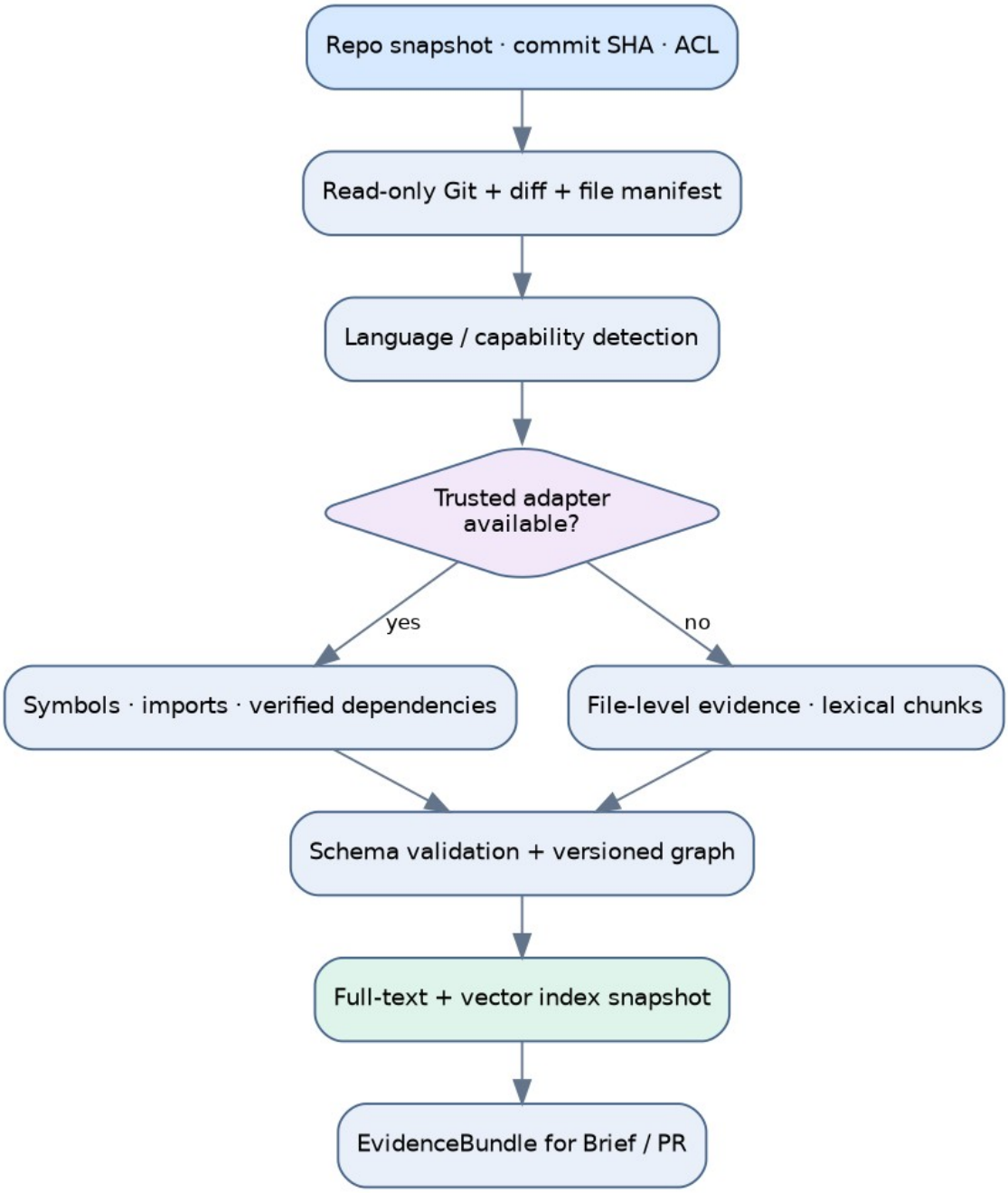


Figure 15 — Repository intelligence is pin-to-commit and adapter-based; unsupported languages degrade honestly.

Repository pipeline contract: (1) Register {provider, organization, project, repository_id, default_branch, tenant, acl_policy}; (2) fetch source at SHA and changed commits without executing it; (3) compute immutable revision manifest and file hashes; (4) detect languages using file signatures and manifests; (5) invoke optional trusted parser/dependency adapters with fixed versions; (6) validate structured output before atomic publication; (7) upsert versioned graph nodes and evidence chunks; (8) embed only changed permitted chunks and update lexical index; (9) mark the snapshot READY only after ACL and source-pointer checks. An adapter crash leaves the previous published snapshot active.

Adapter contract: detect(repo_snapshot) → capabilities; extract_files(sha) → file_revisions; extract_symbols(sha, paths) → symbol_revisions; extract_dependencies(sha) → edges; discover_tests(sha) → tests; build_test_plan(sha, test_ids) → execution_manifest. Return SUPPORTED, PARTIAL, UNSUPPORTED, or ERROR per capability. Every result includes adapter_id, adapter_version, source_sha, source_span, content_hash, warnings and evidence references. No language gets an implied safe-to-patch status merely because Tree-sitter can parse it.

Repository example	File-level support	Optional code intelligence	Build/test execution
Python/FastAPI	Commit/diff, requirements, source chunks	Functions, imports, routes, pytest mapping	Only with approved Python runner and required fixtures
Java/Spring	Commit/diff, Maven/Gradle descriptor	Class/method, package, REST mapping	Only with approved JDK/build image
C#/.NET	Commit/diff, .sln/.csproj metadata	AST parser/language tooling as adapter, **not Roslyn as core dependency**	Only with approved SDK image
Vue/TypeScript	Commit/diff, package.json/lockfile	Components, imports, routes, typed symbols	Only with Node and configured tests
Unsupported/proprietary language	Commit/diff, file names, work-item links	UNSUPPORTED symbol claims	NOT_RUN; guidance only unless a safe runner exists

Refactor lineage: symbol_revision_id is tied to repo + commit + file + qualified name + span/content signature + parser version. A probable rename/move is POSSIBLE_SUCCESSOR_OF until verified by Git move evidence or human review. Avoid pretending stable line numbers prove identity across refactors.

Worked example: A bug report says "payment succeeded but ticket not printed." Hybrid retrieval first finds matching historical work items (including other repositories); the graph joins Payments.Api/retry.py@shaA to shared/transaction.py@shaB through extracted dependencies. A Confluence ADR states the API must be idempotent. The brief cites both the historical PR and the ADR. If only file-level indexing is available, VisionIQ says "candidate files identified, symbol relationships unverified" instead of asserting call paths.

Source confidentiality: Source code and AI prompt contexts are permission-filtered before provider calls. Protected source files may still be indexed as metadata without transmitting their contents. Revoked ACLs invalidate cached briefs, retrieval visibility and prompt context; tenant boundaries apply to graph traversal, full-text search and vector search equally.

6.20 Confluence ingestion and curated OKF knowledge lifecycle [P0 source; curated export P1]

Confluence is a first-class historical source, not an automatically authoritative rulebook. Capture **space/page IDs, versions, title, labels, headings, body representation, author, edit timestamps, attachment references, page restrictions, canonical URL, source checksum and ingestion timestamp**. Use cursor pagination and page-version APIs where supported; handle deletes, permission changes and version gaps via scheduled reconciliation. Keep a source snapshot per observed revision; re-index changed sections rather than the entire space.

Worked example: Confluence page Payment Reliability ADR version 7 states: "Retry operations must use a stable transaction idempotency key." Ticket BUG-1024 concerns duplicate charges. A PR touching retry logic must retrieve the ADR's exact version plus earlier regressions, but the document's guidance remains CLAIMED until the owning team reviews its continuing applicability. A revised version 8 supersedes the published guidance only after permission-aware ingest and curated review.

OKF decision: Adopt optional OKF-compatible Markdown/YAML for high-value curated engineering lessons, not as the primary online database or vector engine. An approved lesson can be exported into Git/GCS, code-reviewed, versioned, and reindexed to the Knowledge Base. OKF supports provenance/verification/freshness metadata; it has **no inherent latency advantage** over indexed SQL retrieval and does not execute constraints. For MVP, a DB-backed lesson object is sufficient; an OKF export is desirable when curation is operationally ready.

Example OKF concept (illustrative, conformant structure; links must be replaced by actual authorized evidence):

```
---
type: Engineering Lesson
title: Retry operations must preserve idempotency
resource: urn:visioniq:lesson:retry-idempotency
```

```
tags: [payments, retry, regression]
status: current
generated: {by: process:visioniq-curator, at: 2026-10-26T10:00:00Z}
verified:
 - {by: human:payment-owner, at: 2026-10-26T11:00:00Z}
sources:
 - {id: adr-v7, resource: https://confluence.example/wiki/pages/7007}
 - {id: pr-245, resource: https://dev.azure.com/example/project/_git/repo/pullrequest/245}
---
# Rule
Use a stable idempotency key for retryable transaction commands.[^adr-v7]
# Counterexamples
A prior retry-only patch was reverted; see the linked PR.[^pr-245]
[^adr-v7]: Confluence ADR revision 7
[^pr-245]: Linked historical PR
```

OKF publication rules: Separate `generated` from `verified`; never set `verified` merely because an LLM generated the file, a PR merged or an engineer clicked Accept on advice. Gate `status=current` on verified current applicability; otherwise keep candidate/unverified or stale status. Source URLs in exports must honor access controls; avoid putting restricted evidence into broadly accessible Git or Cloud Storage buckets. Always store DB `lesson_id` ↔ OKF path/revision mapping.

7. Historical matching, reasoning and algorithm design (LLD)

7.1 Hybrid candidate retrieval and reranking

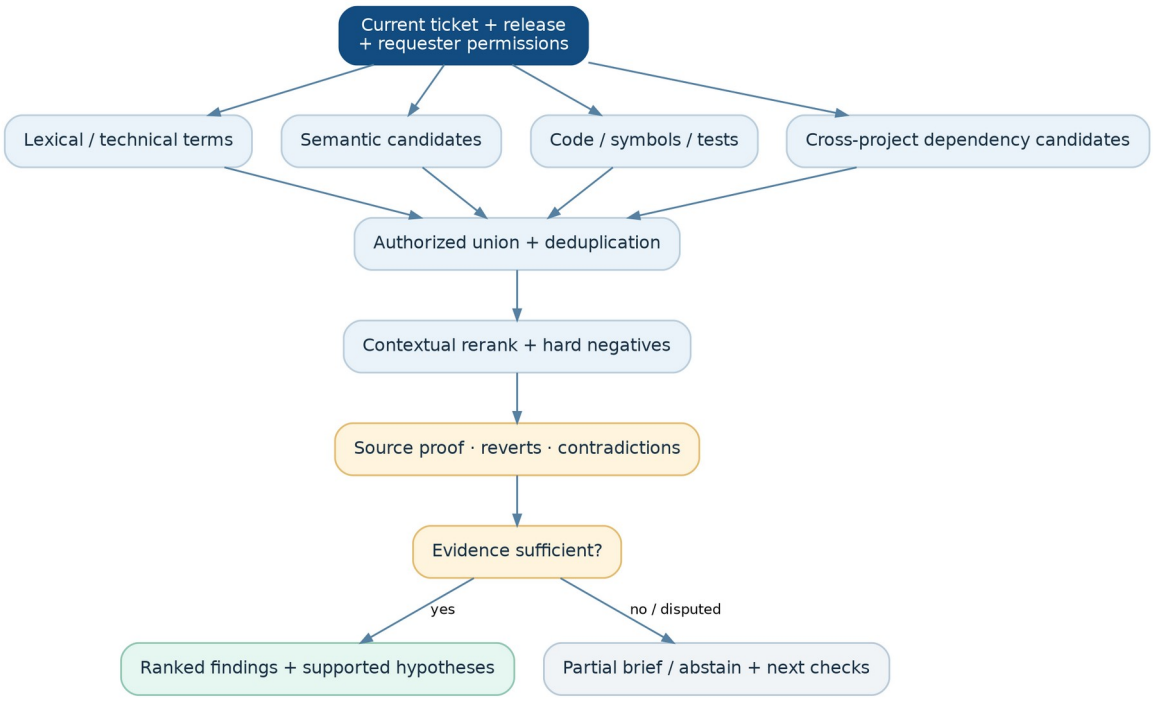


Diagram 9 — A multi-lane candidate union prevents premature module/root-cause filters from discarding hidden matches.

Algorithm (candidate):

25. Build query using ticket symptoms, explicit technical terms, logs, module hints, repository/source branch and release ID; preserve missing fields.
26. Run a **union** of lexical search, semantic nearest neighbors, symbol/path matching and authorized dependency-neighborhood retrieval. Scope must apply at candidate retrieval and again at hydration/citation time.
27. Normalize/deduplicate by canonical revision; diversify candidates across repositories and symptom variants.
28. Rerank using textual relevance, version compatibility, structural/dependency proximity, explicit linkage, fix/revert/test outcome and evidence freshness. Penalize unsupported or contradictory claims rather than hiding them.

29. Retrieve source snippets and link proofs for a bounded top-K, preserving multiple competing hypotheses.
30. Produce typed `EvidenceBundle`; never assume a ticket's assigned root-cause label is ground truth.
31. On weak evidence: return useful partial brief, unknowns and suggested discriminating checks, not invented certainty.

Avoid known errors from the P4 alternative: do not filter first to the same project/module and a guessed root-cause class; do not choose arbitrary cosine cutoffs (`0.85`) as an accuracy guarantee; do not assume BigQuery latency or vector dimensions without measuring the deployed design; do not send entire commit histories to the model by default.

7.1.1 A concrete search plan for a 10,000+ ticket corpus

The alternative P4 document proposed hard narrowing from **10,000 → 100 (same project/module) → 20 (predicted root cause) → 5–10 (semantic)**. That is a useful efficiency idea but **not a safe default** for VisionIQ: a wrong module or cause prediction can remove the true cross-project match before semantic search begins. These numbers are examples, not measured stage counts.

Recommended first implementation — bounded retrieval across independent paths:

Stage	Candidate policy	Why / verification
0. Offline preparation	Ingest authorized, revisioned ticket text, linked changes, tests, component and release references. Maintain indexes incrementally.	Avoid re-reading the entire historical corpus or embedding every ticket at query time.
1. Hard eligibility	Apply requester/source authorization and explicit project exclusions only; preserve relevant historical and cross-project versions.	Security is a hard filter; guessed root cause and same-project classification are **not**.
2. Parallel retrieval	Search lexical terms and error signatures, vector semantics, code paths/symbols, and dependency-linked projects independently. Start by requesting **up to 30–50** per lane as a tunable experiment.	Broadens recall; lane sizes are configurable starting values, not design facts.
3. Candidate union	Deduplicate by source identity/revision and retain candidates from more than one project.	Prevent duplicate evidence from dominating; don't silently eliminate a cross-project match.
4. Rerank	Reconsider symptoms, explicit links, version compatibility, topology, tests/reverts, contradicting evidence; initially rerank **approximately 20–40** candidates.	Do not use cosine score alone or automatically treat a closed ticket's cause as verified.
5. Evidence hydration	Retrieve source excerpts, linked diffs/PRs, tests and outcomes for an initially bounded **5–10** evidence-rich candidates, keeping competing hypotheses where relevant.	Limit tokens, latency and cost without discarding why a candidate matched.
6. Evidence decision	Return supported, disputed or unknown conclusions. State coverage; retrieve further evidence or abstain when key context is missing.	No unsupported 95% confidence labels or fabricated root causes.

Required tests before accepting this search design: include known same-project matches, cross-project matches, a ticket with missing module metadata, same-symptom/different-root-cause negative examples, old-but-applicable fixes, relevant reverts, and cases where the source root cause was wrong. Record Recall@5/10, Precision@5, nDCG/MRR where applicable, citation correctness, false matches, permission leakage and end-to-end p50/p95 latency. Test a hard-cascade baseline against this multi-path design on the **same held-out labeled set**. Tune lane limits and reranking **based on measured recall and latency**, not assumed 100→20→5 reductions.

Retrieval latency is not brief latency. The proposed indexed lookup target and the separate ≤15-second p95 ticket brief target in §11.1 remain experiments. PR generation, compilation and historical regression replay run asynchronously under their own budgets. The existence of 10,000 ticket records alone does not justify

BigQuery, Dataflow or a separate vector cluster; evaluate the real data volume, access boundaries and query workload in ADR-01.

7.2 Hypothesis selection and discrimination

For each candidate cause store supporting/contradicting evidence, affected code/release, predicted symptoms and a cheap test that distinguishes it from alternatives. Prefer **reproduction and differential checks** over model self-assessment. Stop at a bounded budget or missing environment and ask the engineer for a specific required artifact. Root-cause status vocabulary: `unverified`, `supported`, `disputed`, `reproduced`, `refuted`.

7.3 Confidence and evaluation scheme

Use **relevance/rank**, **evidence coverage**, **root-cause verification state** and **patch test status** as separate output fields. Only publish a percentage when the target event and calibration procedure are defined on a representative held-out dataset. Approval rate, code reuse and simple correlation between model score and approval are **not equivalent** to root-cause correctness or probability calibration. Evaluate using labeled Precision@K, Recall@K, MRR/nDCG, false-negative stress tests, source-citation accuracy, selective prediction (coverage vs error), and per-project/release slices. A model's stated certainty, SonarQube match and recency cannot be added using invented 40/30/20/10 weights and called "92.6% correct."

7.4 Ticket brief response contract

```
{
  "run_id": "run_example",
  "ticket": {"system": "ado", "project": "TVM", "id": "1234", "revision": 8},
  "scope": {"data_snapshot": "snapshot_example", "release": "unknown"},
  "risk_areas": [{"component_id": "printer-api", "relationship": "suspected", "evidence_ids": ["e1"]}],
  "historical_matches": [{"artifact_id": "incident-27", "rank": 1, "relevance_label": "strong", "reason":
"same retry symptom and interface", "evidence_ids": ["e2"]}],
  "hypotheses": [{"cause": "retry-state handling", "status": "unverified", "supporting_ids": ["e2"],
"contradicting_ids": [], "next_check": "reproduce duplicate-print retry"}],
  "guidance": {"do": ["test idempotent retry"], "dont": ["repeat reverted retry behavior"], "approach":
"diagnose before patching", "evidence_ids": ["e2"]},
  "coverage": {"has_runtime_logs": false, "has_fix_link": true, "has_release": false},
  "warnings": ["Ticket symptoms are not proof of printer-layer root cause"]
}
```

Illustrative schema; exact API naming and data types require team approval. Material assertions must reference access-controlled evidence IDs resolving to immutable source revision and URI.

7.4.1 Atomic advice objects — required correctness tracking [REQUIRED]

Production briefs must emit individually identifiable advice items in addition to the summary guidance. Each material risk, do/don't or best approach gets an advice ID scoped to the immutable brief revision; advice type, exact statement, evidence references and applicability scope are retained. Different recommendations in one brief may have different outcomes.

Example advice fields: brief_revision_id, advice_id (ADV-001), kind (RISK / DO / DONT / APPROACH), statement, evidence_ids, applicability (component/release), initial_evidence_state. Feedback and verification separately reference brief_revision_id + advice_id, authenticated evaluator, timestamp, tests/PR/release and underlying evidence. Corrections produce new versions, never silent mutation.

See §§8.10–8.11 for the outcome and embedded engineer-interaction contract. Approval of an entire PR must never auto-label every advice item correct.

7.5 Explainable engineering risk assessment [P1; P0 transparency for the demo]

Produce an **explainable risk profile**, not a fabricated numerical probability that a change will fail. Candidate signals include recent defect/fix density normalized by module size, confirmed backfire rate with observation-window coverage, change churn, co-change dependencies, meaningful test coverage gaps and concentration of ownership. Include software version and environment where known; use a missing-data status when values are unavailable. Avoid penalizing teams merely because they create detailed tickets or have one primary maintainer.

Risk assessment has two outputs: **(a) evidence components** with ticket/PR/test IDs, counts, time windows, missing-data flags and links, and **(b) an ordinal risk band** (`LOW`, `MODERATE`, `HIGH`, `INSUFFICIENT_DATA`) whose thresholds are agreed using a labeled held-out set. Optional numeric rank scores may aid sorting but **must not be labeled calibrated failure probabilities**. A verified revert or regression is stronger evidence than raw commit churn; any weighting must be recorded as an ADR and evaluated for stability across repositories.

Example *illustrative*: `Payments retry interface: HIGH` — 6 relevant fixes in 90 days, 2 later reverted, and one known duplicate-issue regression test. Show each supporting ID and whether the incidents occurred on the same deployed version; do not call the label predictive until backtested. A recent spike should surface in Learn's hotspot sweep and be available to the flagship Impact Agent as an authorized risk-breakdown response.

Acceptance tests: all displayed factors have supporting evidence and window definitions; missing coverage is not treated as zero risk; two projects with different sizes are meaningfully normalized; a false-positive dependency can be disputed; top-risk areas are compared with held-out outcomes and engineering review. Time-bounded backtesting in §11.2.1 prevents future defects from leaking into historical risk labels.

8. Workflows, contracts and integration behavior (LLD)

8.1 Event and state contract

Durable events: `WorkItemPickedUp`, `SourceRevisionObserved`, `IndexUpdated`, `BriefRequested`, `BriefPublished`, `DeepAnalysisRequested`, `PatchProduced`, `ValidationCompleted`, `DraftPRCreated`, `ReviewDecisionObserved`, `OutcomeVerified`. All include `{event_id, origin, source_system, organization, project, external_item_id, item_revision, occurred_at, correlation_id, actor_scope, idempotency_key}`. Run uniqueness proposal: `(source, ticket ID, revision, workflow policy version)` plus explicit re-analysis revision. Reject stale events; filter bot-generated updates to prevent feedback loops.

Mandatory pickup event semantics: eligible = source project's configured defect/enhancement types + configured actionable transition + authorized organization/project. Record the triggering actor, work-item revision and analysis policy. Produce at most one active analysis per `(source, work_item, revision, policy_revision)` unless an explicit reanalysis is requested. A separate scheduler sweeps for missed events and indexing gaps. Brief statuses: `queued`, `building`, `published`, `partial`, `blocked`, `failed`, with timestamp and link in the source ticket when possible. Do not re-trigger from VisionIQ's own comments.

8.2 Proposed internal endpoints

API	Responsibility	Security and output
`POST /webhooks/ado`	Verify webhook, record ID, enqueue durable run	Quick 202; no model execution on request thread
`POST /connections/{id}/sync`	Trigger bounded initial/incremental sync	Admin scope, job ID, cursor
`POST /tickets/{ticketRef}/analyze`	Create explicit analysis run	Requester ACL and expected ticket revision
`GET /runs/{runId}`	Return state, partial results and failure reason	Scope-filtered read
`GET /runs/{runId}/evidence`	Return citations, hypotheses and supporting artifacts	ACL recheck; no unrestricted snippets
`POST /runs/{runId}/draft-fix`	Request a narrow code attempt	Repository/branch allowlist; audit
`POST /runs/{runId}/validate`	Request build/tests/performance	Exact base/patch SHAs and test recipe
`POST /runs/{runId}/feedback`	Capture engineer decision and rationale	Authorized reviewer; immutable audit
`GET /integrations/impact-agent/history`	Provide scoped historical context	**Provisional until Impact Agent owners specify contract**

Use versioned JSON schemas or typed DTOs for all boundaries and schema-migration rules. Public URLs/field names for vendor APIs must be verified during implementation against access and current vendor documentation.

8.3 Azure DevOps and Jira interaction

The source work item and existing Impact Agent are the required user surfaces; no standalone VisionIQ app is required. An embedded feedback interaction is still necessary for individual advice items. Use supported service hooks/webhooks and REST APIs to detect eligible work-item pickup and publish a **clearly attributed bot comment/extension panel**. Avoid changing engineer-authored titles, root-cause fields, descriptions or ownership silently. The draft PR is created by a bot identity with only read, branch-create and PR-create permissions, and no merge/completion ability. Configure branch policies independent of the model. Jira is an

adapter expansion if required; ADO is the primary MVP integration unless team source access dictates otherwise.

8.4 Flagship Impact Agent — required historical view, interface TBD [GIVEN / REQUIRED]

The phrase **"Adds the history view to the flagship's Impact Agent"** means the existing Impact Agent should be able to show a **historical risk/evidence view** in its existing interaction surface. It does **not** mean building a second VisionIQ UI or assuming an undisclosed Impact Agent API already exists.

Business contract — decided: given a ticket or proposed change and the authorized engineer, VisionIQ supplies affected components, relevant historical incidents, prior fixes and backfires, cross-project dependency paths, do/don't guidance, provenance, timestamps, supported/disputed/unknown labels and a link to the native ticket/PR. The Impact Agent owns presentation and must not lose its existing impact-analysis responsibility.

Suggested logical response (illustrative schema, not a discovered vendor API):

```
{
  "ticket_ref": "ADO:ProjectA:1234",
  "as_of": "source-snapshot-id",
  "affected_components": ["ticketing-ui", "printer-service"],
  "historical_matches": [
    {"source_ref": "ADO:ProjectB:567", "relevance": "high", "cause_status": "supported", "why_related": "same
retry pathway"}
  ],
  "prior_backfires": [{"source_ref": "PR:ProjectB:88", "failure": "duplicate-print regression"}],
  "recommended_checks": ["replay idempotent retry regression"],
  "limitations": ["deployment link for one incident unverified"],
  "evidence_links": ["authorized source URI"]
}
```

Technical contract — open with the Impact Agent owner: determine its actual extension mechanism (service call, plugin, shared query API, supported event, or other owner-approved method), identity propagation, access controls, history-view layout, error handling, payload versioning, latency expectations and testing rights. VisionIQ will expose a scoped adapter whose implementation is decided **after** that interface is confirmed. Do not claim integration complete merely because the adapter design exists.

Acceptance test: using an allowed sandbox ticket/change, the existing Impact Agent visibly presents at least one permitted historical failure, linked source evidence, affected component/risk and an uncertainty label furnished by VisionIQ; inaccessible cross-project evidence is omitted; Impact Agent remains usable if VisionIQ is temporarily unavailable. A simulated independent history dashboard **does not satisfy** this requirement.

8.5 Enhancement guidance contract

Enhancement output must include acceptance-criteria gaps, touched contracts/APIs, deployment and migration risks, related past changes, do/don't, recommended sequence, tests and reviewer decisions. For a breaking API change, the guidance must name known dependent clients and versioning constraints *only when evidence supports them*. No "always generate complete PR" assumption; starter PR is optional.

8.6 Code execution and PR pipeline

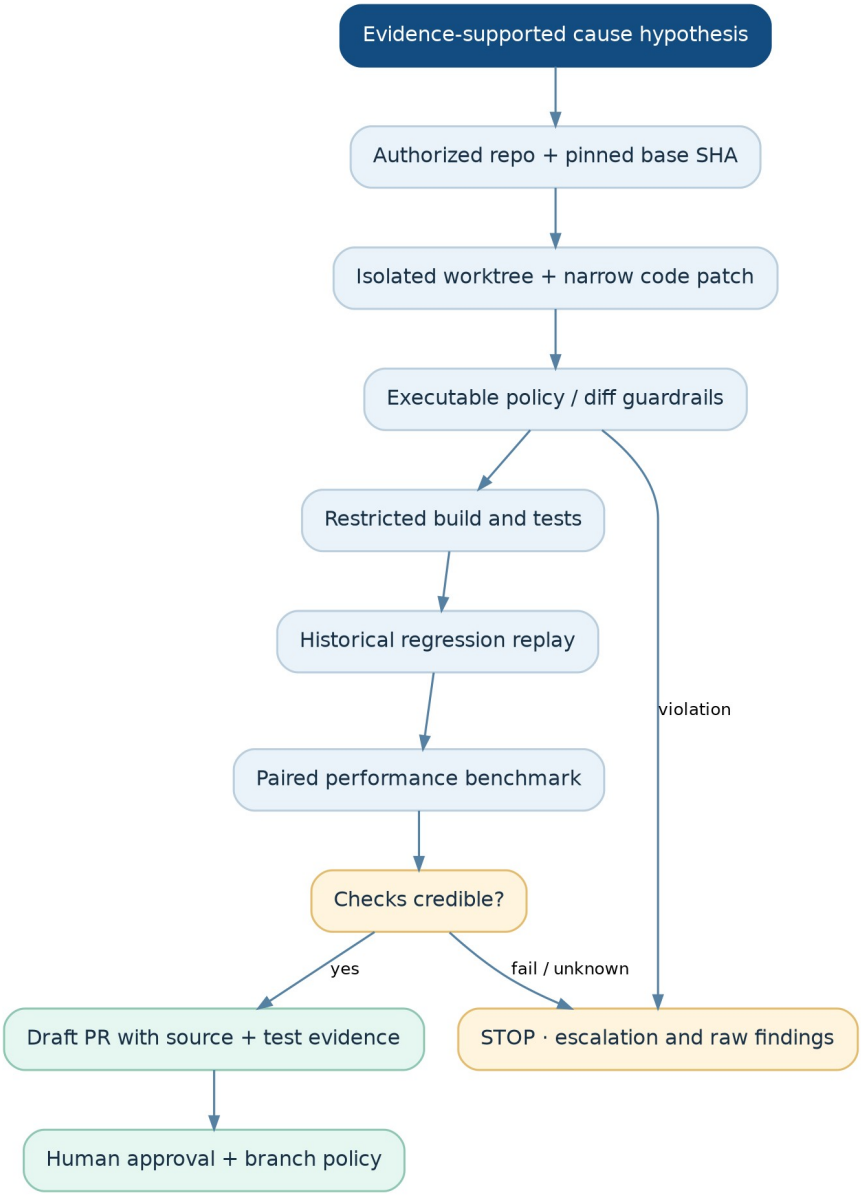


Diagram 10 — Safety controls are executable gates, not words in an LLM prompt.

The runner requires non-production credentials, limited network egress, ephemeral filesystem/workspace, non-root or equivalent isolation, resource/time limits, restricted file paths, build/test recipe allowlist and output artifact hashes. CI logs, runner image digest, test names, failures, benchmark methodology and raw metrics accompany PR. Tests that need unavailable kiosk hardware must be marked unverified or simulated, with real-hardware follow-up outside the hackathon proof.

8.6.1 CI/CD status gates and review ownership [REQUIRED]

VisionIQ must use the repository's **actual** validation pipeline, not a fabricated local "green" label. It may invoke authorized sandbox reproduction before opening a draft PR, but the submitted PR must also pass the relevant configured branch policies and required CI checks. Keep the statuses separate: sandbox feasibility, PR CI success, historical-regression result and touched-path performance comparison are **different evidence objects**. Every result stores repo, base SHA, patch SHA, command/job, environment, timestamp, exit code and log URI.

Check / event	Native integration	Required behavior
Draft PR opened or updated	Azure Repos / Git PR API	Annotate historical

Check / event	Native integration	Required behavior
		evidence, assigned reviewers, unknowns, verified vs proposed findings
Build, unit, integration and contract checks	Existing CI pipeline	Read and surface **actual** per-check pass/fail/not-run; respect existing branch policy
Historical regression and performance checks	Approved CI job or isolated runner with linked results	Prevent a false 'pass' when unavailable, flaky, timed-out or inconclusive
Reviewer decision	Existing PR approval/rejection API/events	Capture who decided and when; only a human with existing permissions can complete merge
CI or review state changes	Signed/authorized webhook or polling reconciliation	Re-evaluate status; do not assume a prior green run applies to a changed patch SHA
After merge or later incident	CI/release/incident evidence sync	Record verified outcome separately from the approval action

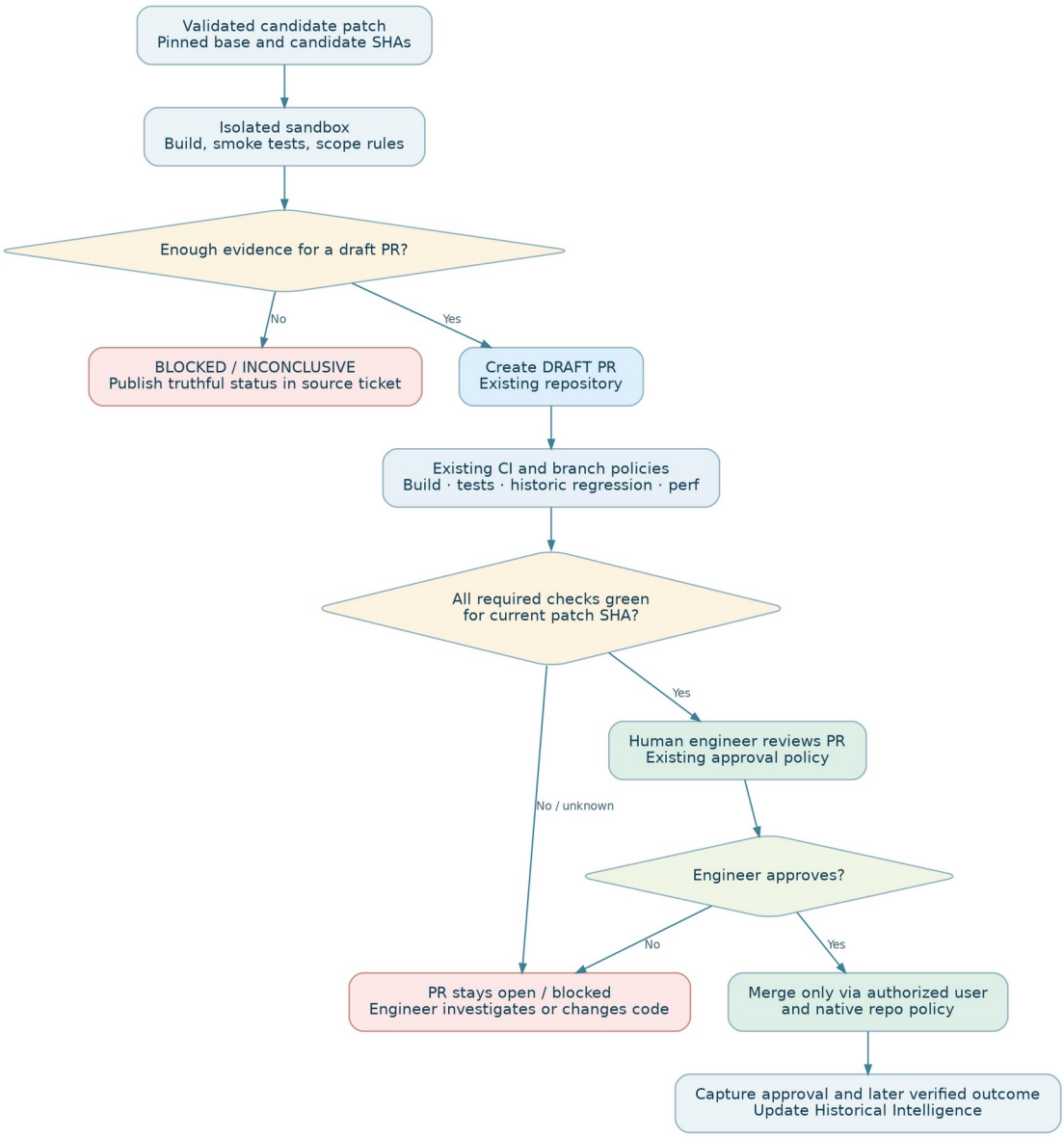


Diagram 11 — CI and reviewer gates belong to the existing repository platform; VisionIQ cannot bypass them.

8.6.2 Defect reproduction first — red-green patch verification

For defects that can be reproduced safely, the strongest proof is **red-green testing**. (1) Pin the baseline revision SHA and environment. (2) Derive a minimal failing test from the new ticket, existing tests and verified historical evidence. (3) **Execute it on the unchanged baseline**; confirm it fails for the targeted defect. (4) Apply a bounded candidate patch in a restricted workspace. (5) Run the **same test** and verify it passes; also run related historical regressions, native CI and applicable performance/contract checks. (6) Attach baseline/fix SHAs, test identities, runner image, full logs and any inconclusive results to the reviewable PR. Rejection, cancellation and retry states remain auditable; no auto-merge.

A new test that only passes on the patched branch is **not** enough to prove it detects the original bug. A failing test can be nondeterministic or fail because of an unrelated environment error, so reproduction classification must check the failure signature. Rerun flaky tests under a fixed policy; unresolved flakiness must not be presented as verified. **Not all defects are reproducible in a sandbox** (e.g. specialized hardware or inaccessible services); mark REPRODUCIBILITY=UNAVAILABLE and do not invent a green result or bypass a human gate. PR creation and CI status follow the official win-condition requirements and repo policies.

Definition of a demonstrated red-green case: same test case and controlled conditions; baseline targeted failure; patched targeted pass; previously known regression test(s) passing; native CI green for the current commit; relevant performance evidence or explicit not-measured state; engineer approval. This strengthens the Failure Memory Firewall without pretending tests can prove universal correctness.

8.6.3 Reproducible runner manifest and test-result taxonomy

Before executing generated code, the sandbox writes an immutable ExecutionManifest with: ticket_id, base_sha, candidate_sha, repository, runner/container **digest** (not latest), OS/architecture, language/runtime and package lock hashes, dependency restore policy, environment variables **by non-secret name only**, fixture and seeded-data hashes, simulator/service versions, network restrictions, resource limits, commands and expected test IDs. Link every result to this manifest and to the exact code SHA under test.

Test outcomes are separate from PR approval: PASS, FAIL_TARGET_SIGNATURE, FAIL_DIFFERENT_SIGNATURE, FLAKY, ENVIRONMENT_ERROR, TIMEOUT, NOT_RUN, UNAVAILABLE, INCONCLUSIVE. The baseline defect test must produce the **target failure signature**; unrelated compilation or environment errors do not establish red. The candidate must pass the same test under equivalent conditions; historical regressions must be executed where available. PR status must never display all green when a **required** gate is NOT_RUN or INCONCLUSIVE.

Flakiness policy [PROPOSED STARTING POINT]: run once normally, then rerun an inconsistent or suspected-flaky case a bounded number of times (initially up to two additional executions), recording each run and signature. A result alternating between pass and fail becomes FLAKY, not an averaged PASS; retries **do not erase the first failure**. Use per-suite timeouts based on existing CI baselines, not one universal 30-second limit. Review the retry budget with the owning team before enabling automated approvals. Test infrastructure faults are separately retriable at the job level with bounded attempts.

Environmental fallback: where hardware, private services or licenses are unavailable, run appropriately scoped mocks/contract fixtures and report their limitations; physical or unavailable checks remain UNAVAILABLE with a human escalation. Recorded/simulated tests never prove real printer/card/payment hardware behavior. Do not manufacture tests, credentials or simulator equivalence to satisfy the demo.

8.6.4 Scope-aware patch guardrails and PR policy

Prefer minimal, reviewable patches, but **do not invent a universal five-file or 200-line safety guarantee**. The authorization policy limits writable repositories, branches and paths; flags security, payment, identity, schema migration and hardware-control areas; disallows secret changes, unapproved dependency upgrades and unexpected generated artifacts. Broader-than-expected changes are stopped for engineer review regardless of whether the changed-file count is small. Establish configurable patch size/iteration budgets based on repository conventions and testability.

The automation may open a **draft** PR with accurately reported failed or incomplete checks when permitted by policy, but must never approve its own change, complete the merge or bypass protected-branch requirements. If a safe candidate cannot be produced, report a blocked case truthfully; this is a deviation from the organizer's ideal 'PR for every defect' that must remain visible in compliance reporting.

8.7 Performance validation rules

Performance comparison should use an identical fixture/workload and environment for base and candidate, repeat runs and record variability, startup/warmup effects and relevant metric(s): e.g., latency p95 or throughput under specified concurrency. A single noisy measurement or blanket 1% threshold is insufficient. Show **no demonstrated regression under the tested conditions**, not “proved performance cannot regress.” Any rule for acceptable change must be configurable and reviewed by owner.

8.8 Post-merge watcher and observation-event contracts [REQUIRED FOR COMPLETE PRODUCT]

Inputs: PR approval/merge events, commit SHA, linked work item, build status, deployment manifest/release ID, relevant incident/revert telemetry where available; each event is authenticated, normalized, idempotent and stored with source provenance. **Triggers:** merge, deploy, rollback/revert, new related incident, expiry of observation window and owner adjudication. None initiate autonomous production changes.

Process: verify PR and change hash → link actual release(s) (do not infer from merge alone) → start a configured observation window for touched paths/environments → collect related signals and negative cases → classify `VERIFIED_EFFECTIVE`, `REGRESSED_OR_REVERTED`, `DISPUTED`, `UNKNOWN_OR_UNOBSERVED`, or `OBSERVING` → append a new outcome revision → notify engineers through existing work-item/PR channels where materially adverse. An observation's status is accompanied by explicit coverage and evidence references. Data access limits and absent telemetry stay visible.

Failure modes: missing deployment data, unavailable telemetry, rollback for unrelated reasons, stale time windows, missing release mapping, replicated incidents and partial outage feeds produce *unknown/inconclusive*, not forced success/failure. Never correlate solely by timestamps. Adapters degrade gracefully if an environment has no deployment or APM integration.

8.9 Selective knowledge-learning service contract [REQUIRED]

Idempotent operation: `ingest_observation(source_event_id, artifact_revision_id)` stores unique occurrence once; `resolve_pattern(observation_id)` emits a reasoned `LINK_EXISTING` / `REVISE_EXISTING` / `FORK_BY_SCOPE` / `CREATE_NEW` / `HUMAN_REVIEW`; `publish_lesson_revision(pattern_id, approved_evidence_ids)` creates a new knowledge version only if material evidence changed or an owner approved the correction. These are proposed internal signatures, not external endpoints.

Never turn a PR-approval rate into factual diagnosis accuracy. Track feedback on usefulness separately from objective reproduction, revert and deployment evidence. Confidence/ranking models update only via controlled labeled evaluation; no automatic model weight changes after every accepted PR. Preserve the disputed old advice in history for audit and prevent it from being surfaced as current best practice without a warning.

8.10 Advice Outcome Intelligence — per-recommendation tracking [REQUIRED]

Official quality bar: each brief recommendation is evaluated independently. An engineer's decision is evidence about usefulness and adoption, not automatic proof of correctness. Each published advice statement is immutable and links to source evidence, ticket/brief revisions, relevant component/release and generated-version metadata.

Two independent state dimensions:

Dimension	States	Meaning
Engineer disposition	PRESENTED; ACCEPTED; MODIFIED; REJECTED; NOT_APPLICABLE	What the engineer did, not a truth claim
Evidence / outcome judgment	UNVERIFIED; SUPPORTED_BY_TEST; CONTRADICTED_BY_TEST; VERIFIED_CORRECT; VERIFIED_WRONG; DISPUTED;	What reproducible checks, expert review and later outcomes support

	SUPERSEDED	

A verified outcome requires reproducible test or owner-reviewed evidence with base/candidate SHA, check recipe and environment. CI green, PR approval and no observed incident are not sufficient to declare historical guidance correct. Contradictions or missing telemetry remain UNVERIFIED/DISPUTED. Post-merge data may update judgment as a versioned event.

Proposed contracts: POST /runs/{runId}/advice/{adviceId}/feedback (disposition, reason, authenticated reviewer, brief revision); restricted POST /advice/{adviceId}/verification (evidence_state, test/incident IDs, adjudicator). Emit AdviceDispositionRecorded and AdviceEvidenceUpdated using idempotency keys. Track accuracy on adjudicated advice only, and report verified coverage, unresolved, adoption and repeated-wrong advice separately.

Canonical knowledge is selectively revised only when material new evidence appears. Repeated approvals do not create duplicate lessons or mathematically increase confidence. Verified-wrong guidance is flagged/superseded in new briefs and the Impact Agent while its historical version is preserved.

8.11 Embedded engineer experience — UI without a separate VisionIQ app [REQUIRED]

A usable engineer-facing interface is required, but a separate VisionIQ application is not. The required surfaces are the existing ADO/Jira work item, native PR/CI review pages and the flagship Impact Agent. Briefs still appear automatically on ticket pickup with no VisionIQ login or button.

MVP: attributed ticket comment with advice IDs, citations and uncertainty; authorized structured work-item comments/fields can capture ADV-001 ACCEPTED / REJECTED / NOT_APPLICABLE with optional reason. The bot correlates the response to the immutable brief item and leaves unparseable comments unverified. Native PR approval remains the merge gate.

Preferred product UX when permitted: embedded Azure DevOps work-item extension/panel with per-advice Accept, Reject, Not applicable and comment controls; separate read-only evidence/verification state and source links. Impact Agent receives and displays the same canonical advice, provenance and superseded warnings through its approved interface. Admin-only operations UI may exist, but never forms part of the required ticket-to-PR flow.

Acceptance: the engineer can receive the brief, review individual advice and handle the PR entirely within their existing engineering tools; no side-tool login or auto-merge privilege is required. Non-response remains UNVERIFIED, never CORRECT.

8.12 PR-driven Knowledge Base updates and selective continuous learning [P0; MAJOR REQUIREMENT]

Required design rule: Every meaningful PR event updates historical evidence; no PR event automatically establishes a verified root cause or creates a new canonical lesson. Learning is two-tiered: (A) event-driven append-only evidence sync, fast and idempotent; (B) targeted knowledge reconciliation for changed components and patterns, asynchronous and gated by actual proof. This is the internal implementation of organizer **LEARN — ALWAYS ON**, and it reinforces post-merge observation (§3.6), non-repetitive learning (§3.7) and advice correctness (§8.10).

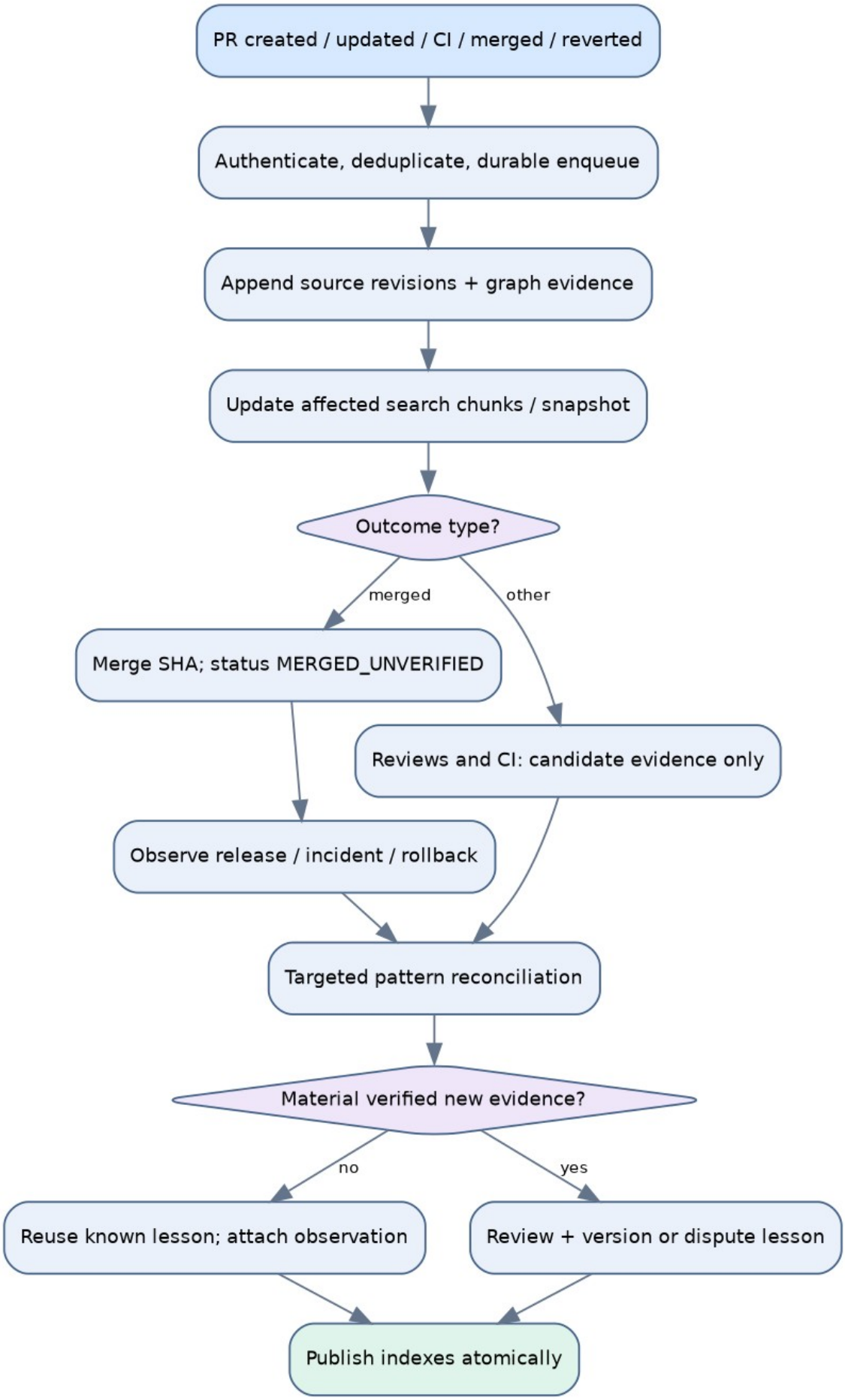

PR created / updated / CI / merged / reverted
Authenticate, deduplicate, durable enqueue
Append source revisions + graph evidence
Update affected search chunks / snapshot
Outcome type?
merged
other
Merge SHA; status MERGED_UNVERIFIED
Reviews and CI: candidate evidence only
Observe release / incident / rollback
Targeted pattern reconciliation
Material verified new evidence?
no
yes
Reuse known lesson; attach observation
Review + version or dispute lesson
Publish indexes atomically

Figure 16 — PR events always enrich evidence; only substantiated, material outcomes update the reusable engineering memory.

Incoming event	Immediate evidence work	Deferred learning effect	May claim correct fix?
git.pullrequest.created	Capture PR ID, source/target, source SHA, work-item links, initial changed files and patch intent	Link hypotheses to past patterns; initiate scope-specific search	**No**
git.pullrequest.updated / source push	New PR_ITERATION, source SHA, file diff, comments/votes, obsolete prior diff pointers	Reindex changed files and refresh candidate test selection	**No**
Build/test/CI result	Persist exact runner manifest, test IDs, pass/fail/inconclusive, report URLs, gate versions	Support/refute specific patch/test claims	**No**, test-bounded only
PR abandoned/rejected	Preserve explicit review rationale and abandoned patch fingerprint	Candidate unsafe approach, pending adjudication	**No**
PR completed/merged	Save merge commit, final patch fingerprint, required human approval and branch-policy evidence	Mark proposed fix as merged; schedule deployment observation	**No**, merge is not production outcome
Deployment success/failure	Correlate release SHA, environment, artifact and rollout evidence	Material support or contradiction within declared observation window	Not by deployment alone
Reopen / rollback / revert / incident	Capture canonical links, versions, production signals and causality review	Escalate disputed lesson, reweight evidence, trigger targeted reconciliation	Only after verified linkage
Engineer advice feedback	Append advice_id, accepted/modified/rejected and rationale	Improve usefulness model; do not overwrite independent correctness status	**No**

Two queues, two SLO classes: Webhook ingestion responds quickly after authentication and durable enqueue. source-sync fetches authoritative source records, deduplicates source-event keys and commits immutable source revisions plus an outbox record. knowledge-reconcile processes only affected repositories/components/pattern IDs after source-sync commits. Do not run a full 10,000-ticket re-ingestion on every PR. Periodic reconciliation repairs missed events, webhooks out of order and connector downtime.

Example idempotent event envelope:

```
{
 "event_id":"ado:orgA:repo7:pr245:iteration3:updated",
 "tenant_id":"tenantA", "provider":"azure-devops",
 "event_type":"git.pullrequest.updated",
 "source_project":"payments", "repository_id":"repo7",
 "pr_id":"245", "iteration":3,
 "source_sha":"abc123", "occurred_at":"2026-10-26T11:00:00Z",
 "received_at":"2026-10-26T11:00:03Z",
 "payload_sha256":"...", "correlation_id":"visioniq-run-9001"
}
```

Transactional persistence rule: Key events by the provider's stable event identity when available; otherwise combine object, revision, type and payload fingerprint. In one DB transaction, insert the idempotency record, source revision, graph/evidence updates and outbox message. Publish from outbox after commit. A duplicate does nothing beyond recording a delivery attempt; an older revision cannot replace newer active source state. Failed extraction does not alter active index snapshot.

Change-impact invalidation: A PR touching shared/transaction.py invalidates only relevant symbol/file chunks, edges, derived risk features and briefs whose evidence snapshots depend on that component. Existing source nodes remain historical. Generate new embeddings for changed chunks, not the whole repository. If an adapter is unavailable, publish file-level evidence and retain the last trusted symbol map until the new parse is validated; mark stale lineage.

Lesson state machine: CANDIDATE → EVIDENCE_SUPPORTED → REVIEW_REQUIRED → VERIFIED_CURRENT; alternate outcomes DISPUTED, STALE, SUPERSEDED, REJECTED. Transition criteria are configuration-controlled and auditable. A source-linked merge may support EVIDENCE_SUPPORTED for “change was applied,” but not

"root cause fixed." A later revert must append contradictory evidence, notify the pattern owner and temporarily downgrade advice assertiveness until resolved. Canonical pattern deduplication links new observations into the same pattern when validated fingerprint and contextual scope match; superficially similar symptoms with different causes remain separate patterns.

Worked lifecycle (illustrative, not real project evidence): On Monday, BUG-1024 has an associated PR-245. VisionIQ records proposed idempotency handling and the tests added. On Tuesday, PR-245 is merged with passing CI; the KB records a MERGED observation, not a verified production fix. On Thursday, release R-77 deploys the merge SHA and telemetry is normal, raising evidence strength for that limited observation window. Two weeks later, BUG-1102 reopens a duplicate-payment case and an engineer links it to the same codepath; VisionIQ does **not** invent a new pattern. It reopens review of the existing retry-idempotency lesson, marks it DISPUTED if causality is uncertain, and records any confirmed regression in the graph. After review, a verified correction creates lesson version 2 and supersedes version 1. Every response can still cite both versions and when each was believed valid.

Engineer-facing observations: Ticket and Impact Agent history show (1) linked PR status, (2) evidence freshness, (3) test/runner proof, (4) lesson confidence/evidence state, (5) later regression/revert evidence and (6) advice_id feedback state. No separate VisionIQ screen is required; native PR approval is mandatory.

Acceptance scenarios: (1) Duplicate webhook delivered five times results in one logical revision; (2) updated PR changes only its affected graph/search partitions; (3) a merged PR with no release data remains MERGED_UNVERIFIED; (4) a later revert links as contradiction and does not erase old assertions; (5) same defect recurs across another project without duplicating a canonical lesson; (6) ACL revocation removes query visibility and cached brief access; (7) advice Accept never changes verified_correct; (8) a failed worker resumes from checkpoint and the previous snapshot remains queryable.

9. Security, architecture governance and operational reliability

Risk	Mitigation and verification
Unauthorized cross-project disclosure	Source-derived ACLs before search **and** again before summarizing/citing; auth tests with two users and restricted repo
Prompt injection in ticket, comment or code	Retrieved content always untrusted data; protect tools/policies; structured-output validation
Secrets or sensitive data in embeddings/LLM input	Secret scanning/redaction, provider approval, minimal context, retention policy
Generated code causes exfiltration/crypto-mining	Restricted no-prod-secret runner, resource budget, network allowlist/deny, cleanup, audit
Bot auto-merge or broad modifications	Credentials cannot merge; path/diff budgets; branch protection; reviewer enforcement
Replayed webhook / bot self-trigger loop	Authentication, idempotency, origin filter, durable retry/dead-letter queue
Outdated link, index or branch	Immutable source revisions, freshness flags, resync before PR; track deployed version
Documentation/rule drift	Reviewed policy PR or ticket; no direct unreviewed Confluence writes
Outage/timeouts/cost	Circuit breaker, provider fallback where approved, per-run budgets, queued work and partial brief
False diagnosis / automation bias	Explanations, counterevidence, abstention, independent review and eventual outcome tracking
Transport Ops business consequences	Include payment integrity, duplicate print prevention, idempotency, hardware constraints in targeted tests

Observability: correlate all source events, indexing runs, retrieval queries, model calls, runner jobs, PRs and feedback via trace/run IDs. Report queue lag, source freshness, p50/p95/p99 stage latency, failures/retries, model tokens/cost, citation accuracy sampling, abstention rate, retrieval recall and regression detection results. Do not report ROI or prevented incidents without a defensible counterfactual baseline.

10. Technology architecture: Google Cloud, RAG, OKF, ADK and model providers

Technologies are described by responsibility: Google Cloud services provide hosting, Gemini and Claude provide model capabilities, and deterministic retrieval, testing and approval layers govern results. Backend implementation examples may use ASP.NET Core or Python/FastAPI.

10.1 Technology roles and what they do not do

Technology / pattern	Contribution to VisionIQ	Limits and required companion
Hybrid RAG (core pattern)	Retrieves grounded evidence from authorized tickets, commits, PRs, test failures, documents and dependency records, then supplies a bounded EvidenceBundle for Brief and investigation.	Retrieval can return irrelevant or omit crucial history; RAG cannot establish causal truth. Use multi-path search, provenance, version context, counterevidence and evaluation.
OKF (optional knowledge format)	A portable Markdown + YAML metadata representation for curated architecture decisions, verified historical lessons and human-reviewed operational rules. Provenance, review status and freshness can travel with the document.	OKF is **not** a policy engine, vector database or universal factual authority. Keeping 10,000 raw tickets as OKF files is not a requirement. Use explicit validators for enforceable rules.
Google ADK (conditional orchestration framework)	Coordinates model-based agents and deterministic tools; its sequential, parallel and loop workflow primitives can organize hypothesis investigation, evidence checks and tool calls.	ADK does **not** guarantee root-cause correctness or make jobs durable by itself. Persist states, enforce permissions/time budgets and make test/PR gates independent from the model.
Deterministic engineering tools (core)	Git, language compiler/test runner, schema and contract checks, static analysis, CI, sandbox and PR branch policies create *observable* pass/fail evidence.	Even green tests are bounded by fixture coverage; performance measurements need controlled baselines and variance handling.

Key boundary: AI may propose an explanation or patch, but only the authorized execution services and Git/CI policies may determine test execution, PR eligibility and actual approval state. Engineer feedback is evidence about usefulness, not automatic proof of correctness.

10.2 Technology interaction in the four-phase workflow

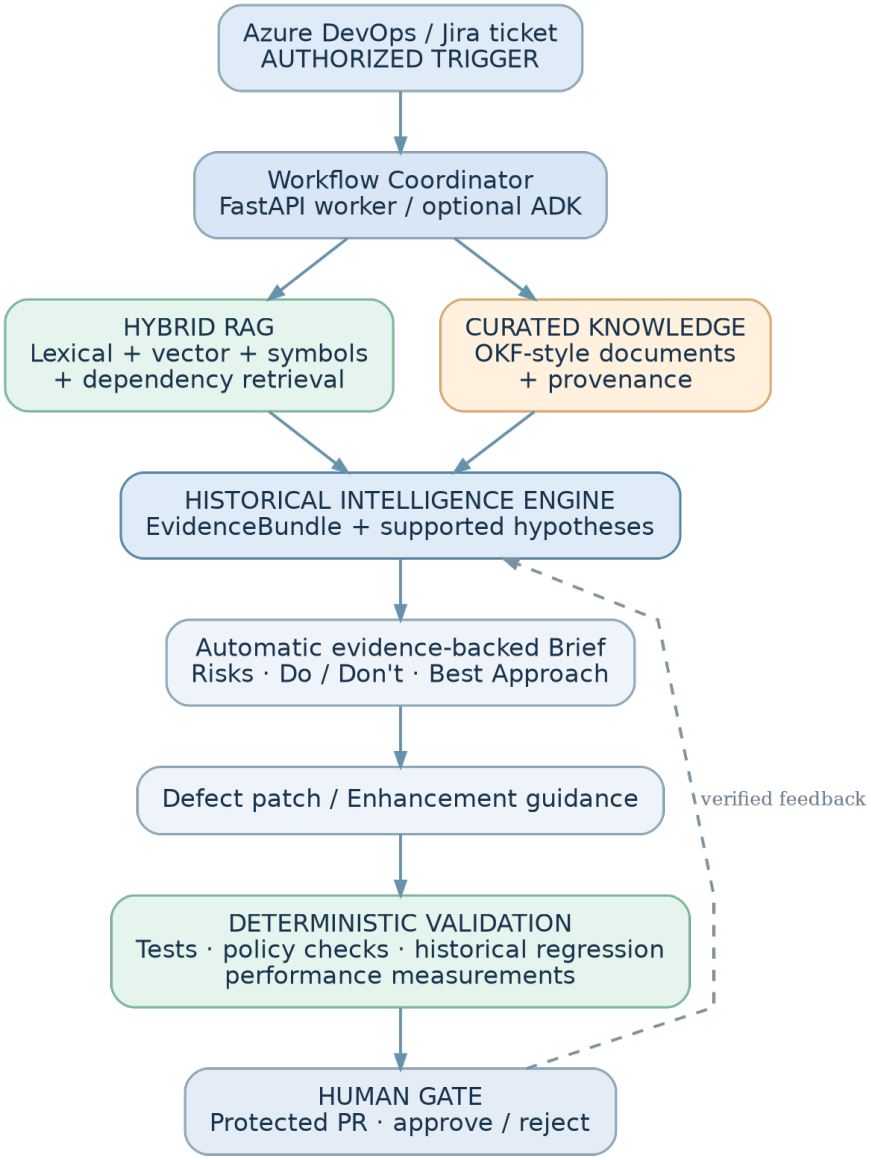


Diagram 12 — RAG obtains evidence, optional OKF organizes curated knowledge, ADK or ordinary workers coordinate tasks, and deterministic tools independently gate code execution.

Lifecycle separation: a reliable event/scheduling layer runs the post-merge watcher and material-change knowledge jobs regardless of whether ADK is selected for LLM orchestration. Retrieval and knowledge-index updates are deterministic services; LLMs provide grounded analysis but never directly certify production outcomes. OKF/curated rules must be versioned and owner-approved, with executable validators providing enforcement.

10.3 Technology components and implementation alternatives

- **Application/API and worker: ASP.NET Core or Python/FastAPI; typed input/output contracts, durable job states, idempotent Azure DevOps events and trace identifiers. Optional orchestration libraries can be introduced when justified by a concrete workflow.**
- **Canonical data store:** PostgreSQL for tickets/commits/evidence edges, revisions, audit and tenant/source ACLs. Prefer immutable source IDs and exact source links. The ingestion process stores observed links separately from inferred links.

- **Search:** PostgreSQL full-text + **pgvector** as a low-infrastructure starting candidate; rank lexical, semantic and symbol/dependency matches independently, merge and rerank. Compare Qdrant or managed vector search only if real corpus benchmarks justify it.
- **Historical evidence and rule artifacts:** Raw source payloads in controlled artifact storage; curated OKF-compatible Markdown/YAML can record human-reviewed insights and provenance. Machine-checkable policies live in version-controlled rule definitions interpreted by validators.
- Agent reasoning: Gemini through Vertex AI and Claude through an approved provider-neutral interface; require structured output, source citations, controlled tool execution and abstention. Provider choice needs an equal-data quality/cost evaluation.
- **Code analysis and verification:** Git APIs, Tree-sitter or language-native parsers for supported repositories; pinned Docker/CI runner images, network limits, CPU/memory/time budgets, selected regression tests and baseline performance runs.
- Coordination/deployment: A durable queue and workers, Google Cloud Run, Pub/Sub, Jobs and Scheduler. Cloud Run and Pub/Sub are included in the deployment design; Google ADK remains optional. ADO webhook acceptance should never wait for expensive AI processing.
- **UI integration:** Post a non-destructive ADO work-item comment or history-view link; show a source-linked Brief and real PR status. Avoid silently rewriting ticket titles/descriptions or Confluence rules.

Technology options are shown for implementation flexibility. The Google Cloud deployment architecture is detailed in §10.8.

10.4 Technology comparison matrix

Choice	Minimal candidate	Alternative from external design	Decision rule / concern	Status
App/API	Python FastAPI on Cloud Run	ADK multi-agent workflow	Favor explicit durable states and typed boundaries; ADK optional only if tested to help	SELECTED
Persistence	Cloud SQL PostgreSQL (MVP)	BigQuery	Transactional updates, rights, revision queries vs analytics scale	SELECTED MVP
Retrieval	Cloud SQL full-text + pgvector	Vertex Vector Search / Qdrant	Compare Recall@K, permission filtering, cost and p95 on actual corpus	SELECTED MVP (benchmark pending)
Relationship graph	SQL edges and adjacency indexes	Separate graph / in-memory JSON	Need versioned, explainable edges; lookup not universally O(1)	**[PROPOSED]**
Code parsing	Tree-sitter for supported languages + language-native tools	Prompt-only language detection	Correct symbol parsing and build tooling must be proven; not “any language automatically”	**[OPEN]**
LLM	Gemini (Vertex AI) + Claude adapter	Claude or Gemini asserted as superior	Evaluate on same held-out tickets; legal/data access and model cost first	PROVIDERS AVAILABLE; ROUTING OPEN
Code execution	Isolated CI runner / sandbox with pinned image	Cloud Run execution sandbox	Must demonstrate isolation, controls, reproductions and CI interoperability	**[OPEN]**
Rule guardrails	Versioned YAML rules + executable validators	“OKF” format	Format alone does not enforce policy; validator test is acceptance	**[PROPOSED]**
Event bus	Pub/Sub + idempotent Cloud Run	Pub/Sub / Tasks / Cloud Scheduler	Use existing authorized infra; avoids deploying unnecessary components	SELECTED
ADO ticket presentation	Bot comment/extension panel	Automatic title/description mutation	Preserve human-authored source fields by default	**[PROPOSED]**

10.5 Technology proof tests and adoption gates

Decision / proof	Small real-data experiment	Adopt only when
Hybrid RAG retrieval	On a labeled set of defects with known relevant history, measure Recall@10, Precision@5, cross-project recall, p95 end-to-end retrieval and ACL leakage rate.	Useful history is recovered under an agreed latency target, permission checks pass, and reranking outperforms the strict cascade baseline.
OKF / curated rule format	Encode 5 reviewed architectural lessons with provenance, owner and review date; change one rule and revalidate downstream consumers.	Knowledge remains auditable/portable and at least one enforceable rule has a passing/failing validator test.
ADK vs ordinary orchestrator	Implement identical 2–3 step investigation in both approaches; induce retry, timeout, insufficient evidence and restart.	ADK meaningfully improves observability, testing or maintainability without weakening durable state, idempotency or human gates.
Database/vector platform	Index the permitted historical corpus in PostgreSQL/pgvector, then measure ingestion time, query p95, search quality and footprint.	The chosen option satisfies functional and cost criteria; upgrade only for measured requirements.
Sandbox & PR	Execute a reproducible historical regression on an isolated patch candidate; force timeout, failing test and forbidden code change.	No failing or unauthorized change is represented as green; PR remains reviewable and cannot merge without a human.
Language-neutral adapter registry	Parse/test one supported toolchain; ingest unsupported language and polyglot repository at file/contract level.	Symbols/edges have citations; unsupported code does not get invented symbols, impact, or green-test claims.

10.6 Technology risks and security boundaries

Data protection: Send model providers only project-authorized and policy-permitted code/tickets; protect secrets and personally identifying ticket content, retain audit of what evidence was sent, and implement retention/deletion rules. Vector metadata, reranker context and knowledge files must obey the same source ACLs as raw evidence.

Model/agent failures: Use deterministic maximum iteration counts, budget limits, schema validation, source-citation checks, explicit stop conditions and a human-visible inconclusive outcome. ADK callback or prompt instructions are not substitutes for server-side authorization and CI gates.

Operational complexity: PostgreSQL + a worker + a runner is a reasonable baseline for the pilot. Adding Dataflow, BigQuery, external Qdrant, Pub/Sub and multi-agent orchestration all at once is likely unnecessary without evidence from performance or security tests.

10.7 Technology references

- Google Agent Development Kit: official documentation — supports agents, tools, workflow composition and evaluation; this is orchestration infrastructure, not a correctness proof.
- Google Open Knowledge Format v0.2 specification — portable Markdown + YAML, optional provenance/trust/freshness; not an executable guardrail by itself.

- pgvector: hybrid search documentation — supports integration with PostgreSQL full-text search and reranking methods.
- Azure DevOps service hook events and draft PR creation — tenant permissions and versions remain integration tests.

Platform feasibility caveat: The other AI documents assume particular organizational scale, cloud permissions, SonarQube availability, API costs and performance. Repository and historical-data access have been confirmed; actual corpus shape, source ACL coverage, code/test execution rights and benchmark results must still be measured. This document does **not** lock a cloud stack or claim RAG, OKF or ADK guarantees correctness.

10.8 Google Cloud deployment stack and service boundaries

Deployment boundary: All VisionIQ-owned application services, databases, queues, batch jobs, observability, artifact storage and deployment run on Google Cloud. Source systems (Azure DevOps, Jira, Confluence and source repositories) remain external integrations; their existence does not violate Google Cloud hosting. Gemini on Vertex AI is available; direct Anthropic Claude API access is available. The direct Claude API is a separately hosted model endpoint even when the calling service runs on GCP. Confirm corporate policy before sending restricted source content outside the approved Google Cloud processing boundary. If strict GCP-only model processing is required, evaluate supported Claude partner models on Vertex AI and provision them explicitly; do not assume the direct API credential authorizes Vertex AI model access.

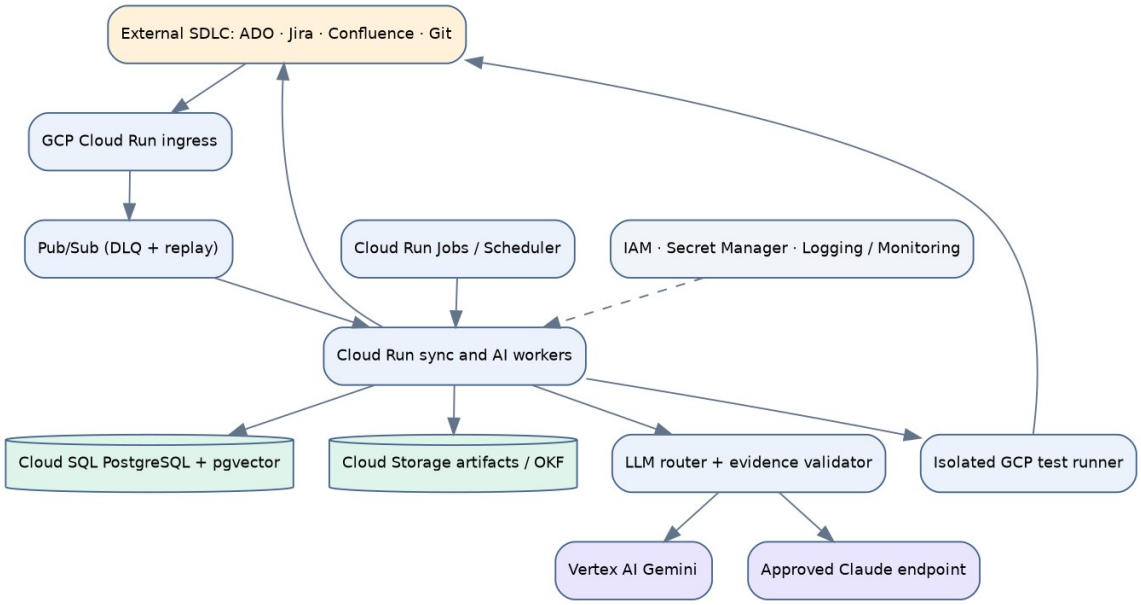


Figure 17 — Google Cloud deployment boundary with external SDLC systems and clearly separated model endpoints.

Concern	Selected MVP technology	Responsibility / deployment rule	Status
API / event ingress	**Cloud Run + FastAPI**	Signed/authorized webhook intake; return after durable enqueue; private internal services	**SELECTED**
Message processing	**Pub/Sub** with dedup + dead-letter topics	At-least-once delivery; DB event idempotence is mandatory	**SELECTED**
Historical batch	**Cloud Run Jobs + Cloud Scheduler**	Paginated initial backfill, incremental reconciliation, index repair, snapshots	**SELECTED**
Durable evidence / relational graph	**Cloud SQL for PostgreSQL**	Source versions, graph nodes and edges, ACL references, advice, PR outcome ledger	**SELECTED for MVP**
Similarity and lexical search	**pgvector + PostgreSQL full-text**	Authorized hybrid candidate retrieval; test index and recall with	**SELECTED for MVP**

Concern	Selected MVP technology	Responsibility / deployment rule	Status
		actual corpus	
Source and run artifacts	**Cloud Storage**	Encrypted restricted revisions, signed controlled references, OKF bundle, run manifests	**SELECTED**
AI embeddings	**Vertex AI embeddings**	Versioned embeddings for changed authorized chunks; exact model/dimension must be pinned	**MODEL OPEN**
AI reasoning	**Gemini (Vertex AI) + Claude provider adapter**	Structured hypotheses, briefs, explanations and patch proposals via common contracts	**TWO PROVIDERS CONFIRMED; routing proposed**
Observability/security	**Cloud Logging, Monitoring, Secret Manager, IAM**	Correlation IDs, error budgets, tenant isolation and least privilege	**SELECTED**
Safe test execution	**Approved isolated GCP job/build runner**	Pin toolchains, isolate untrusted code, block outbound credentials, publish PR checks	**IMPLEMENTATION GATE OPEN**
CI/CD and deployment	**Cloud Build + Artifact Registry / Terraform**	Build, scan, immutable image digest, deploy GCP services and verify rollback	**PROPOSED DEFAULT**
Agent framework	**Explicit Python state machine first; Google ADK optional**	ADK only for proven benefits in bounded agent workflows; durable job state remains in DB	**CONDITIONAL**
User interface	**ADO/Jira + Impact Agent + native PR/CI**	Automatic briefs, advice feedback, history and human approval; no required standalone app	**REQUIRED FLOW**

Recommended deployable units: visioniq-api (work item briefs/internal APIs); visioniq-webhook (source ingress); visioniq-ingestor (Cloud Run Job); visioniq-indexer (job/worker); visioniq-retriever (private API or in-process module); visioniq-ai-worker (router and bounded analysis); visioniq-pr-worker (PR creation/status, no merge permission); visioniq-outcome-worker (post-merge watch and lesson reconciliation). Consolidate components into fewer Cloud Run services for the hackathon if this reduces operations, but retain module boundaries, persistent task states and separate IAM identities.

Terraform/config minimum: separate service accounts for source reads, model invocation, code execution, PR creation and deployment; Secret Manager secrets not accessible to untrusted runners; Cloud SQL private connectivity; Cloud Storage bucket-scoped access; environment-specific config; provider budgets, retries and region; VPC egress restrictions where required; separate Pub/Sub dead-letter and replay tooling. Capture region/residency and external-Claude egress in the security ADR before production use.

10.9 Dual-LLM contract and routing policy: Gemini + Claude

Design principle: The Knowledge Base retrieves and qualifies evidence independently of the LLM. LLMs analyze bounded evidence and propose structured hypotheses; they never decide whether an ID exists, whether an ACL grants access, whether code tests passed, or whether a PR merged. Do not call both providers for every ticket. Assign roles by evaluated quality, cost and latency, and never infer one provider is universally superior without a benchmark.

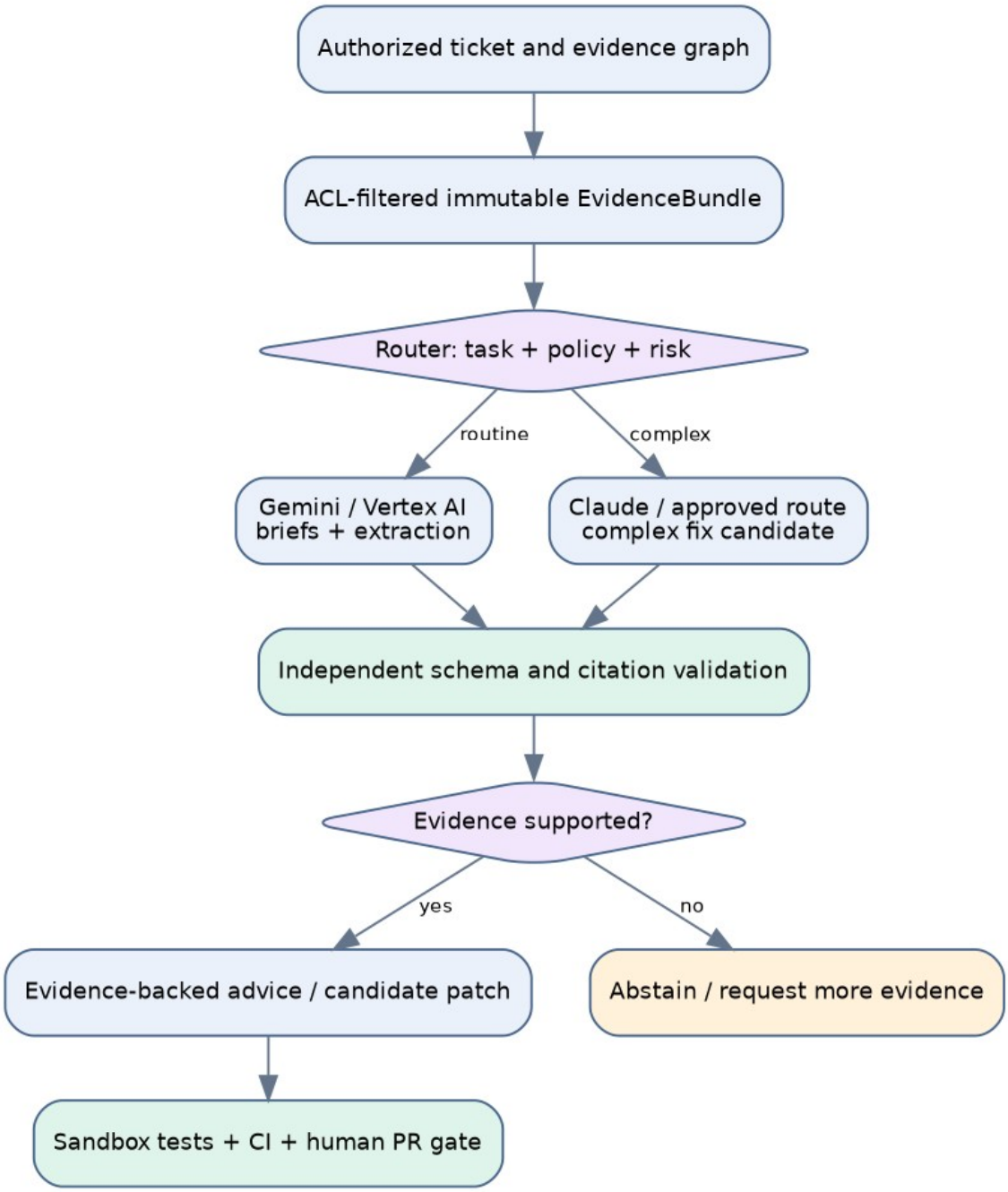


Figure 18 — Models are alternate reasoning engines; citations, tool results and approval are validated independently.

ModelProvider interface (illustrative typed contract):

```
class ModelProvider(Protocol):
  async def generate(self, task: TaskSpec, context: EvidenceBundle,
                     policy: ModelPolicy) -> ModelResult: ...

class ModelResult(BaseModel):
  provider: str; model_id: str; model_version: str | None
  prompt_template_version: str; evidence_snapshot_id: str
  output: dict; cited_evidence_ids: list[str]
  input_tokens: int; output_tokens: int; elapsed_ms: int
  invocation_id: str; status: Literal['OK','REFUSED','ERROR','TIMEOUT']
```

Routing matrix (initial experiment, not a performance claim):

Task	Primary proposal	Escalation / second provider	Deterministic controls
Ticket classification / symptom extraction	Gemini	Claude only on ambiguous or conflicting labels	Schema check; IDs and symptoms from source, not invented
Historical evidence brief	Gemini	Claude critique for evidence conflict or high-impact ticket	Every advice statement cites retrievable source and actual supporting span
Root-cause hypotheses	Gemini or Claude after quality evaluation	Opposite provider for disputed, expensive or safety-critical cases	Maintain alternatives, disconfirming facts and UNVERIFIED status
Code-fix and regression-test proposal	Claude as initial coding candidate	Gemini review only if benchmark justifies	Runner red-green checks, policy gates, human approval
Enhancement approach and interface risk	Gemini as initial candidate	Claude if cross-repo contract implications are complex	Contract/test plans tied to exact code/revision evidence
Knowledge extraction candidate	Gemini in batches	Claude only for unresolved clusters	No automatic VERIFIED lesson state; human/CI evidence decides
Advice quality / analytics	Neither by default	LLM summary optional	SQL adjudication, metrics and verified outcomes first

Example 1 — straightforward defect: Ticket "duplicate charge after retry"; retrieved graph shows prior PR-245, Confluence ADR-v7 and a test failure. Gemini receives a bounded EvidenceBundle (ticket text, source excerpts, reviewed lesson state) and returns JSON risk_areas, do, dont, best_approach, citations, alternative_causes, uncertainty. Validator rejects any unsupported citation or overstated causality. Post a ticket brief without calling Claude if evidence is sufficient.

Example 2 — complex defect to PR: A ticket reveals possible cross-project contract behavior and an old backfire. Gemini performs initial scoped history summarization; router escalates to Claude with **only** authorized exact code snippets, pinned SHAs, tests, and the evidence bundle. Claude proposes a failing regression test and candidate patch on an isolated branch. GCP test runner establishes fail-on-base/pass-on-candidate when reproducible, runs historical regression suite and builds; PR worker opens a draft PR with test status and source citations. Model critique is optional. Only native branch policy and engineer approval can complete the PR.

Example 3 — LLM disagreement: Gemini suggests retry idempotency; Claude suggests race condition. Do not average opinions or take a vote. Retrieve the disambiguating historic stack traces, transaction invariants and test evidence. Mark both hypotheses as candidates, request additional reproduction information if necessary, and abstain from a root-cause label until supporting evidence exists. Store provider disagreement as diagnostic context, not verified engineering knowledge.

Example 4 — provider error: Gemini endpoint rate-limited. For non-sensitive authorized tickets and an approved Claude route, a bounded fallback may call Claude once using the same validated schema and evidence version. For restricted source data without approved external transfers, **do not fall back** to direct Claude; return a delayed/incomplete brief and retry Vertex AI. A second model must not be used simply to conceal a failed citation check.

Router implementation rules: (1) classification=PUBLIC/INTERNAL/RESTRICTED, tenant ACL, provider allowlist and allowed regions are evaluated *before* prompt construction; (2) Pydantic validation plus evidence ID + snippet entailment review are required on every result; (3) token, dollar and wall-clock budgets are bounded per ticket and patch attempt; (4) routing configurations and model IDs are versioned and captured on each run; (5) cached results are keyed by requester entitlement/evidence snapshot, not only by text; (6) vendor errors use explicit states and an idempotent retry policy; (7) no secret/credential content goes into prompts.

Evaluation gate: On the same engineer-labeled corpus, report evidence citation validity, actionable advice correctness, defect scope recall, harmful advice rate, time-to-brief p50/p95, tokens/cost, retry rate and abstention. Compare Gemini-only, Claude-only and selective dual-model routing. Do not ship a dual-call default unless it improves a measured requirement over single-model routing.

10.10 Hybrid retrieval + graph traversal + OKF, with a worked query

Retrieval algorithm: (1) caller-scoped permissions and ticket version; (2) detect exact signals (stack trace, code symbol, affected service, trace ID); (3) parallel lexical, semantic/vector, graph-neighbor, code-path and cross-project dependency searches; (4) merge candidate IDs without prematurely discarding cross-project results; (5) rerank with recency/relevance and counterexamples; (6) fetch authorized raw evidence spans and curated OKF lessons; (7) validate chronology and evidence support; (8) assemble an immutable `EvidenceBundle` with uncertainty, provenance and version manifest. The graph *expands* known relations, while RAG finds related text; neither can assert correctness by itself.

Query example: "New defect: kiosk payment captured, barcode print failed after retry." Exact search finds `PAYMENT_CAPTURED` and endpoint method names; vector retrieval finds past similar symptoms in other projects; graph traversal identifies a shared transaction dependency and PR with a later revert; curated OKF lesson records a reviewed idempotency rule. The brief returns a ranked candidate risk area, a Do ("assert idempotency in retries"), a Don't ("repeat reverted retry-only patch"), links to the exact PR/test/ADR versions, and a warning when the printer hardware path cannot be reproduced.

Suggested storage/index structure (conceptual):

```
Cloud SQL / PostgreSQL
  source_object, source_revision, raw_payload_pointer, principal_acl
  work_item, pr_iteration, commit, file_revision, symbol_revision
  graph_node, graph_edge, evidence_claim, candidate_pattern
  canonical_lesson, lesson_revision, advice_record, advice_outcome
  embedding_chunk(vector, model_id, dimension, snapshot_id)
  ingestion_cursor, webhook_event, transactional_outbox, job_checkpoint
Cloud Storage (restricted, immutable where required)
  raw-revisions/{source}/{id}/{revision}.json
  snapshots/{snapshot-id}/manifest.json
  code-index/{repo-id}/{commit-sha}/extractor-output.json
  evidence-runs/{run-id}/test-manifest.json
  okf/lessons/{pattern-id}/lesson.md
Search
  PostgreSQL full-text/B-tree indexes + pgvector ANN where benchmarked
  In-process graph traversal over typed, authorization-filtered edge rows
```

Speed note: OKF is a governed knowledge representation, not an acceleration engine. A verified lesson may be mirrored into ordinary SQL/search indexes and delivered as a compact citation-bearing chunk. The fast path does not scan every OKF Markdown file at request time. For v1.11, select OKF only as a curated export/knowledge interchange layer until its review and publication process is validated.

10.11 End-to-end cloud processing, interfaces and operational failure modes

API or event	Producer → consumer	Minimum idempotent contract	Required failure treatment
POST /webhooks/ado	ADO → GCP ingress	Signature/credential, source event ID, tenant, payload hash, received timestamp	Reject invalid auth; enqueue once; replay from DLQ
POST /webhooks/jira	Jira → GCP ingress	Event ID, issue version, project ID, ACL scope	Never publish unavailable restricted metadata
Confluence page sync	GCP job → Confluence API	Space/page ID, version, checksum, cursor, ACL revision	Resume cursor; tombstone deletes; avoid broken pages
knowledge.reconcile	Evidence outbox → worker	tenant, source version, affected object IDs, index snapshot	Idempotent, targeted rebuild; keep last good published snapshot
POST /internal/briefs	Worker → private API	ticket revision, requester entitlement, knowledge snapshot	Return PENDING/INCOMPLETE when model/evidence unavailable
POST /internal/fix-attempts	Authorized request → runner	pinned SHA, runner digest, allowed tools, budgets, evidence bundle	No unrestricted network/secret mounts; require human PR gate
pr.completed	ADO webhook → evidence update	merge SHA, PR iteration, reviewer decision, check runs	`MERGED_UNVERIFIED` until release/outcome evidence exists

API or event	Producer → consumer	Minimum idempotent contract	Required failure treatment
deployment.observed	Approved source → postmerge worker	release SHA/environment/time, observation provenance	Correlation missing → UNKNOWN, never infer success
advice.decision	Ticket panel/comment → feedback API	advice_id, actor, revision, decision/rationale, idempotency	Separate usefulness from verified correctness

Cross-cloud boundary: Azure DevOps and Confluence receive outbound authenticated calls and may deliver signed webhooks to an ingress endpoint. Google Cloud is VisionIQ's deployment platform, not the hosting platform of those external tools. Claude direct API egress must be approved for the exact data classification; otherwise use Gemini/approved Claude via Vertex AI or abstain. ADO build/PR policies remain the authoritative merge checks even if test execution runs on Google Cloud.

Operational recovery: Pub/Sub delivery may duplicate or reorder events; store processed keys and compare source revisions before updates. Worker failures use dead-letter and replay. Embedding model updates require a parallel re-embedding namespace and atomic index version switch, not mixing incompatible dimensions. Permission revocation invalidates cached prompt/results and traversal visibility before the next response. Cloud SQL failure leaves the ticket flow in PENDING/UNAVAILABLE; do not generate unsourced recommendations from old unrestricted caches. Test-runner failure is INCONCLUSIVE, not PASS.

10.12 Technology architecture notes and implementation examples

Architecture interface examples: Google Cloud hosts VisionIQ workloads; Gemini is accessed via Vertex AI and Claude via an authorized API route; the Knowledge Base is stored in PostgreSQL with graph relationships and search indexes. Framework and adapter choices can vary without changing the domain contracts.

Practical acceptance checklist: Start one Cloud Run service and one Pub/Sub topic; capture a real ADO event idempotently; backfill work items + at least one Confluence page revision; ingest two language-distinct sample repos at file level; link one ticket→PR→commit→test chain; verify a graph traversal with source references; construct authorized hybrid retrieval; produce a Gemini evidence-backed brief and a Claude patch candidate using the same validated schema (on eligible sample data); run a safe sandbox red-green example; publish draft PR with no merge privilege; record PR merge as unverified; apply a synthetic later revert to dispute an existing lesson without duplicating it; run an ACL-denied case and a missing-data abstention case.

References verified 26 October 2026 (official docs): Cloud Run Pub/Sub/Eventarc: https://docs.cloud.google.com/run/docs/triggering/pubsub-triggers ; Cloud SQL vector embeddings: https://docs.cloud.google.com/sql/docs/postgres/generate-manage-vector-embeddings ; Vertex AI embeddings: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings ; Claude through Vertex AI: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/partner-models/use-claude ; ADO service hooks: https://learn.microsoft.com/en-us/azure/devops/service-hooks/events?view=azure-devops ; Confluence pages and versions: https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-page and https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-version ; OKF canonical repo/spec: https://github.com/GoogleCloudPlatform/open-knowledge-format .

11. Detailed non-functional acceptance and experimental design

11.1 Speed + quality together

Measurement	Initial proposed target	How to test
Webhook acknowledgement	Prompt, without LLM in request	Duplicate/burst events; durability
Indexed retrieval	Record p95 and Recall@5 jointly; target determined by corpus	Fixed labeled corpus, cross-project positive and hard negatives
Automatic brief publication	Investigate p95 ≤15 s for indexed sandbox tickets	End-to-end ADO event → published comment, multiple repetitions
Root-cause support	No verified label without supporting proof	Expert adjudication + reproduction where possible
Citation validity	Every material recommendation cites a visible supporting source or states unknown	Manual evidence-level audit and ACL test
Unsafe historical patch	Must fail a real historical regression in demo	Archived base, unsafe and corrected branches with runnable logs
Defect PR	Reviewable draft, all required tests green	CI and repository policy audit
Touched-path performance	No demonstrated regression under agreed workload	Paired repeated base/patch runs, noise interval
Enhancement	Automatic real-history brief and specific implementation guidance	Separate enhancement walkthrough
Governance	Zero auto-merges and no unauthorized code/read access	Negative permissions tests

These are targets and test obligations, **not claimed current measurements**. Do not adopt alternative designs' guaranteed `<1s`, `<2s`, 100% precision, $25k/$73k savings, 3.5× productivity or 85–90% uniqueness figures as measured facts.

11.1.1 Native integration reliability tests [REQUIRED]

- **Default coverage:** one picked-up real defect and one enhancement both create in-ticket briefs **without a VisionIQ login or manual API call**.
- **Delivery integrity:** duplicate/replayed pickup events produce no duplicate brief; a missed webhook is recovered by reconciliation; bot updates do not self-trigger.
- **Time separation:** measure ticket-to-brief latency separately from queued PR generation and CI completion; no dependency on synchronous model calls at webhook acknowledgement.
- **CI and review:** a failing test blocks claiming 'green'; a modified commit invalidates earlier check results; engineer approval is retained; bot credentials are unable to complete or merge PRs.
- **Outcome quality:** engineer approval, CI success and later deployed behavior are recorded as different facts; inaccurate advice can be corrected and traced to source revision.
- **No separate UI:** a user can complete both demonstration workflows entirely within the existing ticket, PR and CI systems; optional operations UI cannot be part of the success path.

11.1.2 Post-merge and knowledge-quality acceptance [REQUIRED FOR COMPLETE PRODUCT]

Test	Required result / proof artifact
Same defect mechanism appears in 3 tickets	One canonical pattern with three authorized observations; no duplicate lesson or unnecessary full-corpus re-embedding
A later revert invalidates an earlier	New versioned negative-evidence revision; earlier guidance flagged/superseded in

Test	Required result / proof artifact
fix	fresh briefs and Impact Agent history
Reviewer approves, PR never merges	Record approval; **do not** mark success/deployed
PR merges but has no release evidence	`MERGED_NOT_DEPLOYED` / unknown, not verified success
Deployment/CI/incident evidence conflicts	`DISPUTED` with sources and owner-review state; do not auto-adjust model confidence
Historical data access revoked	Retrieval/index/cache purge or authorization recheck per policy; old audit references do not leak source content
Impact Agent history-view integration	Actual existing Impact Agent shows authorized history, risk, source links and unknowns; no separate UI is required

Metrics include duplicate-pattern rate, evidence-linked revision accuracy, percentage of outcomes with verifiable deployment mapping, material-change indexing lag and coverage of post-merge observation. Targets require corpus benchmarks; no invented universal accuracy percentage.

11.1.3 Advice-level outcome and embedded UI acceptance [REQUIRED]

Test one brief with multiple advice IDs, including a demonstrably wrong item: only that item becomes VERIFIED_WRONG. ACCEPTED advice with a failing regression remains a separate contradicted-by-test outcome; REJECTED without independent verification remains UNVERIFIED. Verify later incidents can supersede findings without erasing audit. Show existing-ticket feedback and Impact Agent history, including an ACL denial case; no separate VisionIQ app and no auto-merge.

MVP proof: at least one accepted and one rejected/not-applicable item across the sandbox defect and enhancement tickets, with independent evidence status, authenticated feedback and native PR review. An extension panel may be deferred if comment/field feedback provides auditable per-advice tracking; Impact Agent history is still required by the official problem.

11.1.4 Knowledge Base readiness — specific executable tests

Given	When	Required assertion
A source revision page is received twice	Replay the same cursor and event ID	Exactly one canonical revision and one completed outbox job per version
A ticket has two associated PRs, one reverted	Reconstruct evidence	Both branches visible; no automatic “verified root cause” claim
A symptom appears in two projects with different causes	Search history from an authorized ticket	Both candidates retrievable; conflicting causes remain separate
A source’s permissions are revoked	Reuse cached brief/retrieve direct link	No unauthorized snippet, source count or citation disclosed
An LLM-readable ticket contains malicious tool instructions	Generate evidence brief	Instructions remain inert data; no tool/data exfiltration
An incremental update changes an incident outcome	Reindex/rebrief	New snapshot references revised lesson; prior revision remains auditable
A bad index build or connector outage occurs	Retry or roll back	Last consistent permitted snapshot served with freshness warning, or fail closed

Definition of passed: for each test, save source fixture/revision, input event, run/snapshot ID, database assertions, retrieval response and observable logs. False or missing links are recorded as quality failures, not hidden by a total “documents ingested” count.

11.2 Evaluation dataset and baseline

Collect a permissioned benchmark with historical resolved/unresolved incidents, links validated by engineers, **same-symptom different-cause negatives**, cross-project positives, old and recent releases, and enhancements

with real compatibility concerns. Split by time/project to avoid leakage and keep held-out cases unobserved during tuning. Two engineers independently label relevant historical items and root-cause status, resolving disagreements through review. Compare (a) ordinary current-ticket/code-only analysis versus (b) VisionIQ with the Historical Intelligence Engine. Measure evidence retrieval quality, review time, diagnosis correctness, regression catches, abstention and failures; disclose sample size and confidence intervals when meaningful.

11.2.1 Point-in-time backtesting — "what VisionIQ could have known then"

To avoid evaluating on future information, replay 30–50 representative resolved defects (initial suggested sample, extend as data permits). For each historical ticket T created at time $t0$, reconstruct a **time-consistent knowledge snapshot** containing only source revisions, commits, PRs, decisions, deployment evidence and cross-project relationships observable by $t0$ to that authorized engineer. $observed_at$ matters: a retroactively edited ticket or imported document published later cannot enter an earlier snapshot merely because it carries an older business date.

Run **the same brief-generation pipeline** as the live ticket at snapshot $t0$, then compare its predicted areas and retrieved history with engineer-reviewed outcomes observed *after* $t0$. Report $Recall@K$, $Precision@K$, cross-project recall, correct component/module localization, grounding correctness, advice correctness/abstention coverage, latency and false positives. Keep code snapshots at the corresponding commit SHA; avoid embeddings indexed from later ticket comments, repaired tickets, or tests written after the defect was fixed. Separate test sets by time and by project to expose leakage and generalization limitations.

Evidence package per replay: ticket ID, as-of timestamp, source snapshot manifest/hash, user authorization scope, model/prompt and index versions, ranked evidence with source creation/observation times, output brief, reviewer ground truth, comparison metrics and failure notes. Include difficult negatives: same symptom with different root causes, obsolete versions, missing PR links and legitimate cross-project similarities. No one may tune retrieval weights using the held-out test set and then report those same cases as independent results.

Acceptance: a reviewer can reproduce the replay and show that a future fix or post-incident explanation is absent from $t0$ retrieval; the system can abstain when adequate past evidence did not exist; speed/quality claims are based on measured held-out runs rather than synthetic percentages.

11.2.2 Implementable point-in-time evaluation and decision thresholds

Start small without weakening historical correctness. A first evaluation can select 15–30 engineer-reviewed tickets (expand to 30–50 after the pipeline works), build an ephemeral snapshot/index from only records **observable as of each ticket's pickup timestamp**, execute the existing fast Brief path, and preserve the replay manifest. Snapshot construction may use timestamp-filtered **revisions**, temporal joins and an index built for the cut-off; it does **not** inherently require storing embeddings for every historical commit. Merely filtering on ticket $created_at$ is unsafe: later comments, state changes, source ACLs and linked fixes can leak future knowledge.

For an initial MVP, restrict the fixture to a known set of projects and source types, retain raw source revisions and $observed_at/effective_at$, exclude future test artifacts and record authorization scope used for evaluation. If historical ACL reconstruction is not available, replay using a present-day scope **intersected with source visibility**, clearly note the limitation and never claim the replay reproduces historical permission state. Don't silently weaken cross-project ACL checks.

Measurement	Definition and decision treatment
Recall@10, Precision@5, cross-project Recall@K	Evaluate against independently reviewed relevant-history labels; include same-symptom/different-cause negatives
Evidence grounding	Every displayed citation must support the specific claim, not merely exist as an ID
Component localization	Whether relevant file/module/service appears in the predicted top-K, with version and coverage scope
Brief p95	Pickup event to published in-ticket brief, plus separate indexed-retrieval p95
Abstention / unsupported advice	Measure both wrong confident claims and correctly withheld claims; report coverage

Measurement	Definition and decision treatment
Red-green and performance	Record baseline failure signature, candidate pass, actual historical replay, and benchmark variability

Do not treat 70% recall, 80% precision, 20% flaky tolerance, or < $0.50/brief as facts. The team should set tentative release gates after measuring a baseline, publishing sample sizes, and agreeing which failures are unacceptable. The official demo has unconditional behavioral gates (two briefs, real defect PR, green required tests, human gate, embedded Impact Agent history); measured quality targets are a distinct technical decision. Protect a held-out sample against tuning leakage.

11.3 Build-quality thresholds and exit criteria

A demo is not accepted merely because a UI says “tests passed.” It must preserve branch SHA, CI run URL, console logs, test summary, evidence citations, patch diff, test fixture and version information. **No auto-merge** is independently validated by permissions. Failure Memory Firewall is convincing only if the unsafe historical case is actually executed and observed failing; there must be at least one safe candidate that passes the agreed checks, or a truthful explanation of a blocked case.

12. Development plan and definition of done

12.1 Proof-first implementation gates

Gate	Build focus	Demonstrable exit condition
G0 — access and fixtures	Repository/history access **user-confirmed**; inventory source scopes, ticket revisions, one known fix/revert, runnable tests, PR bot credentials and Impact Agent owner	Authorized source inventory and reproducible fixture; remaining permissions and execution conditions verified
G1 — trustworthy history	Recoverable source backfill and incremental sync, canonical evidence revisions, link resolver, permission-aware hybrid retrieval	Reconciled backfill, idempotent replay, a sourced cross-project match, a documented false-positive refusal and an ACL revocation test
G2 — always-on Learn + automatic Brief	Work-item webhook, queue/idempotency, scheduled hotspot, evidence briefs	Defect **and** enhancement both receive real-history auto briefs
G3 — defect PR and human gate	Scoped code change, isolated runner, actual CI, draft PR/review	Green tests, traceable PR, bot cannot merge
G4 — signature prevention proof	Recovered old failure, unsafe/safe candidate comparison	Executable historical regression catches unsafe candidate
G5 — measurement + flagship	Benchmarks, labels, observability, Impact Agent contract	Measured speed/accuracy, documented limits, approved integration if available

Full hackathon DoD: all official phase deliverables; tested defect and enhancement; real sandbox historical links; one approved draft PR, green tests and no demonstrated performance regression; human gate; hotspot evidence and correct/right-wrong feedback; Impact Agent historical view where contract/access is actually supplied. If the flagship interface is absent, report its status explicitly as an unmet dependency, not a completed feature.

Additional internal proof — G6 (post-merge and dedup): demonstrate repeated tickets linking to one known pattern, a later revert changing the lesson, and an observed merge/release outcome that remains *unknown* if deployment evidence is missing. This is a product-quality gate and should not delay proving the organizer's four official phases on the first sandbox pair. The Impact Agent history view remains a **separate mandatory acceptance item**, not a substitute for G6.

12.2 Suggested developer workstreams

Workstream A — integration and canonical evidence: ADO/Git connectors, source inventory, historical backfill, incremental checkpointing, ACL/revocation, canonical revisions, link resolver, index publication, hotspots and sync observability.
Workstream B — intelligence and quality: hybrid retrieval, evidence bundle, citation validator, confidence policy, defect/enhancement briefs, evaluation fixtures.
Workstream C — safe change execution: branch/PR integration, runner/test adapter, historical regression case, performance harness, human gate and audit.
Shared: acceptance tests, architecture decisions, security/Impact Agent access, observability and integration checks.

12.3 Priorities for the MVP

Must implement: ticket trigger, Learn ingestion/hotspot sweep, source-aware retrieval, evidence-labeled brief for both ticket kinds, defect patch/tests/draft PR, restricted runner, no-merge gate, feedback log, one historical failure catch, benchmark reporting, Impact Agent contract investigation.
Useful only if core is stable: cross-project UI/timeline, AST parser for an additional language, policy-rule UI, SonarQube, additional Jira connector, simple dependent-team notification behind reviewer consent.
Defer: speculative enterprise-wide forecasting, automatic Confluence edits, guaranteed self-improving confidence scores, general support for every language, autonomous multi-repo patches, automatic Git bisect for every ticket, several independent vector/index platforms, expensive cloud stack without evidence.

12.4 Practical starting sequence — KB first, not 31 separate features

Foundation sequence: (1) capture actual ADO ticket types, source access and sample link structures; (2) create canonical revision/cursor/edge migrations; (3) ingest and replay one project; (4) connect 2–3 related projects, label a small benchmark and build lexical search; (5) add vector/symbol/dependency candidate paths only when measured benefits justify them; (6) publish evidence-bounded briefs; (7) connect CI, sandbox, PR and human gate; (8) validate post-merge observations and selective lesson updates. Track results in the same master ADR/acceptance register, not in a separate conflicting design document.

Ownership proposal for team review: engineer A owns connectors/canonical evidence; engineer B owns hybrid retrieval/evaluation; engineer C owns PR/CI safety and Impact Agent extension investigation. Every workstream participates in fixture quality, code review and security tests. Actual allocation depends on team size and skills. Begin implementation against real **authorized sample histories** without waiting to process every ticket in the enterprise.

13. Issues, challenges and risk register

ID / Severity	Challenge	Failure mode	Required response
R01 / Critical	Missing/incorrect historical causes	Incorrect “same defect” diagnosis	Treat source cause as hypothesis; contradictions and reproduction
R02 / Critical	Missing work-item ↔ code links	Misattribute a patch	Explicit/inferred edge types; review link recall on known defects
R03 / Critical	Cross-project authorization	Unauthorized disclosure	Filter at index, retrieval and citation; negative access tests
R04 / High	Cross-project recall loss	Hidden real fix excluded by early filter	Multi-lane candidate union and held-out cross-project tests
R05 / High	Confidence overclaim	Engineer trusts fake 95%	No uncalibrated percentage; independent ground truth
R06 / High	Speed versus recall	Fast narrow search misses cause	Measure latency and Recall@K jointly
R07 / High	Wrong deployed version	Current `main` misleads	Branch/build/environment-aware match labels
R08 / Critical	Unsafe generated code	Malicious or destructive runner step	Isolate, restrict egress/secrets, policy checks, audit
R09 / Critical	Old regression repeats	New tests too narrow	Targeted historical failure execution and documented gaps
R10 / High	Flaky benchmarks/hardware	False performance pass/fail	Repeated paired tests and inconclusive outcome when needed
R11 / High	Prompt injection in sources	Tool misuse and fabricated claims	Untrusted content boundaries and structured validation
R12 / High	Index staleness/revocation	Wrong/unauthorized output	ACL refresh, revisions, invalidation and snapshot links
R13 / High	Workflow retries and loops	Duplicate PRs/briefs	Durable idempotency, origin filters and state-machine tests
R14 / High	Feedback bias	Approved suggestion labeled “correct”	Track CI, reverts, regressions and adjudication separately
R15 / High	Model latency, quota and expense	Brief too slow or fails	Fast/deep split, limited context, partial outputs and quotas
R16 / High	Scope explosion	Many weak features, incomplete proof	Vertical slice and explicit P0/P1/P2 gatekeeping
R17 / High	Impact Agent unknown	Required integration unfulfilled	Contract discovery and owner sign-off in G0
R18 / High	Enterprise data-size assumptions	Overbuilt platform or bad cost	Benchmark actual accessible corpus before database choice
R19 / Medium	Documentation drift	Old rules override new behavior	Versioned rules, owner approval, drift suggestion
R20 / High	Enhancement path underbuilt	Only defect demo passes	Separate enhancement acceptance test
R21 / High	Transport payment/hardware risk	Duplicate transaction/ticket, unavailable kiosk	Business invariants and simulator/real-hardware limitations
R22 / High	Unsupported ROI/uniqueness claims	Mislead reviewers	Keep observed data distinct from scenarios
R23 / High	Product quality under uncertainty	“No evidence” shown as confident guidance	Abstention with explicit next evidence request

ID / Severity	Challenge	Failure mode	Required response
R24 / Critical	Stale or partially ingested knowledge base	Missed incidents, obsolete fixes or misleading briefs	Source watermarks, completion state, index snapshot age and reconciliation
R25 / High	Loss of source/derived distinction	An inferred edge becomes an asserted root cause	Immutable provenance, edge status, curator review and contradictory evidence tests
R26 / Critical	Data permission revoked after indexing	Historical snippets or embeddings leak across projects	ACL-aware indexing, immediate invalidation and negative permission tests
R27 / High	Backfill failures and duplicate events	Missing/duplicate history and feedback pollution	Checkpointing, idempotency, source count reconciliation and replay tests

New risks requiring assigned owners: duplicate pattern proliferation or feedback loops from repeated tickets; inference that an approved PR is automatically correct; inability to map merged commits to deployed versions; late or contradictory incident evidence; overfitting retrieval ranking to accepted fixes; and historical evidence leaking through the Impact Agent if authorization is not propagated. Mitigations are versioned canonical patterns, independent outcome states, event idempotency, scope-aware caching, owner adjudication and negative test cases in §11.1.2.

13.1 Strong corrections from the external reviews

- The P4 Detailed Design's “100% architecture decided” and pivot-on-85%-linkage logic are not engineering evidence. The former is premature; the latter confuses uncertain linkage coverage with whole-project impossibility.
- The P4 sample BigQuery schema uses SQL patterns that cannot be presumed deployable (e.g., bare `TEXT`, `PRIMARY KEY ... CLUSTER BY` in the shown arrangement). If BigQuery is selected, write correct dialect DDL and run migrations in a test project.
- The P4 documents alternate between Gemini and Claude as “locked” choices and give differing SLA/cost/ROI claims; these require experimental selection rather than inherited assertions.
- The VisionIQ Whitepaper's “100% mathematically proven” and “zero-latency”, “O(1) exact blast radius” claims are technically unsound. Dependency traversal and correctness are workload- and evidence-dependent.
- The Whitepaper's “automatic Confluence correction” must instead create a reviewer-approved change proposal. A deterministic state graph does not certify that model output is correct.
- AST parsing and typed policy validation are promising real additions, while broad vendor/service selections need separate ADRs.

13.2 Native-pipeline integration risks [REQUIRED]

Risk	Consequence	Validation / mitigation
Work-item rule only fires on assignment, but teams use state transitions differently	Some tickets never receive briefs	Per-project pickup policy, integration tests and periodic source reconciliation
Replayed hooks, bot-generated edits or updated revisions	Duplicate briefs or recursive jobs	Signed events, revision/policy idempotency key, origin filter and deduplicated reconciliation
Separate UI becomes the only way to see guidance	Violates organizer's 'default step, not a side tool'	Native ticket comment/field is P0; UI remains optional diagnostics
Checks apply to old SHA or private runner only	Unsafe PR incorrectly shown as green	Pin results to latest patch SHA; require native CI status for PR decisions
Bot token can complete PR or	Violates the human gate	Least-privilege bot and

Risk	Consequence	Validation / mitigation
bypass policies		negative permission test in sandbox
Impact Agent adapter built without owner contract	False impression of completed integration	Contract/sign-off gate, owner-scoped integration test; report as open until verified

14. Architecture decision records and sign-off

ADR	Decision to make	Candidate choices	Required experiment	Owner / status
ADR-01	Canonical storage/index	PostgreSQL+pgvector; BigQuery+Vertex; SQL+Qdrant	Authorized corpus + retrieval & ACL benchmarks	**Unassigned / OPEN**
ADR-02	Orchestration	Queue + explicit states; ADK	Failure recovery and implementation complexity	**Unassigned / OPEN**
ADR-03	LLM provider	Approved Claude/Gemini/other	Same tasks, security, latency, cost, quality	**Unassigned / OPEN**
ADR-04	Sandbox	CI runner / restricted Cloud Run workload	No prod secret, reproducible tests/perf	**Unassigned / OPEN**
ADR-05	Language-neutral code intelligence	Pluggable approved parser/build/test adapters; file-level fallback; no Roslyn dependency	Same contracts work with supported and unsupported/mixed-language repos without false claims	Language-agnostic boundary DECIDED; adapter selection OPEN
ADR-06	Rules format	Versioned YAML/Markdown, optional OKF	Enforceability of concrete policy violations	**Unassigned / OPEN**
ADR-07	Ticket/PR adapter	ADO source hook/comment/draft PR	Actual end-to-end API + permissions	**Unassigned / OPEN**
ADR-08	Impact Agent integration	Flagship-owned history contract	Owner demo, auth and payload compatibility	**Unassigned / OPEN**
ADR-09	SLO thresholds	Balanced retrieval recall/latency budget	Representative ticket corpus and CI timings	**Unassigned / OPEN**

ADR-LANG-01 — Language-neutral repository support: VisionIQ is not dependent on a specific programming language or framework. Evidence, relationships, retrieval and ticket briefs use common contracts; code parsing and test execution are pluggable. Unsupported languages fall back to cited file-level history, without invented symbol or green-test results. No .NET-only or Roslyn dependency in the core platform.

ADR-PIPE-01 — Native SDLC pipeline integration [REQUIRED / decided principle]: VisionIQ is a background service integrated into work-item pickup, native ticket briefs, the existing Git/PR/CI pipeline and engineer approvals. A separately opened UI is not a required user surface. Exact ADO/Jira project trigger rules, webhook payload schemas, CI status interface and the flagship Impact Agent contract remain environment-specific design decisions. This ADR is not evidence that those integrations have already been implemented.

ADR-LEARN-01 — Non-repetitive pattern learning [REQUIRED / decided principle]: raw source evidence is append-only and every event is recorded idempotently, but reusable lessons are deduplicated and revised only on material validated change. Engineer acceptance remains separate from correctness. Owner-curated corrections supersede, rather than erase, past advice.

ADR-OBS-01 — Post-merge outcome observation [REQUIRED FOR COMPLETE PRODUCT / proposed integration]: follow authorized merge, release, revert and incident evidence; model deployment status explicitly; do not infer deployment or success from PR approval alone. Concrete data feeds, supported environments and observation windows require team validation.

ADR-IMPACT-01 — Flagship Impact Agent extension [GIVEN / behavior decided, interface OPEN]: provide an authorized historical evidence view within the existing flagship Impact Agent through its owner-approved extension point. Do not create a mandatory new user interface or mark integration complete until demonstrated in the actual Impact Agent.

ADR-GCP-01: All VisionIQ-owned compute, storage, queues, observability and deployment run on Google Cloud; ADO/Jira/Confluence remain external sources. ADR-LLM-01 : Gemini through Vertex AI and Claude API via approval-aware adapter. ADR-KG-01 [SELECTED MVP]: temporal typed graph in Cloud SQL PostgreSQL. ADR-OKF-01 [CURATED]: reviewed OKF-compatible lessons, not database replacement. ADR-PR-LEARN-01 [REQUIRED]: every PR revision updates evidence; only substantiated change updates canonical lessons.

14.1 Design sign-off rules

The team can begin **G0/G1 discovery and foundation code**, but should not approve a full implementation schedule or advertise enterprise-grade outcomes until:

- The available repository/history corpus is inventoried; source ACL policies and automation authorization are verified separately.
- At least one complete ticket→PR→commit/test chain is inspected, with fallback for broken links.
- One reproducible defect, one enhancement and one historical regression fixture are confirmed.
- Database/index, runner, model provider and orchestration ADRs are reviewed with measured trade-offs.
- Required no-merge and runner isolation policy is tested.
- The owner and supported integration interface of the flagship Impact Agent are confirmed and an actual embedded history view is tested; otherwise mark the mandatory requirement **not yet proven**.
- The post-merge event feeds, release SHA mapping and canonical pattern deduplication can be demonstrated using at least one verified example each.
- An engineering owner signs P0 deliverables, latency/quality methodology, cost constraints and open risks.

14.2 Decision log template

Date	Decision / experiment	Evidence link	Owner	Outcome	Revisit trigger
2026-10-26	Repo/history access reported available; scope and fixture inventory still required	User confirmation / pending audit	[assign]	Access confirmed, quality open	Incorrect or missing links
[fill]	ADR-01 corpus search benchmark	[report]	[name]	Open	Recall/latency insufficient
[fill]	G4 historical regression rehearsal	[CI run]	[name]	Open	Test not reproducible

14.3 Knowledge Base decisions to approve before scaling

Decision	Proposed default	Evidence required to finalize
Canonical DB	PostgreSQL with migrations and append-only evidence revisions	Actual source volume, concurrency, audit and ACL tests
Search engine	PostgreSQL lexical + optional pgvector for MVP; revisit specialized search if warranted	Hybrid Recall@10 and p95 latency on held-out corpus
Graph storage	Typed relational edge table initially	Cross-project traversal and query-cost measurements
Raw historical payloads	Restricted object storage, immutable revision pointers, redaction process	Enterprise data approval and retention requirements
Incremental feed	ADO reporting revisions / hooks + periodic reconciliation	Organization-specific API test, permissions and cursor recovery
Knowledge curation	Candidate lessons automatically; approved rules by named human owner	Sample disputed/root-cause cases and approval workflow
Snapshot publication	Immutable manifest plus transactional outbox and atomic active-pointer switch	Simulated partial failure, rollback and permission change tests

A decision remains **OPEN** until a named engineer records the selected option, experiment, observed result, security approval and rollback/revisit trigger. Only the official four-phase behavior, human gate and no-side-tool requirement are treated as settled product obligations.

15. Source reconciliation: what stays, what changes

Source	What we retain	What we modify or remove
Engineering Blueprint v1.2	Detailed organizer alignment, HIE, evidence labels, 31-capability inventory, four phases, speed/accuracy, Failure Memory Firewall, challenges, Impact Agent concern	Merge overlapping feature subsections into capability groups; retain original requirement verbatim; replace generic "architecture complete" impression with concrete gate/ADR status
P4 Detailed Design v3.0	Stepwise ingestion/retrieval example, ADO draft-PR and comment approach, test-gap ideas, hotspot examples, language adapters, release validation plan, API contracts	Remove unsupported 50 projects/10k actual corpus, false 85–90% uniqueness, blanket confidence formula, 1-second "guaranteed" retrieval, unrealistic ROI, flawed DDL, auto-pivot and premature technology locks
VisionIQ Architectural Whitepaper	Event-driven non-blocking architecture, AST/Tree-sitter, versioned rule guardrails, isolated tests, Git-bisect as optional targeted tool, telemetry context	Remove 100% mathematical accuracy, zero-latency, O(1) exact impact, arbitrary 1% slowdown hard-stop, absolute no-hallucination claim and automatic authoritative wiki rewrite
Independent Architecture Review	Explicit critical findings, adopt/adapt/reject reasoning, assurance that similarity/approval aren't truth	Use as review input integrated here, not a parallel master document
Technical System Design v0.1	Contracts, data model, event types, fast/deep paths, security, feasibility gates	Harmonized into this master and corrected where needed; obsolete as a competing implementation source

Colleague's Hackathon Analysis — adopted as design enhancements: point-in-time historical backtesting, explainable risk factors and red-green defect verification. The .NET-specific Roslyn proposal was deliberately generalized into a language-neutral adapter contract; SZZ and automatic backfire hints remain hypotheses, not verified causes.

16. Independent validation assessment and release gates

The purpose of this section is **team decision-making**, not a claim of production verification. The official problem is completely mapped at design level; integration and quality are only `VALIDATED` when linked evidence exists. The validation record tracks `owner`, `status`, `proof`, `decision_date` and `blocker_reason`; unresolved concerns never become green merely by appearing in a document.

Gate	Required evidence	Release decision
Organizer workflow	Real defect and enhancement, automatic briefs on original tickets, evidence links, defect PR with passing required tests, actual engineer approval, no auto-merge	Non-negotiable hackathon demonstration
Impact Agent	Historical risk and evidence visible in the existing flagship experience, scoped to authorized identity	Explicit organizer requirement; cannot silently descope
Knowledge integrity	Source backfill + incremental sync, source	Must pass for credible historical intelligence

Gate	Required evidence	Release decision
	revisions, verified ticket/PR/test links, duplicate-pattern reuse, ACL-negative tests	
Retrieval evaluation	Held-out relevance labels; Recall@K, precision, grounding, abstention, cross-project cases, latency	Thresholds proposed only after baseline measurement
Fix safety	Pinned SHAs, matching baseline failure, passing candidate, relevant historical regression checks, CI, touched-path performance evidence	Missing required evidence is INCONCLUSIVE, not pass
Observability and learning	Advice IDs, engineer disposition vs correctness, merge/deployment status where available, versioned knowledge updates	Must show no repetitive relearning or false positive success
Security	Authorized source scopes, malicious ticket content, secret/egress restrictions, no policy bypass, audit	Fail closed on sensitive authorization uncertainty

Current status: specification **ready for team review and parallel P0 prototypes**. Not signed off as a universally working implementation. Actual data and repository access are user-confirmed, while linkage quality, Impact Agent extension mechanism, runner feasibility and quality outcomes are **not yet measured or contractually confirmed**.

17. Proof-first development and validation roadmap

Do not impose an unsupported four-to-six-week freeze or dollar estimate. Prepare the minimum vertical slice and validate critical assumptions **alongside** implementation, using the team's actual time box and capacity.

Work package	First deliverable	Dependency / handoff
A — Knowledge foundation	Recoverable ADO/Git backfill, revisions, provenance, ACL, incremental sync, canonical patterns, hybrid index	Provides evidence bundle for Brief; must run on real authorized data
B — Search and evaluation	Multi-path search, evidence validator, cross-project test cases, point-in-time replay, latency instrumentation	Uses A; fixes precision/recall and trust bottlenecks early
C — Native ticket + Impact Agent	Event trigger, automatic brief on original ticket, authenticated historical-view adapter in flagship	Runs parallel to A; confirm owned extension surface and policy
D — Verified defect PR	Language-neutral approved runner for demo repo, reproduction signature, red-green test, regression discovery, CI, draft PR and human gate	Requires sample real defect and runnable isolated tests
E — Enhancement and outcomes	Enhancement guidance on original ticket, atomic advice feedback, post-merge/correction event links	Required for complete workflow; no new developer-facing phase

First milestones: (1) select one real fixed defect with recoverable history and executable test plus one real enhancement; (2) get ticket-to-code evidence into the Knowledge Base and review five known links; (3) publish an automatic brief for both; (4) implement the draft defect PR and show required CI green with human review; (5) demonstrate one recovered regression or clearly report its absence; (6) measure quality and write the next decision log. These can be planned in parallel; dates and owners are set by the actual team, not inferred from an AI-generated project plan.

18. External review consolidation — decisions and rejected assumptions

Preserved from v1.8: complete core design, four-phase organizer traceability, language-neutral evidence/adapter architecture, Knowledge Base specification, hybrid search, Impact Agent requirement, continuous learning, 13 diagrams and original acceptance tests. The other AI's uploaded v1.8.1 COMPLETE PDF contained a placeholder for §§1–15; it is **not used as the master source**.

External suggestion	Consolidation decision	Rationale
Three-tier historical test discovery	ADOPT with qualification	Explicit evidence first; inferred relationships remain unverified
Runner/environment manifest and failure-signature matching	ADOPT	Necessary to establish meaningful red-green behavior
Flaky test retry handling	MODIFY	Bounded reproducibility policy; do not approve a 20%-flaky gate
Adapter failure recovery and versioning	ADOPT	Atomic publication, schema validation and last-good snapshot
Patch scope limits	MODIFY	Risk and authorization based; arbitrary file/line caps are not a safety proof
Time-filtered 'MVP-Lite' replay	MODIFY	Use as-of revisions, not created_at alone; no future evidence leakage
Python-only adapters	REJECT as default	Language-neutral platform; first build/test adapter follows the actual sandbox repo
Fixed percentages, cost savings and 4–6-week pre-development hold	REJECT as factual claims	No measured corpus or staffing basis
Defer Impact Agent from hackathon	REJECT	Contradicts the original WIN condition

19. Engineering integration reference and change management

Decide now: Impact Agent interface/owner; event triggers and actor privileges; selected demo repo and approved test runner; source-data authorization and model egress; test performance workload; first supported adapter; experimental quality targets and held-out cases. Use the single ADR register in §14 for owner, choice, alternative rejected, evidence and change-control date.

Do not create another competing specification. This v1.9 file is the **consolidated development candidate**; subsequent edits should amend this master after concrete review findings or prototype evidence. Feature scope is frozen for the hackathon unless a change is necessary to meet an official requirement or fix a verified technical blocker. A signature from the team indicates acceptance of the current **design and known risks**, not proof of operating accuracy.

Appendix A — Organizer Problem Statement 4 (source-preserving reproduction)

THE PROBLEM — Pillar: T&M – Transport Ops
Every new defect or enhancement starts from scratch. Nobody checks what broke in that area before, why, or which fixes backfired — so the same defects recur, risky areas stay risky, and the lessons live only in people's heads.

WHAT GETS BUILT
Runs by default the moment a defect or enhancement is picked up — history in, a brief for every ticket, a PR for every defect, a human at the end.

1 · LEARN — ALWAYS ON
Mines defect tickets, incidents and the commits that fixed them. Names the root cause behind each recurring pattern and maps it onto the repo and architecture. Also sweeps periodically to surface hot spots before tickets arrive.

2 · BRIEF — EVERY TICKET
Lands automatically on the ticket: Risk areas — the modules, services and interfaces this change touches. Do / don't — from past fixes, regressions and fixes that backfired. Best approach — and why, with evidence from history.

3a · DEFECT → FIX TO PR
Proposes the root-cause fix in the right layer, plus the tests to prove it. Raises a PR with the evidence trail: defect → root cause → risk → fix.

3b · ENHANCEMENT → GUIDED BUILD
What to be aware of, what to do and not do, and the best way to build it — optionally a starter PR the developer takes forward.

HUMAN GATE
Engineer decides. Approve or discard the PR; accept or adjust the guidance. Nothing merges on its own. Every decision feeds back into Learn.

WIN
On a sandboxed real repo, pick up a live defect and a live enhancement: each gets an automatic brief (risk areas, do/don't, best approach) drawn from real history, and the defect ends as a PR an engineer approves — tests green. A default step on every ticket, not a side tool. AI proposes, engineers decide — no auto-merge. Adds the history view to the flagship's Impact Agent.

QUALITY & PERFORMANCE BAR
A fix must not slow the code or reopen old bugs: no performance regression on the touched path, and the brief's advice tracked for right vs wrong.

The above preserves the substantive official wording provided in the conversation; typography and paragraph breaks are normalized.

Appendix B — Reference sources and glossary

Internal review sources: `Defect_Intelligence_Engineering_Blueprint_v1_2.md`; `P4_Detailed_Design.md`; `VisionIQ - Architectural Whitepaper.pdf`; `VisionIQ_Independent_Architecture_Review_v1.md`; `VisionIQ_Technical_System_Design_v0_1.md`.

External integration verification (26 October 2026): Microsoft documentation confirms Azure DevOps `workitem.updated` service hook events, work-item comment creation, and Git pull-request creation with a draft flag. API versions, identity scopes, required permissions, branch protections and responses must still be tested against the team's actual Azure DevOps environment. Cloud runner isolation, model-provider data policy, optional integrations and the Impact Agent interface remain implementation investigations.

Knowledge-sync verification: Microsoft Azure DevOps Reporting Work Item Revisions describes nextLink / isLastBatch; Work Items batching documents a maximum of 200 IDs per call; WIQL supports scoped source inventory (Azure DevOps REST API 7.1). Confirm field mappings, auth and data retention in the actual organization.

Glossary: OKF Open Knowledge Format; **ADK** Agent Development Kit; **HIE** Historical Intelligence Engine; **EvidenceBundle** bounded set of provenance-checked context for one ticket; **AST** Abstract Syntax Tree; **RAG** retrieval-augmented generation; **ACL** access control list; **MRR/nDCG** ranking-quality measures; **p95** 95th percentile of measured latency; **ADR** architectural decision record; **DoD** definition of done; **G0–G5** proof-first delivery gates; **abstention** explicit refusal to overclaim when evidence is insufficient; **regression** reappearance of previously working behavior failure; **patch candidate** isolated unmerged proposed code change.

Appendix C — Revision history

1.10 — 26 October 2026	Selected GCP deployment and Gemini/Claude routing; typed temporal knowledge graph, language-agnostic code ingest, Confluence + curated OKF, PR event evidence sync and selective verified learning; five added diagrams and build tests.
Version	**Summary**
1.9 — 26 October 2026	External review consolidation: test evidence discovery, reproducible runners, adapter integrity, risk-based patches, temporal replay and sign-off gates.
1.7 — 26 October 2026	Detailed knowledge-base implementation spec: ingestion/cursor contract, canonical schema, link confidence semantics, indexing and authorization, deduplication, executable exit gates.
1.6 — 26 October 2026	Added per-advice outcome identity, independent disposition/correctness states, embedded feedback UI, data contracts and required acceptance tests.
1.5 — 26 October 2026	Promoted non-repetitive knowledge reuse and post-merge observation to explicit engineering capabilities; defined outcome states, pattern consolidation, selective updates, stronger Impact Agent interface explanation and mandatory embedded-view acceptance checks. Final team-review draft.
1.4 — 26 October 2026	Confirmed native SDLC integration as required; expanded ticket triggers, source-native briefs, CI/PR checks, review ownership, idempotency/reconciliation, risk and acceptance tests; added pipeline sequence and CI gate diagrams.
1.3 — 26 October 2026	Removed the dated verification note; expanded §6 with knowledge-base creation, canonical evidence lifecycle, curation, governance, reliability and build acceptance tests; updated development gates and risks.
1.2	Consolidated technology roles (RAG, OKF, ADK), platform options and improved reading layout.
1.8 — 26 October 2026	Colleague proposal integration: point-in-time backtesting, explainable risk and red-green testing; repository-agnostic code intelligence, no Roslyn dependence.

End of VisionIQ Master Engineering & System Design v1.11 — consolidated team sign-off candidate.